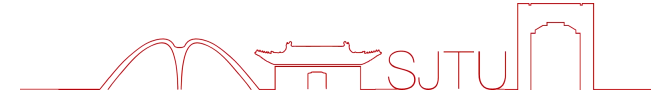




上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Federated Learning (FL) Lab



饮水思源 · 爱国荣校



1

Our simulation platform

2

Hands-on algorithms

3

Advanced algorithms

4

Other algorithms

01

Our simulation platform

- Overview
- Dataset generation
- System introduction
- FedAvg example



Our simulation platform

Overview

- Currently, 1.3k stars, 277 forks
- 37 algorithms, 14 famous datasets + 3 IoT datasets +3 Cross-domain datasets
- Record the GPU memory usage for the model
- Differential privacy

 [TsingZ0](#) / [PFLlib](#) Public

 Notifications

 Fork 277

 Star 1.3k



Our simulation platform

Baselines

Traditional FL (tFL)

- FedAvg — [Communication-Efficient Learning of Deep Networks from Decentralized Data](#) AISTATS 2017

Update-correction-based tFL

- SCAFFOLD - [SCAFFOLD: Stochastic Controlled Averaging for Federated Learning](#) ICML 2020

Regularization-based tFL

- FedProx — [Federated Optimization in Heterogeneous Networks](#) MLsys 2020

- FedDyn — [Federated Learning Based on Dynamic Regularization](#) ICLR 2021

Model-splitting-based tFL

- MOON — [Model-Contrastive Federated Learning](#) CVPR 2021

- FedLC — [Federated Learning With Label Distribution Skew via Logits Calibration](#) ICML 2022

Knowledge-distillation-based tFL

- FedGen — [Data-Free Knowledge Distillation for Heterogeneous Federated Learning](#) ICML 2021

- FedNTD — [Preservation of the Global Knowledge by Not-True Distillation in Federated Learning](#) NeurIPS 2022





Our simulation platform

Baselines

Personalized FL (pFL)

- FedMTL (not MOCHA) — [Federated multi-task learning](#) *NeurIPS 2017*
- FedBN — [FedBN: Federated Learning on non-IID Features via Local Batch Normalization](#) *ICLR 2021*

Meta-learning-based pFL

- Per-FedAvg — [Personalized Federated Learning with Theoretical Guarantees: A Model-Agnostic Meta-Learning Approach](#) *NeurIPS 2020*

Regularization-based pFL

- pFedMe — [Personalized Federated Learning with Moreau Envelopes](#) *NeurIPS 2020*
- Ditto — [Ditto: Fair and robust federated learning through personalization](#) *ICML 2021*

Personalized-aggregation-based pFL

- APFL — [Adaptive Personalized Federated Learning](#) *2020*
- FedFomo — [Personalized Federated Learning with First Order Model Optimization](#) *ICLR 2021*
- FedAMP — [Personalized Cross-Silo Federated Learning on non-IID Data](#) *AAAI 2021*
- FedPHP — [FedPHP: Federated Personalization with Inherited Private Models](#) *ECML PKDD 2021*



Our simulation platform



Baselines

- APPLE — [Adapt to Adaptation: Learning Personalization for Cross-Silo Federated Learning](#) IJCAI 2022

- FedALA — [FedALA: Adaptive Local Aggregation for Personalized Federated Learning](#) AAAI 2023

Model-splitting-based pFL

- FedPer — [Federated Learning with Personalization Layers](#) 2019

- LG-FedAvg — [Think Locally, Act Globally: Federated Learning with Local and Global Representations](#) 2020

- FedRep — [Exploiting Shared Representations for Personalized Federated Learning](#) ICML 2021

- FedRoD — [On Bridging Generic and Personalized Federated Learning for Image Classification](#) ICLR 2022

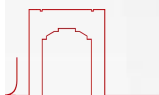
- FedBABU — [Fedbabu: Towards enhanced representation for federated image classification](#) ICLR 2022

- FedGC — [Federated Learning for Face Recognition with Gradient Correction](#) AAAI 2022

- FedCP — [FedCP: Separating Feature Information for Personalized Federated Learning via Conditional Policy](#) KDD 2023

- GPFL — [GPFL: Simultaneously Learning Generic and Personalized Feature Information for Personalized Federated Learning](#) ICCV 2023

- FedGH — [FedGH: Heterogeneous Federated Learning with Generalized Global Header](#) ACM MM 2023





Our simulation platform



Baselines

- FedDBE — [Eliminating Domain Bias for Federated Learning in Representation Space](#) *NeurIPS 2023*
 - FedCAC — [Bold but Cautious: Unlocking the Potential of Personalized Federated Learning through Cautiously Aggressive Collaboration](#) *ICCV 2023*
 - PFL-DA — [Personalized Federated Learning via Domain Adaptation with an Application to Distributed 3D Printing](#) *Technometrics 2023*
- Knowledge-distillation-based pFL*
- FedDistill (FD) — [Communication-Efficient On-Device Machine Learning: Federated Distillation and Augmentation under Non-IID Private Data](#) *2018*
 - FML — [Federated Mutual Learning](#) *2020*
 - FedKD — [Communication-efficient federated learning via knowledge distillation](#) *Nature Communications 2022*
 - FedProto — [FedProto: Federated Prototype Learning across Heterogeneous Clients](#) *AAAI 2022*
 - FedPCL (w/o pre-trained models) — [Federated learning from pre-trained models: A contrastive learning approach](#) *NeurIPS 2022*
 - FedPAC — [Personalized Federated Learning with Feature Alignment and Classifier Collaboration](#) *ICLR 2023*



Our simulation platform

Datasets

Datasets and scenarios (updating)

For the *label skew* scenario, we introduce 14 famous datasets: MNIST, EMNIST, Fashion-MNIST, Cifar10, Cifar100, AG News, Sogou News, Tiny-ImageNet, Country211, Flowers102, GTSRB, Shakespeare, and Stanford Cars, they can be easily split into IID and non-IID version. Since some codes for generating datasets such as splitting are the same for all datasets, we move these codes into `./dataset/utils/dataset_utils.py`. In the non-IID scenario, 2 situations exist. The first one is the **pathological non-IID** scenario, the second one is the **practical non-IID** scenario. In the **pathological non-IID** scenario, for example, the data on each client only contains the specific number of labels (maybe only 2 labels), though the data on all clients contains 10 labels such as the MNIST dataset. In the **practical non-IID** scenario, Dirichlet distribution is utilized (please refer to this [paper](#) for details). We can input `balance` for the iid scenario, where the data are uniformly distributed.

For the *feature shift* scenario, we use 3 datasets that are widely used in Domain Adaptation: **Amazon Review** (fetch raw data from [this site](#)), **Digit5** (fetch raw data from [this site](#)), and **DomainNet**.

For the *real-world (or IoT)* scenario, we also introduce 3 naturally separated datasets: **OmniGlott** (20 clients, 50 labels), **HAR (Human Activity Recognition)** (30 clients, 6 labels), **PAMAP2** (9 clients, 12 labels). For the details of datasets and FL algorithms in IoT, please refer to [my FL-IoT repo](#).





Models

Models

- for MNIST and Fashion-MNIST
 - i. Mclr_Logistic(1*28*28)
 - ii. LeNet()
 - iii. DNN(1*28*28, 100) # non-convex
- for Cifar10, Cifar100 and Tiny-ImageNet
 - i. Mclr_Logistic(3*32*32)
 - ii. FedAvgCNN()
 - iii. DNN(3*32*32, 100) # non-convex
 - iv. ResNet18, AlexNet, MobileNet, GoogleNet, etc.
- for AG_News and Sogou_News
 - i. LSTM()
 - ii. fastText() in [Bag of Tricks for Efficient Text Classification](#)
 - iii. TextCNN() in [Convolutional Neural Networks for Sentence Classification](#)
 - iv. TransformerModel() in [Attention is all you need](#)
- for AmazonReview
 - i. AmazonMLP() in [Curriculum manager for source selection in multi-source domain adaptation](#)



Our simulation platform

Installation

Environments

Install [CUDA](#).

Install [conda](#) and activate conda.

```
conda env create -f env_cuda_latest.yaml # You may need to downgrade the torch using pip to match the C 
```



Our simulation platform

Dataset

Examples for MNIST

- MNIST

```
cd ./dataset
# python generate_MNIST.py iid - - # for iid and unbalanced scenario
# python generate_MNIST.py iid balance - # for iid and balanced scenario
# python generate_MNIST.py noniid - pat # for pathological noniid and unbalanced scenario
python generate_MNIST.py noniid - dir # for practical noniid and unbalanced scenario
# python generate_MNIST.py noniid - exdir # for Extended Dirichlet strategy
```

The output of `python generate_MNIST.py noniid - dir`

```
Number of classes: 10
Client 0      Size of data: 2630      Labels: [0 1 4 5 7 8 9]
              Samples of labels: [(0, 140), (1, 890), (4, 1), (5, 319), (7, 29), (8, 1067), (9, 184)
-----
Client 1      Size of data: 499      Labels: [0 2 5 6 8 9]
              Samples of labels: [(0, 5), (2, 27), (5, 19), (6, 335), (8, 6), (9, 107)]
-----
Client 2      Size of data: 1630     Labels: [0 3 6 9]
              Samples of labels: [(0, 3), (3, 143), (6, 1461), (9, 23)]
-----
```

► Show more



Our simulation platform

Execution

How to start simulating (examples for FedAvg)

- Create proper environments (see [Environments](#)).
- Download [this project](#) to an appropriate location using [git](#).

```
git clone https://github.com/TsingZ0/PFLlib.git
```



- Build evaluation scenarios (see [Datasets and scenarios \(updating\)](#)).
- Run evaluation:

```
cd ./system  
python main.py -data MNIST -m cnn -algo FedAvg -gr 2000 -did 0 # using the MNIST dataset, the FedAv
```



Note: It is preferable to tune algorithm-specific hyper-parameters before using any algorithm on a new machine.





Top-down

- Server: Aggregate, Send, Receive, Evaluate
 - Add
- Client: Train, Receive, Send
 - Dataset
 - Model
 - Optimizer

Algorithm 1 FederatedAveraging. The K clients are indexed by k ; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:

```
initialize  $w_0$ 
for each round  $t = 1, 2, \dots$  do
   $m \leftarrow \max(C \cdot K, 1)$ 
   $S_t \leftarrow$  (random set of  $m$  clients)
  for each client  $k \in S_t$  in parallel do
     $w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$ 
   $w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$ 
```

ClientUpdate(k, w): // Run on client k
 $\mathcal{B} \leftarrow$ (split $\mathcal{P}_k$ into batches of size B)
for each local epoch i from 1 to E **do**
for batch $b \in \mathcal{B}$ **do**
 $w \leftarrow w - \eta \nabla \ell(w; b)$
 return w to server



02

Hands-on algorithms

- MOON
- FedDyn



MOON

Observation

- The global model trained on a whole dataset can learn a better representation than the local model trained on a skewed subset.
- Propose model-contrastive learning (MOON), which corrects the local updates by maximizing the agreement of representation learned by the current local model and the representation learned by the global model.

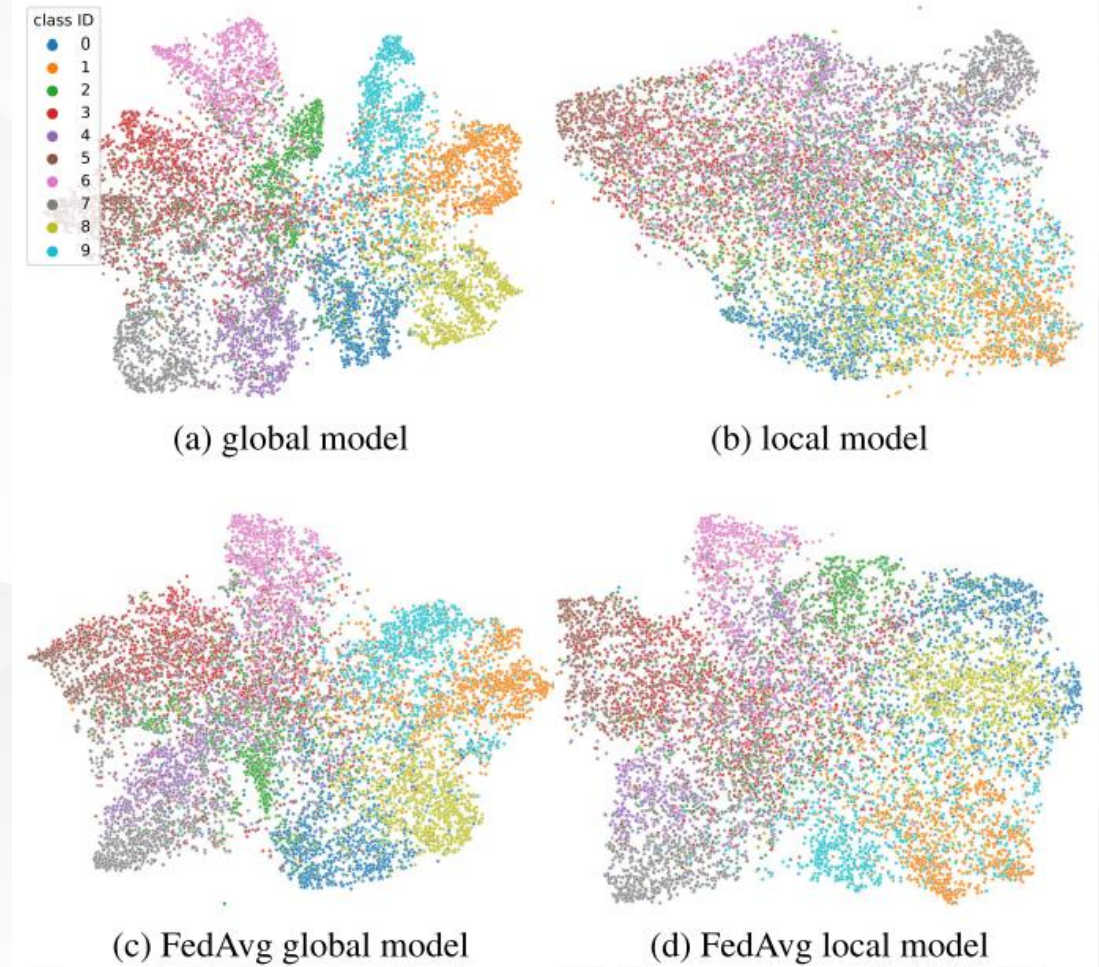


Figure 2. T-SNE visualizations of hidden vectors on CIFAR-10.



Contrastive learning

- The key idea of contrastive learning is to reduce the distance between the representations of different augmented views of the same image (i.e., positive pairs), and increase the distance between the representations of augmented views of different images (i.e., negative pairs)

$$l_{i,j} = -\log \frac{\exp(\text{sim}(x_i, x_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{I}_{[k \neq i]} \exp(\text{sim}(x_i, x_k)/\tau)}$$



MOON algorithm

MOON

- Global representation $z_{glob} = R_{w^t}(x)$
- Local representation w_i^{t-1} (i.e., $z_{prev} = R_{w_i^{t-1}}(x)$)
- Current representation $z = R_{w_i^t}(x)$
-

$$\ell_{con} = -\log \frac{\exp(\text{sim}(z, z_{glob})/\tau)}{\exp(\text{sim}(z, z_{glob})/\tau) + \exp(\text{sim}(z, z_{prev})/\tau)}$$

$$\ell = \ell_{sup}(w_i^t; (x, y)) + \mu \ell_{con}(w_i^t; w_i^{t-1}; w^t; x),$$

$$\min_{w_i^t} \mathbb{E}_{(x, y) \sim D^i} [\ell_{sup}(w_i^t; (x, y)) + \mu \ell_{con}(w_i^t; w_i^{t-1}; w^t; x)].$$



MOON

Algorithm 1: The MOON framework

Input: number of communication rounds T ,
number of parties N , number of local
epochs E , temperature τ , learning rate η ,
hyper-parameter μ

Output: The final model w^T

1 **Server executes:**

2 initialize w^0

3 **for** $t = 0, 1, \dots, T - 1$ **do**

4 **for** $i = 1, 2, \dots, N$ **in parallel do**

5 send the global model w^t to P_i

6 $w_i^t \leftarrow \text{PartyLocalTraining}(i, w^t)$

7 $w^{t+1} \leftarrow \sum_{k=1}^N \frac{|\mathcal{D}^i|}{|\mathcal{D}|} w_k^t$

8 **return** w^T

9 **PartyLocalTraining**(i, w^t):

10 $w_i^t \leftarrow w^t$

11 **for** epoch $i = 1, 2, \dots, E$ **do**

12 **for** each batch $\mathbf{b} = \{x, y\}$ of $\mathcal{D}^i$ **do**

13 $\ell_{sup} \leftarrow \text{CrossEntropyLoss}(F_{w_i^t}(x), y)$

14 $z \leftarrow R_{w_i^t}(x)$

15 $z_{glob} \leftarrow R_{w^t}(x)$

16 $z_{prev} \leftarrow R_{w_i^{t-1}}(x)$

17 $\ell_{con} \leftarrow$

$-\log \frac{\exp(\text{sim}(z, z_{glob})/\tau)}{\exp(\text{sim}(z, z_{glob})/\tau) + \exp(\text{sim}(z, z_{prev})/\tau)}$

18 $\ell \leftarrow \ell_{sup} + \mu \ell_{con}$

19 $w_i^t \leftarrow w_i^t - \eta \nabla \ell$

20 **return** w_i^t to server



FedDyn



Intuition

- Training models on local data that minimize local empirical loss appears to be meaningful, but yet, doing so is fundamentally inconsistent with minimizing the global empirical loss.
- Dynamically modify the device objective with a penalty term so that, in the limit, when model parameters converge, they do so to stationary points of the global empirical loss.

Algorithm 1: Federated Dynamic Regularizer - (FedDyn)

Input: $T, \theta^0, \alpha > 0, \nabla L_k(\theta_k^0) = 0$.

for $t = 1, 2, \dots, T$ **do**

 Sample devices $\mathcal{P}_t \subseteq [m]$ and transmit θ^{t-1} to each selected device,

for *each device* $k \in \mathcal{P}_t$, *and in parallel* **do**

 Set $\theta_k^t = \underset{\theta}{\operatorname{argmin}} L_k(\theta) - \langle \nabla L_k(\theta_k^{t-1}), \theta \rangle + \frac{\alpha}{2} \|\theta - \theta^{t-1}\|^2$,

 Set $\nabla L_k(\theta_k^t) = \nabla L_k(\theta_k^{t-1}) - \alpha (\theta_k^t - \theta^{t-1})$,

 Transmit device model θ_k^t to server,

end for

for *each device* $k \notin \mathcal{P}_t$, *and in parallel* **do**

 Set $\theta_k^t = \theta_k^{t-1}, \nabla L_k(\theta_k^t) = \nabla L_k(\theta_k^{t-1})$,

end for

 Set $h^t = h^{t-1} - \alpha \frac{1}{m} (\sum_{k \in \mathcal{P}_t} \theta_k^t - \theta^{t-1})$,

 Set $\theta^t = \left(\frac{1}{|\mathcal{P}_t|} \sum_{k \in \mathcal{P}_t} \theta_k^t \right) - \frac{1}{\alpha} h^t$

end for

Analysis

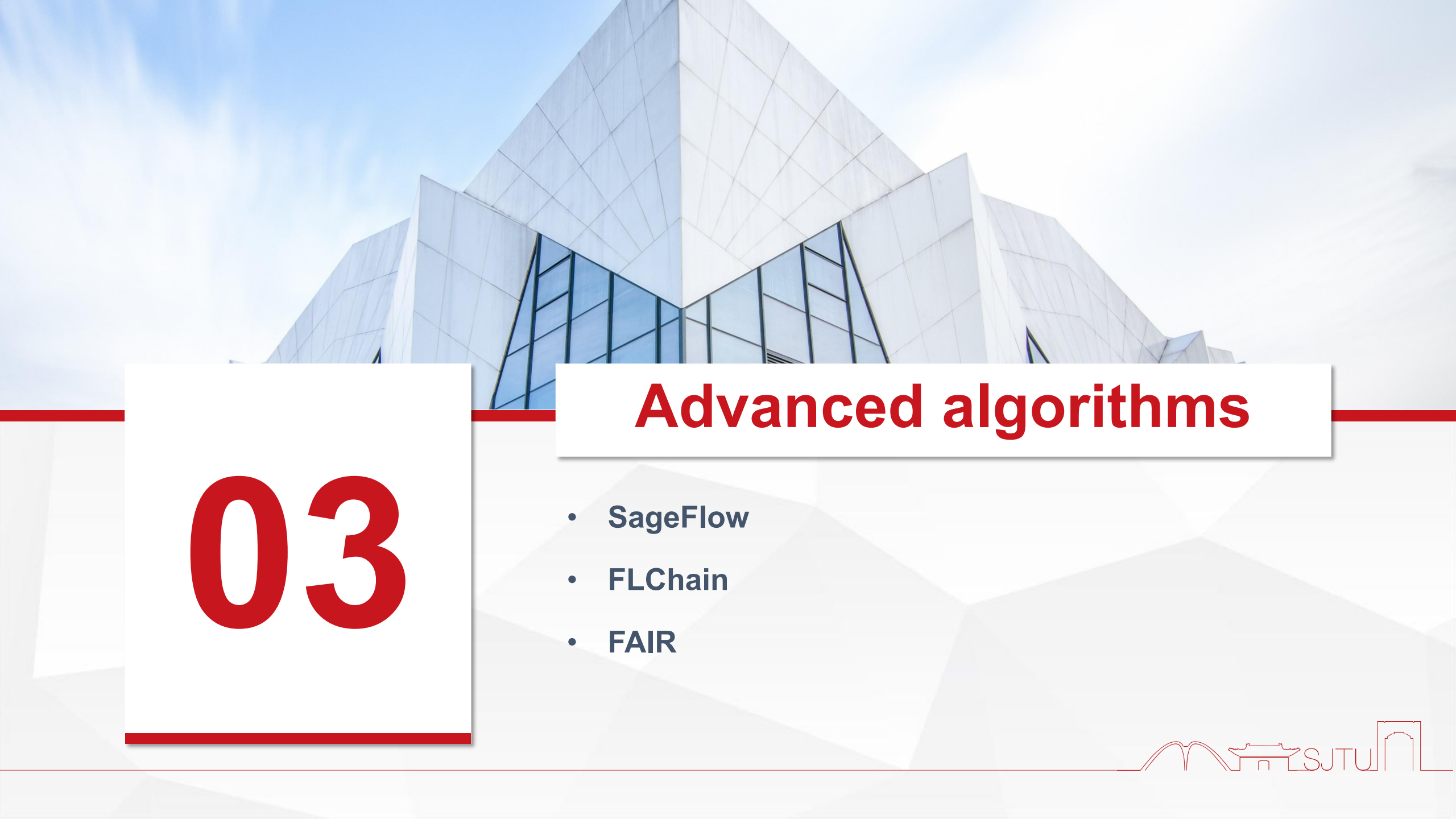
$$\boldsymbol{\theta}_k^t = \operatorname{argmin}_{\boldsymbol{\theta}} \left[\mathfrak{R}_k(\boldsymbol{\theta}; \boldsymbol{\theta}_k^{t-1}, \boldsymbol{\theta}^{t-1}) \triangleq L_k(\boldsymbol{\theta}) - \langle \nabla L_k(\boldsymbol{\theta}_k^{t-1}), \boldsymbol{\theta} \rangle + \frac{\alpha}{2} \|\boldsymbol{\theta} - \boldsymbol{\theta}^{t-1}\|^2 \right]$$

- The first order condition

$$\nabla L_k(\boldsymbol{\theta}_k^t) - \nabla L_k(\boldsymbol{\theta}_k^{t-1}) + \alpha(\boldsymbol{\theta}_k^t - \boldsymbol{\theta}^{t-1}) = \mathbf{0}$$

- If local device models converge, they converge to the server model, and the convergence point is a stationary point of the global loss.

if $\boldsymbol{\theta}_k^t \rightarrow \boldsymbol{\theta}_k^\infty$, it generally follows that, $\nabla L_k(\boldsymbol{\theta}_k^t) \rightarrow \nabla L_k(\boldsymbol{\theta}_k^\infty)$, and as a consequence, we have $\boldsymbol{\theta}^t \rightarrow \boldsymbol{\theta}_k^\infty$. In turn this implies that $\boldsymbol{\theta}_k^\infty \rightarrow \boldsymbol{\theta}^\infty$, i.e., is independent of k .



03

Advanced algorithms

- SageFlow
- FLChain
- FAIR



KT-pFL



KT-pFL algorithm

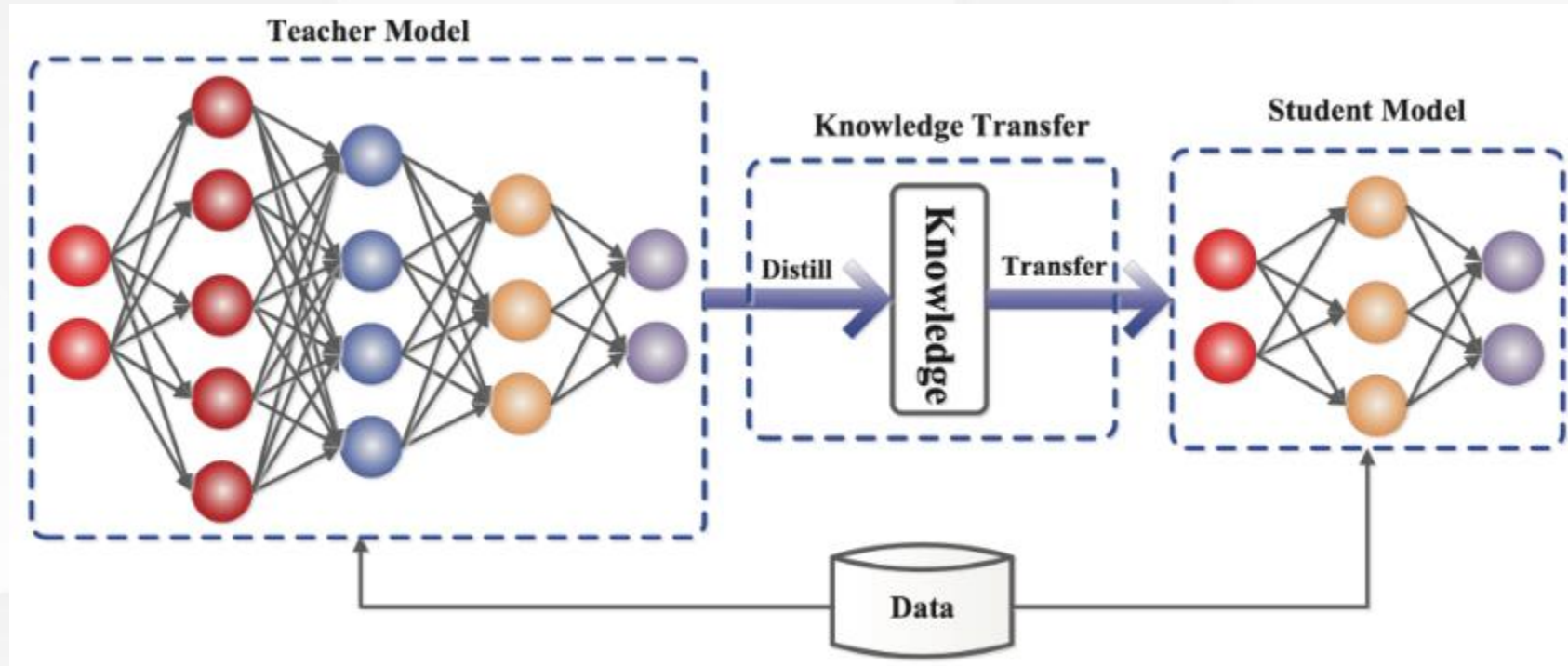
Intuition

- Main idea is to allow each client to maintain a personalized soft prediction at the server that can be updated by a linear combination of all clients' local soft predictions using a knowledge coefficient matrix.
- Regardless of model structures

KT-pFL algorithm

Knowledge Distillation (KD)

- Transfer knowledge from well-learned teacher model to student model

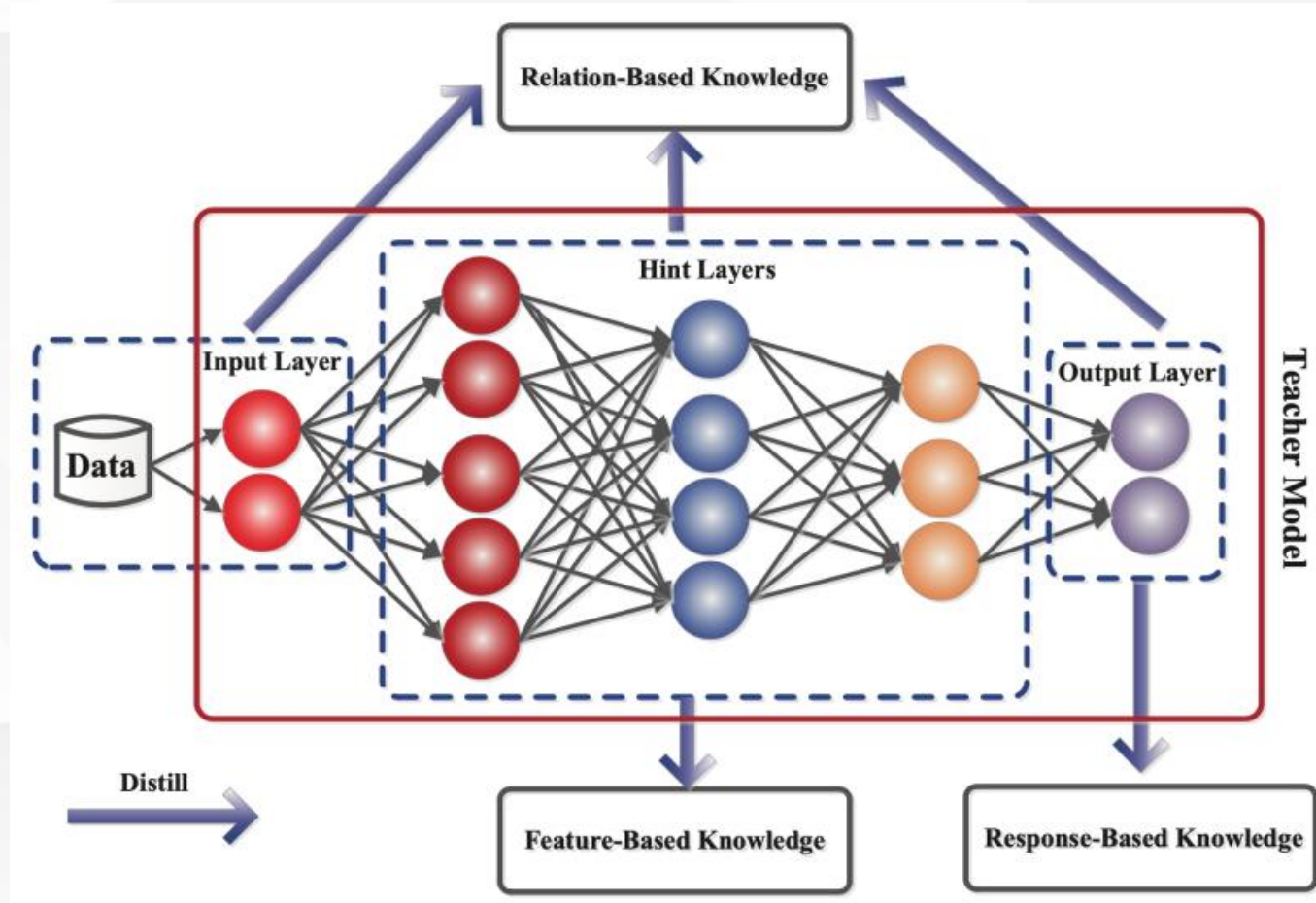




KT-pFL algorithm

Knowledge Distillation (KD)

- Classification

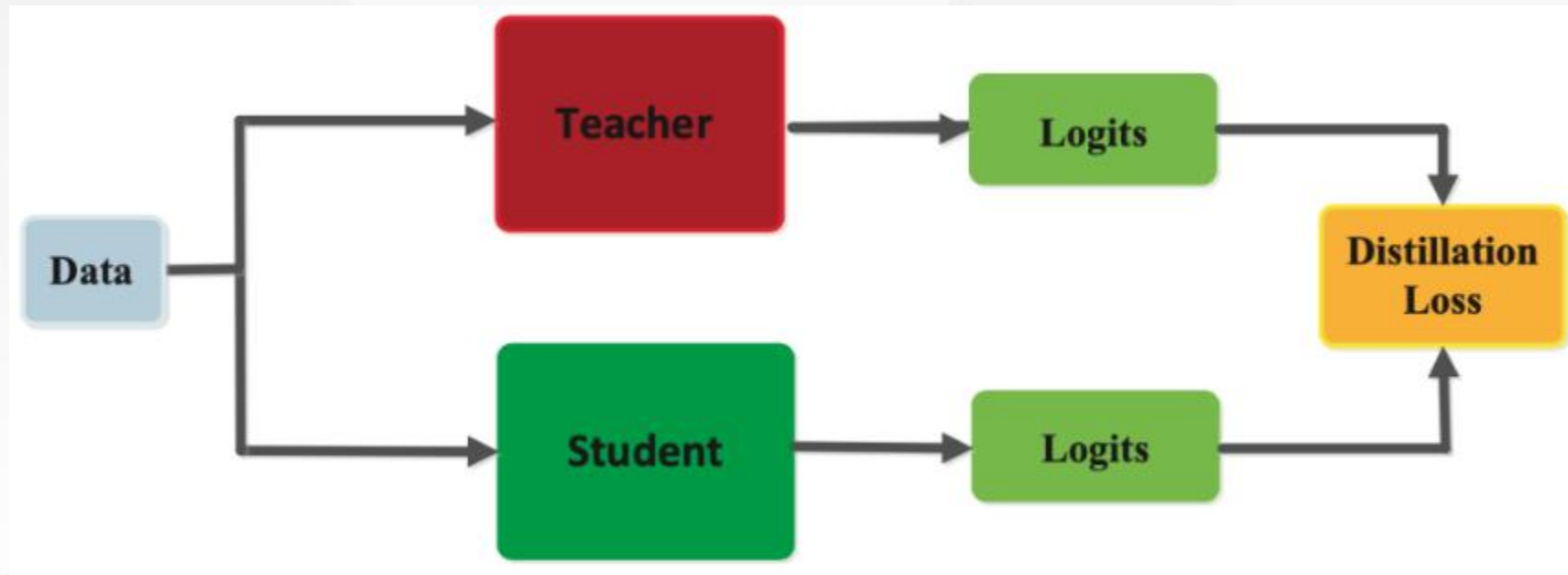




KT-pFL algorithm

Knowledge Distillation (KD)

- Response-based KD





KT-pFL algorithm

KT-pFL

- Objective

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) := \sum_{n=1}^N \frac{D_n}{D} \mathcal{L}_n(\mathbf{w}), \text{ where } \mathcal{L}_n(\mathbf{w}) = \frac{1}{D_n} \sum_{i=1}^{D_n} \mathcal{L}_{CE}(\mathbf{w}; x_i, y_i).$$

$$\min_{\mathbf{w}^1, \dots, \mathbf{w}^N} \mathcal{L}(\mathbf{w}^1, \dots, \mathbf{w}^N) := \sum_{n=1}^N \frac{D_n}{D} \mathcal{L}_n(\mathbf{w}^n)$$

KT-pFL algorithm

KT-pFL

- Personalized loss function
 - Kullback–Leibler (KL) Divergence
 - c_{mn} is the knowledge coefficient which is used to estimate the contribution from client m to n.
 - $s(\mathbf{w}^n, \hat{x})$ can be deemed to be a soft prediction of the client n

$$\mathcal{L}_{per,n}(\mathbf{w}^n) := \mathcal{L}_n(\mathbf{w}^n) + \lambda \sum_{\hat{x} \in \mathbb{D}_r} \mathcal{L}_{KL} \left(\sum_{m=1}^N c_{mn} \cdot s(\mathbf{w}^m, \hat{x}), s(\mathbf{w}^n, \hat{x}) \right)$$

$$s(\mathbf{w}^n, \hat{x}) = \frac{\exp(z_c^n / T)}{\sum_{c=1}^C \exp(z_c^n / T)},$$



KT-pFL algorithm

KT-pFL

- Knowledge coefficient matrix

$$\mathbf{c} = \begin{Bmatrix} c_{11} & c_{12} & \cdots & c_{1N} \\ c_{21} & c_{22} & \cdots & c_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ c_{N1} & c_{N2} & \cdots & c_{NN} \end{Bmatrix}.$$



KT-pFL algorithm

KT-pFL

- Objective

$$\min_{\mathbf{w}, \mathbf{c}} \mathcal{L}(\mathbf{w}, \mathbf{c}) := \sum_{n=1}^N \frac{D_n}{D} \mathcal{L}_{per,n}(\mathbf{w}^n) + \rho \left\| \mathbf{c} - \frac{\mathbf{1}}{N} \right\|^2,$$

$$\mathbf{w} = [\mathbf{w}^1, \dots, \mathbf{w}^N] \in \mathbb{R}^{\sum_{n=1}^N d_n}$$

KT-pFL algorithm

KT-pFL

- Training
 - Update $\mathbf{w}$
 - Local Training
 - Distillation
 - Update $\mathbf{c}$

$$\mathbf{w}^n \leftarrow \mathbf{w}^n - \eta_1 \nabla_{\mathbf{w}^n} \mathcal{L}_n(\mathbf{w}^n; \xi_n),$$

$$\mathbf{w}^n \leftarrow \mathbf{w}^n - \eta_2 \nabla_{\mathbf{w}^n} \mathcal{L}_{KL} \left(\sum_{m=1}^N \mathbf{c}_m^{*,T} \cdot s(\mathbf{w}^m, \xi_r), s(\mathbf{w}^n, \xi_r) \right)$$

$$\mathbf{c} \leftarrow \mathbf{c} - \eta_3 \lambda \sum_{n=1}^N \frac{D_n}{D} \nabla_{\mathbf{c}} \mathcal{L}_{KL} \left(\sum_{m=1}^N \mathbf{c}_m \cdot s(\mathbf{w}^{m,*}, \xi_r), s(\mathbf{w}^{n,*}, \xi_r) \right) - 2\eta_3 \rho \left(\mathbf{c} - \frac{1}{N} \right)$$

Algorithm 1 KT-pFL Algorithm

Input: $\mathbb{D}, \mathbb{D}_r, \eta_1, \eta_2, \eta_3$ and T

Output: $\mathbf{w} = [\mathbf{w}^1, \dots, \mathbf{w}^N]$

- 1: Initialize $\mathbf{w}_0$ and $\mathbf{c}_0$
- 2: **procedure** SERVER-SIDE OPTIMIZATION
- 3: Distribute $\mathbf{w}_0$ and $\mathbf{c}_0$ to each client
- 4: **for** each communication round $t \in \{1, 2, \dots, T\}$ **do**
- 5: **for** each client n **in parallel do**
- 6: $\mathbf{w}_{t+1}^n \leftarrow \text{ClientLocalUpdate}(n, \mathbf{w}_t^n, \mathbf{c}_{t,n})$
- 7: Update knowledge coefficient matrix $\mathbf{c}$ via (7)
- 8: Distribute $\mathbf{c}_{t+1}$ to all clients
- 9: **procedure** CLIENTLOCALUPDATE($n, \mathbf{w}_t^n, \mathbf{c}_{t,n}$)
- 10: Client n receives $\mathbf{w}_t^n$ and $\mathbf{c}_n$ from the server
- 11: **for** each local epoch i from 1 to E **do**
- 12: **for** mini-batch $\xi_t \subseteq \mathbb{D}_n$ **do**
- 13: **Local Training:** update model parameters on private data via (5)
- 14: **for** each distillation step j from 1 to R **do**
- 15: **for** mini-batch $\xi_{r,t} \subseteq \mathbb{D}_r$ **do**
- 16: **Distillation:** update model parameters on public data via (6)
- return** local parameters $\mathbf{w}_{t+1}^n$

KT-pFL

• Illustration

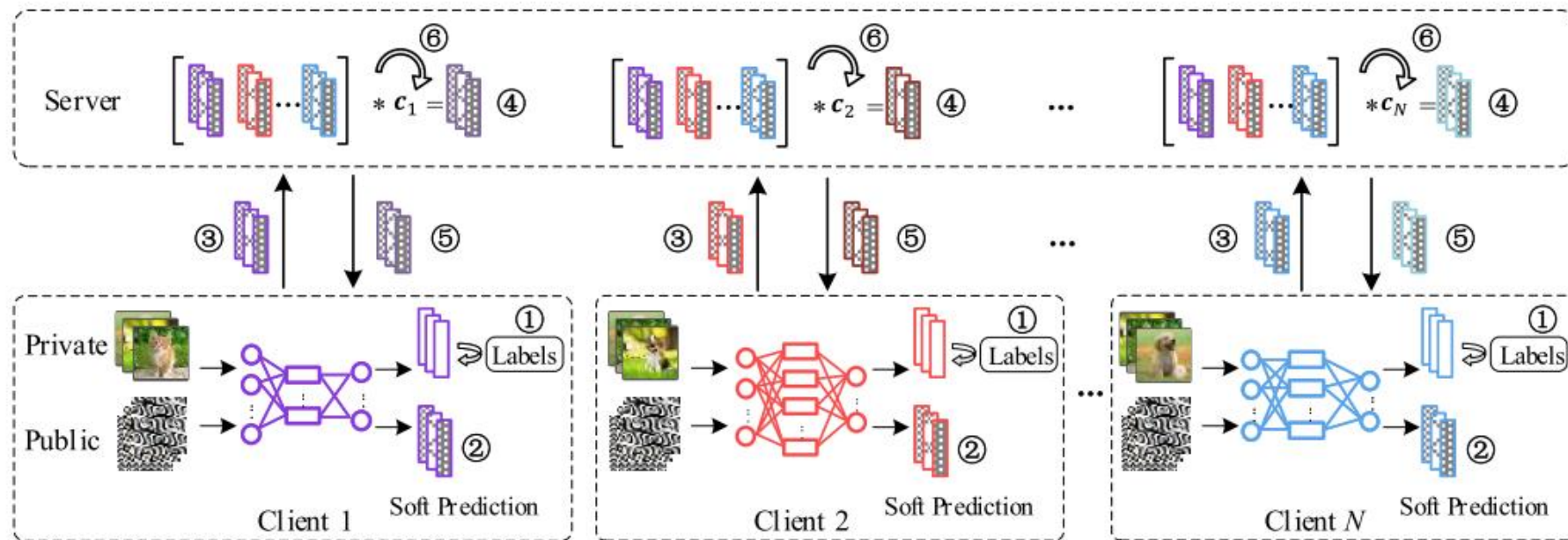


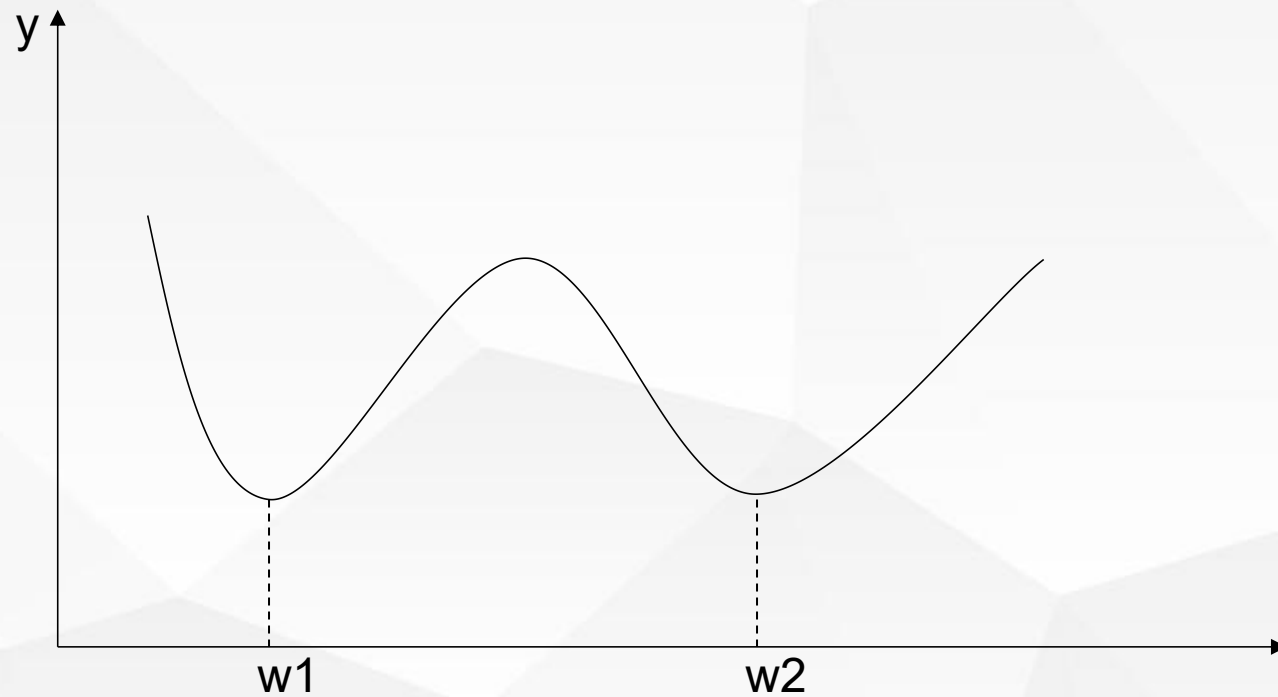
Figure 1: Illustration of the KT-pFL framework. The workflow includes 6 steps: ① local training on private data; ②, ③ each client outputs the local soft prediction on public data and sends it to the server; ④ the server calculates each client's personalized soft prediction via a linear combination of local soft predictions and knowledge coefficient matrix; ⑤ each client downloads the personalized soft prediction to perform distillation phase; ⑥ the server updates the knowledge coefficient matrix.



FedMA



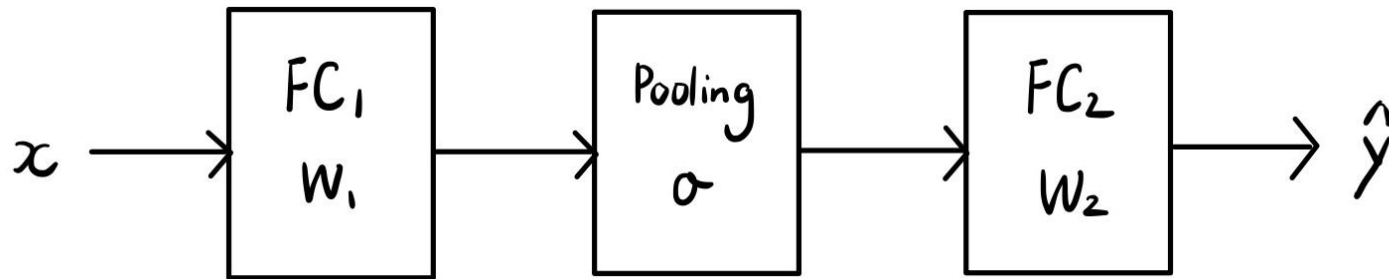
Permutation invariance





FedMA algorithm

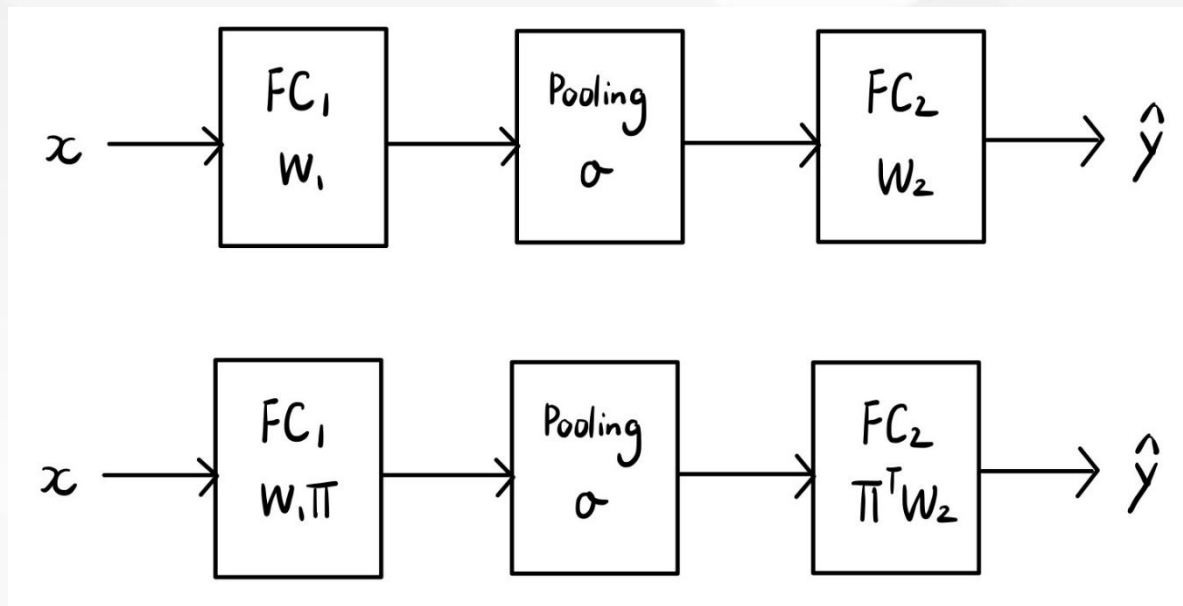
⊗ Permutation invariance (fully-connected (FC) layer)



$$\hat{y} = \sigma(xW_1)W_2$$

FedMA algorithm

⊗ Permutation invariance (fully-connected (FC) layer)



$$\hat{y} = \sigma(xW_1\Pi)\Pi^TW_2, \text{ where } \Pi \text{ is any } L \times L \text{ permutation matrix.}$$



FedMA algorithm

⊗ Permutation invariance (fully-connected (FC) layer)

Client A:

$$\{W_1 \Pi_j, \Pi_j^T W_2\}$$

Client B:

$$\{W_1 \Pi_{j'}, \Pi_{j'}^T W_2\}$$

Aggregation

$$(W_1 \Pi_j + W_1 \Pi_{j'}) / 2 \neq W_1 \Pi \text{ for any } \Pi$$

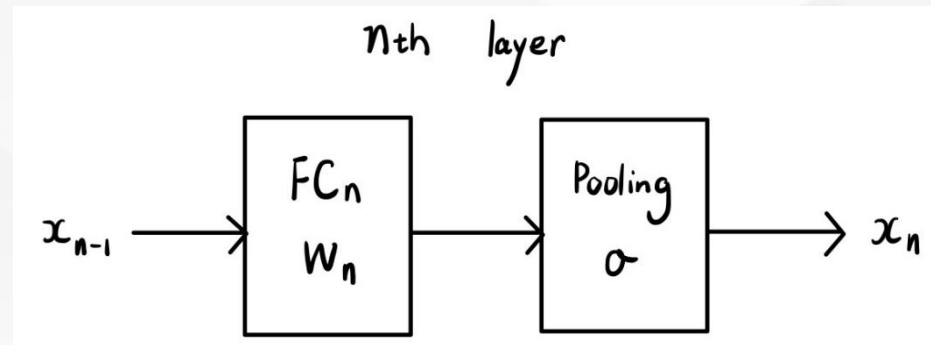
Solution:
$$(W_1 \Pi_j \Pi_j^T + W_1 \Pi_{j'} \Pi_{j'}^T) / 2 = W_1$$





FedMA algorithm

Permutation invariance (FCs)



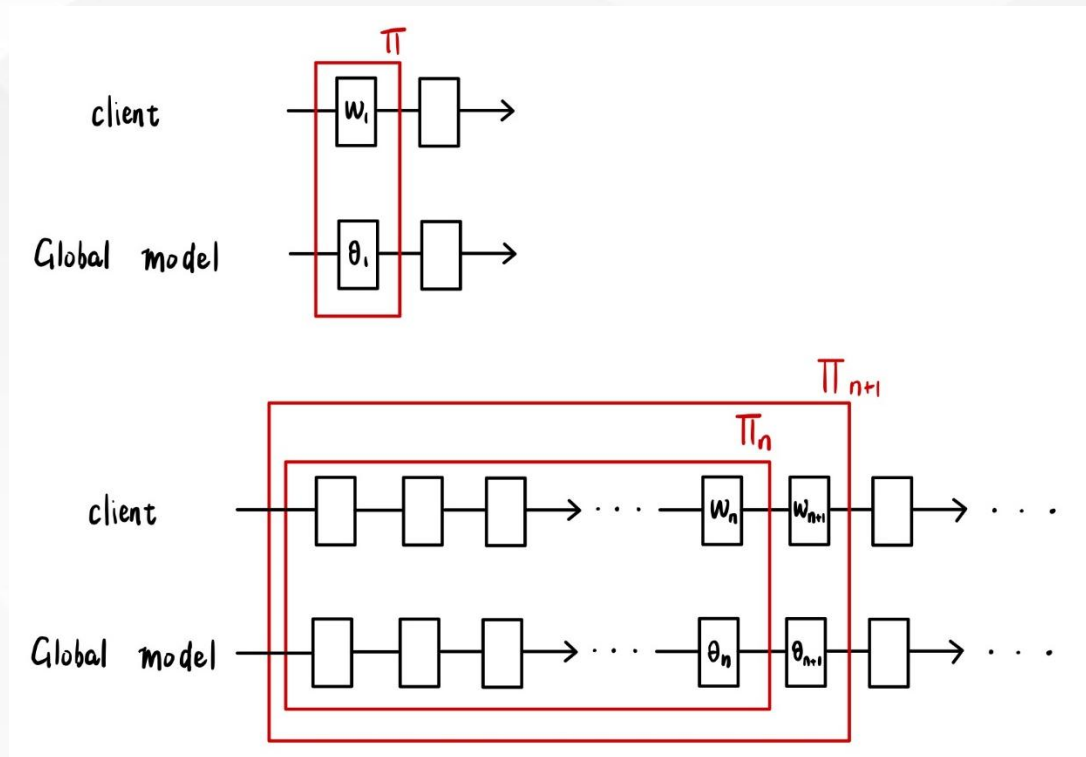
Simple FCs: $\hat{y} = \sigma(xW_1\Pi)\Pi^TW_2$

Deep FCs: $x_n = \sigma(x_{n-1}\Pi_{n-1}^TW_n\Pi_n)$



FedMA algorithm

Permutation invariance (FCs)



$$x_n = \sigma(x_{n-1} \Pi_{n-1}^T W_n \Pi_n)$$





FedMA algorithm

⊗ Permutation invariance (CNN)

$$\text{FC : } x_n = \sigma(x_{n-1} \Pi_{n-1}^T W_n \Pi_n)$$

$$\text{CNN : } x_n = \sigma(\text{Conv}(x_{n-1}, \Pi_{n-1}^T W_n \Pi_n))$$



FedMA algorithm

⊗ Permutation invariance (recall)

Client A:

$$\{W_1 \Pi_j, \Pi_j^T W_2\}$$

Client B:

$$\{W_1 \Pi_{j'}, \Pi_{j'}^T W_2\}$$

Aggregation

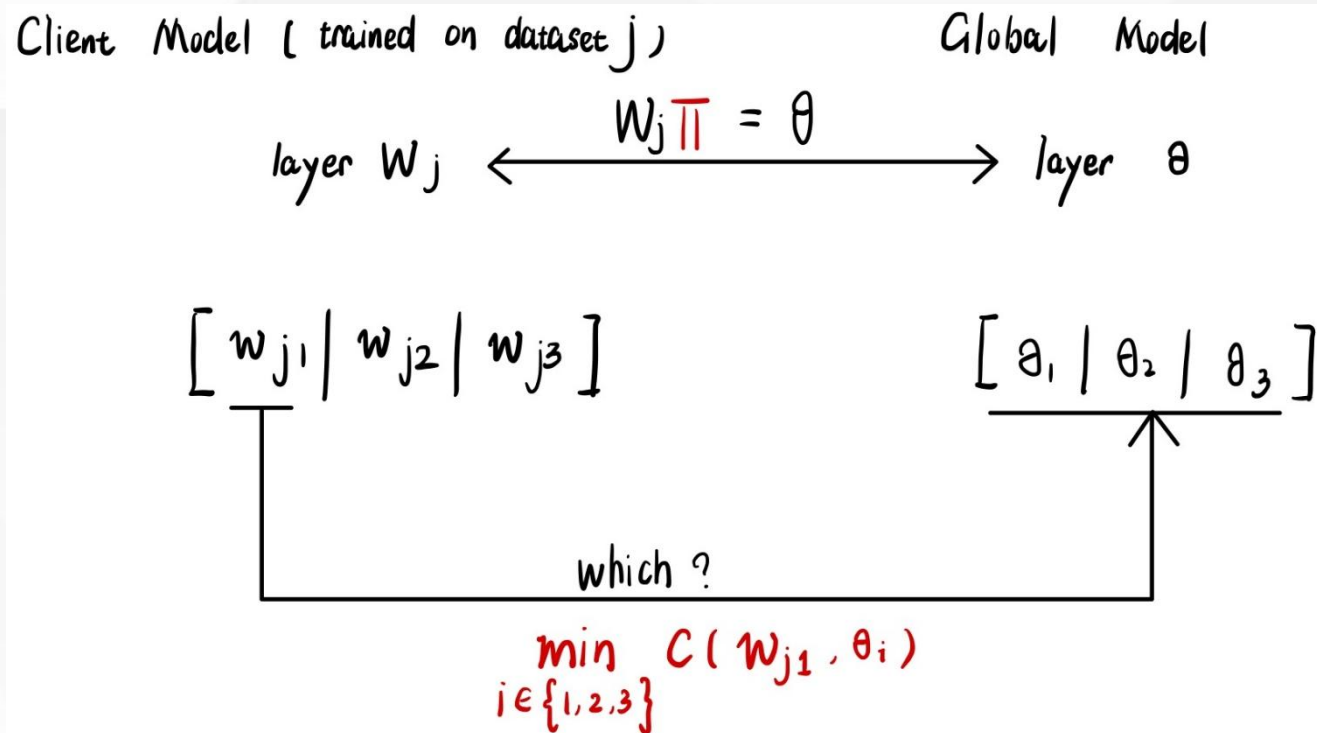
$$(W_1 \Pi_j + W_1 \Pi_{j'}) / 2 \neq W_1 \Pi \text{ for any } \Pi$$

Solution:
$$(W_1 \Pi_j \Pi_j^T + W_1 \Pi_{j'} \Pi_{j'}^T) / 2 = W_1$$



FedMA algorithm

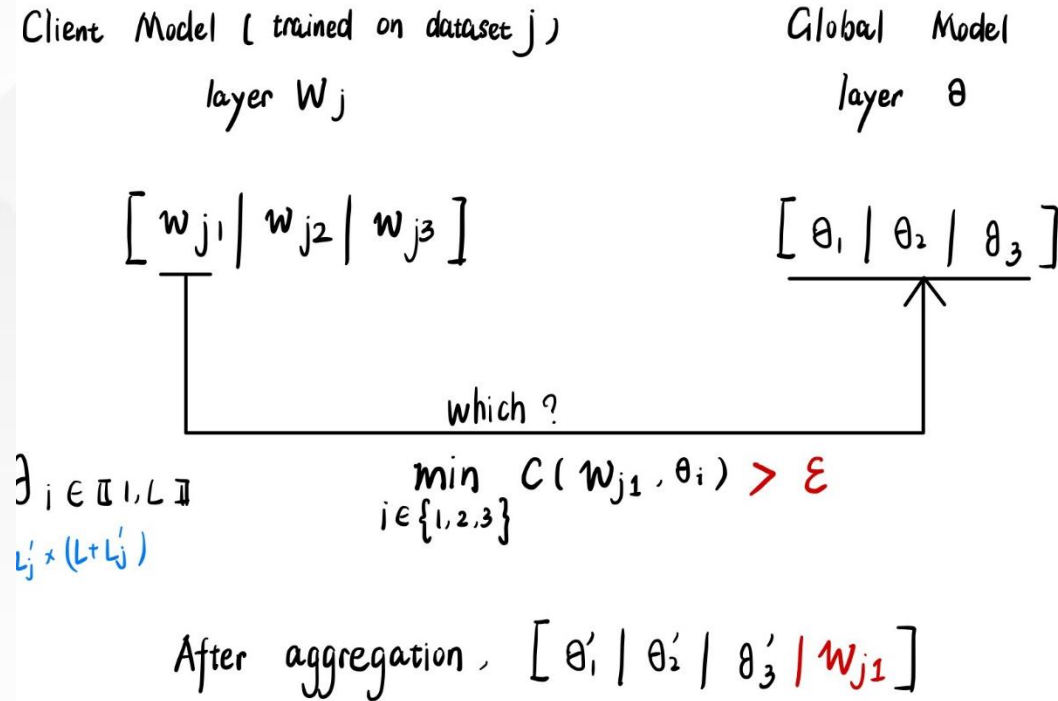
Matched averaging formulation



$$\min_{\{\pi_{li}^j\}} \sum_{i=1}^L \sum_{j,l} \min_{\theta_i} \pi_{li}^j c(w_{jl}, \theta_i) \text{ s.t. } \sum_i \pi_{li}^j = 1 \forall j, l; \sum_l \pi_{li}^j = 1 \forall i, j.$$

FedMA algorithm

Matched averaging formulation



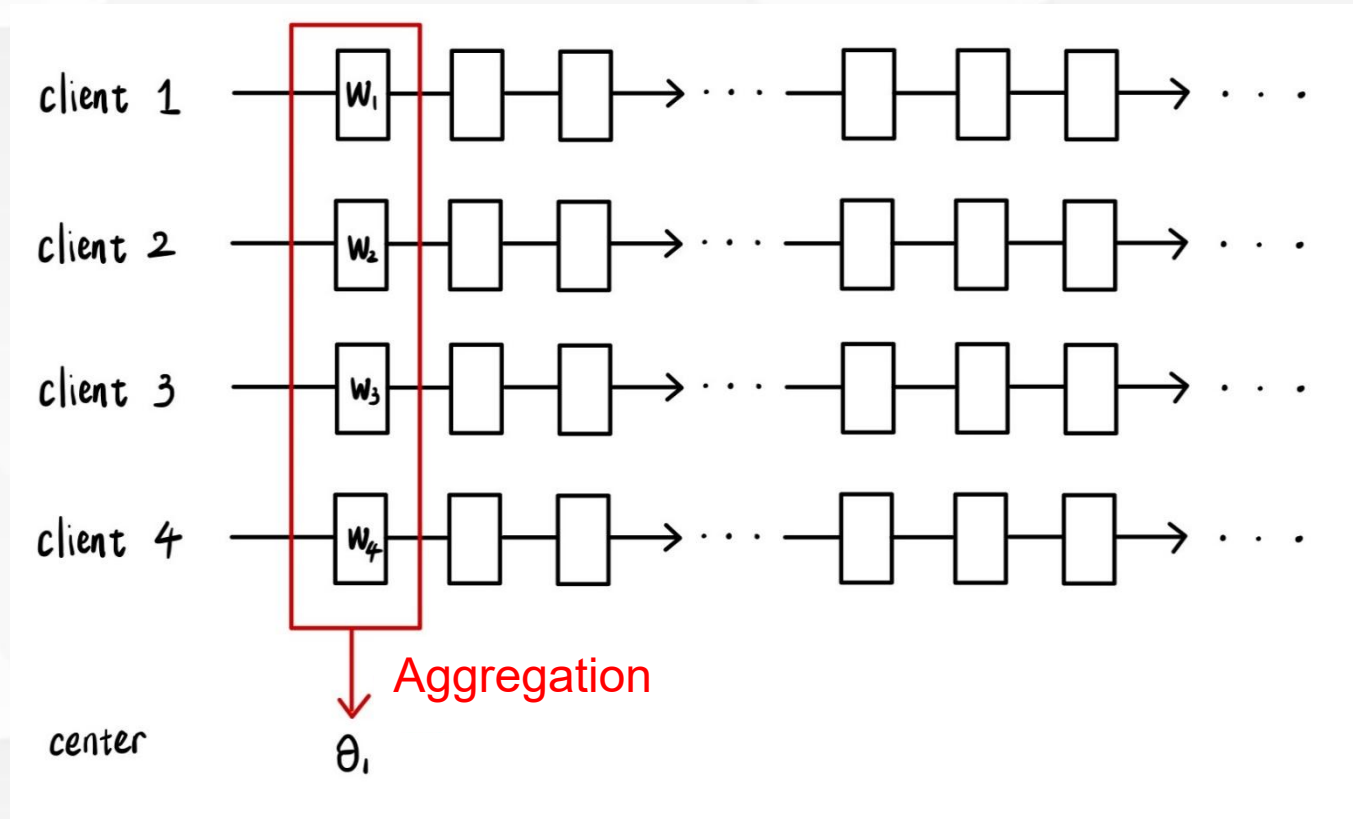
$$\min_{\{\pi_{li}^{j'}\}_{l,i}} \sum_{i=1}^{L+L_{j'}} \sum_{j=1}^{L_{j'}} \pi_{li}^{j'} C_{li}^{j'} \text{ s.t. } \sum_i \pi_{li}^{j'} = 1 \forall l; \sum_l \pi_{li}^j \in \{0, 1\} \forall i, \text{ where}$$

$$C_{li}^{j'} = \begin{cases} c(w_{j'l}, \theta_i), & i \leq L \\ \epsilon + f(i), & L < i \leq L + L_{j'}. \end{cases}$$



FedMA algorithm

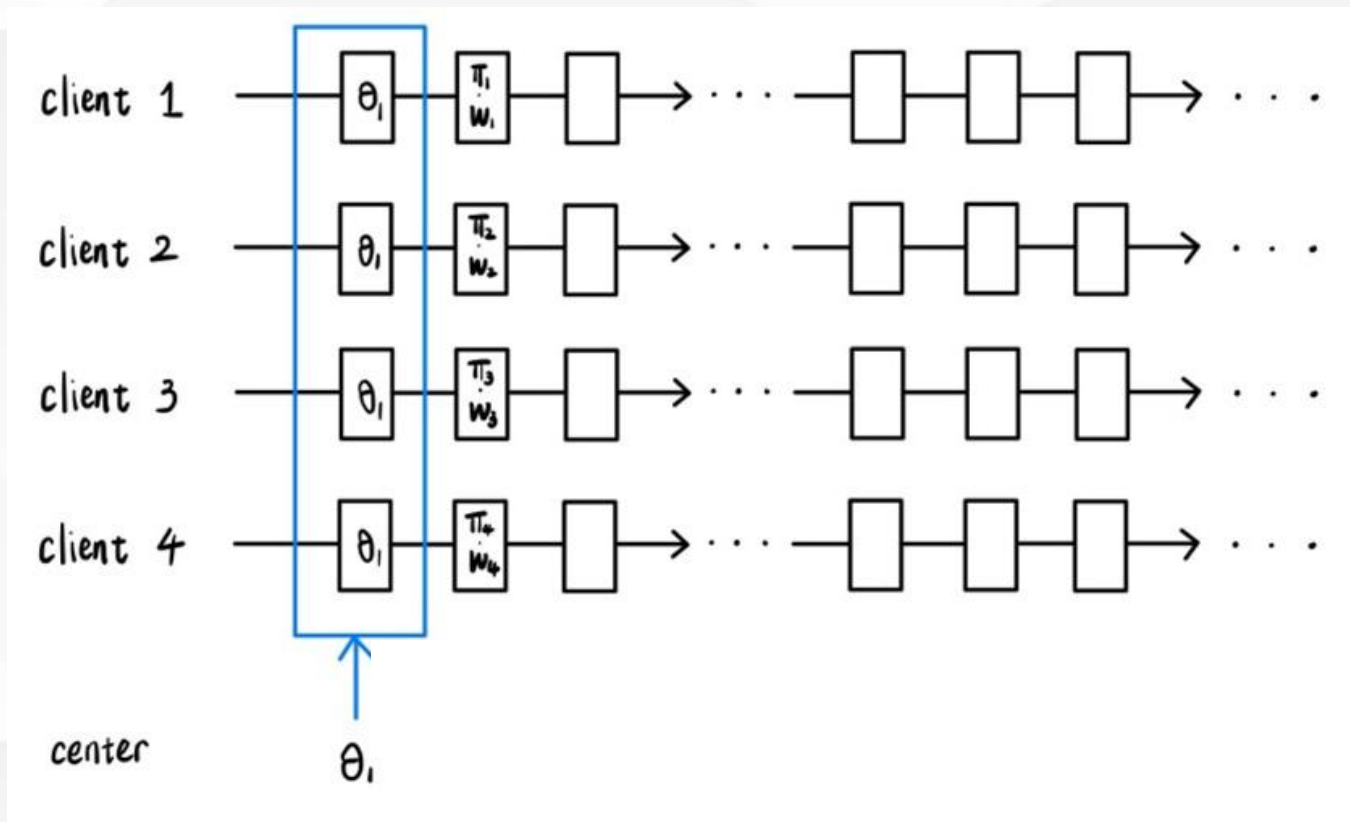
FedMA (<https://github.com/IBM/FedMA>)





FedMA algorithm

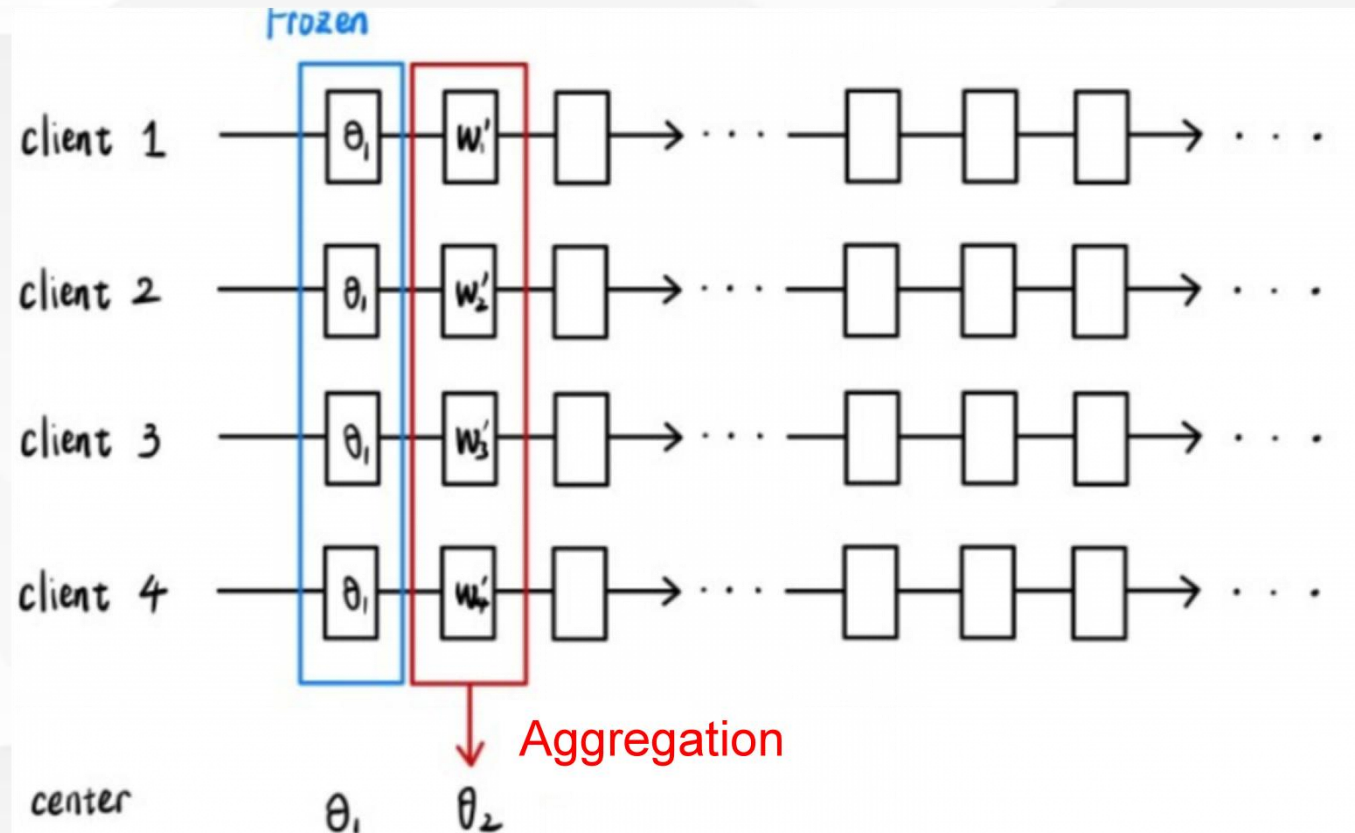
 FedMA (<https://github.com/IBM/FedMA>)





FedMA algorithm

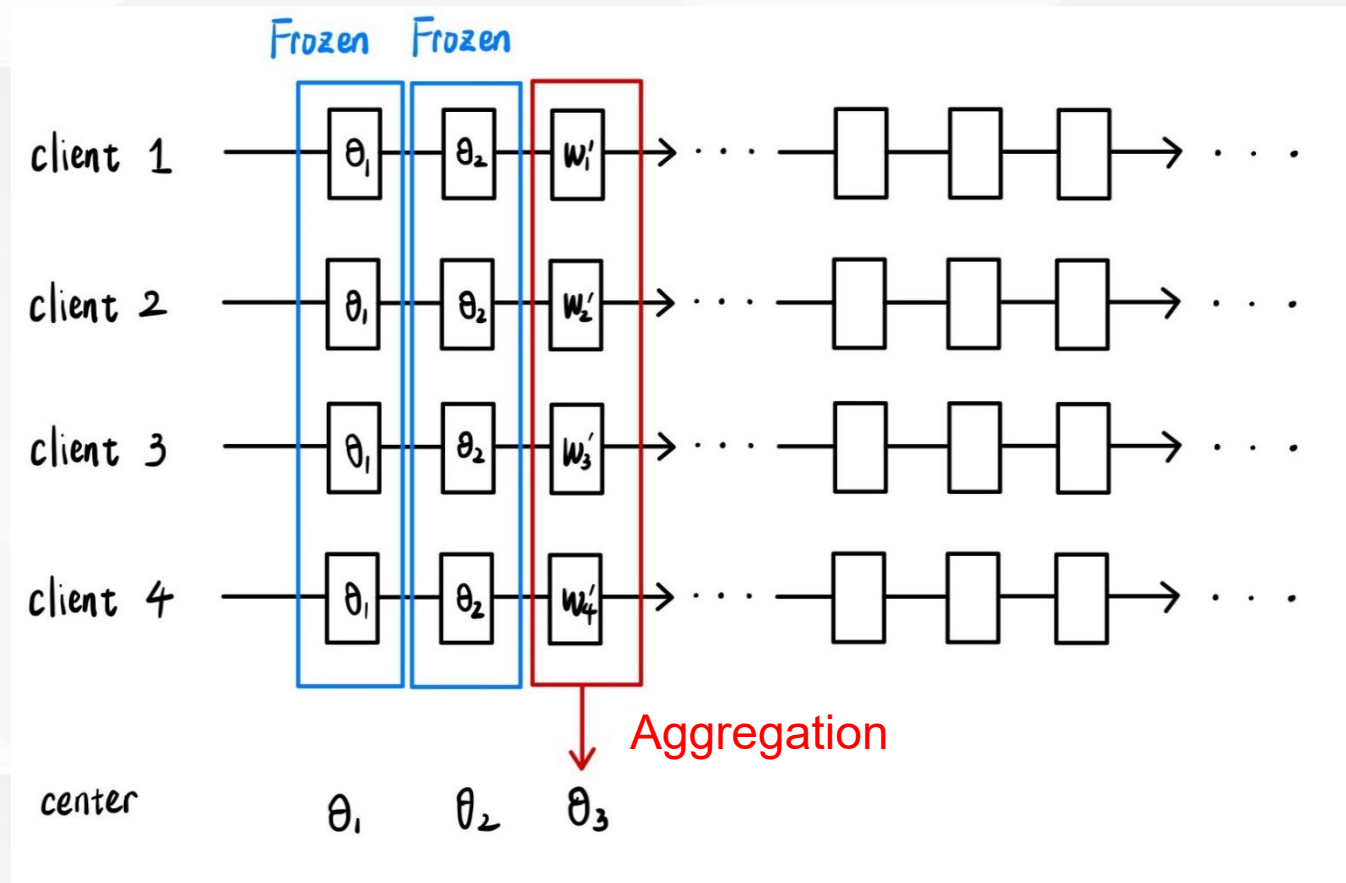
FedMA (<https://github.com/IBM/FedMA>)





FedMA algorithm

FedMA (<https://github.com/IBM/FedMA>)





 FedMA (<https://github.com/IBM/FedMA>)

Algorithm 1: Federated Matched Averaging (FedMA)

Input : local weights of N -layer architectures $\{W_{j,1}, \dots, W_{j,N}\}_{j=1}^J$ from J clients

Output: global weights $\{W_1, \dots, W_N\}$

$n = 1$;

while $n \leq N$ **do**

if $n < N$ **then**

$\{\Pi_j\}_{j=1}^J = \text{BBP-MAP}(\{W_{j,n}\}_{j=1}^J)$; // call BBP-MAP to solve Eq. 2

$W_n = \frac{1}{J} \sum_j W_{j,n} \Pi_j^T$;

else

$W_n = \sum_{k=1}^K \sum_j p_{jk} W_{j,n}$ where p_k is fraction of data points with label k on worker j ;

end

for $j \in \{1, \dots, J\}$ **do**

$W_{j,n+1} \leftarrow \Pi_j W_{j,n+1}$; // permute the next-layer weights

 Train $\{W_{j,n+1}, \dots, W_{j,L}\}$ with W_n frozen;

end

$n = n + 1$;

end

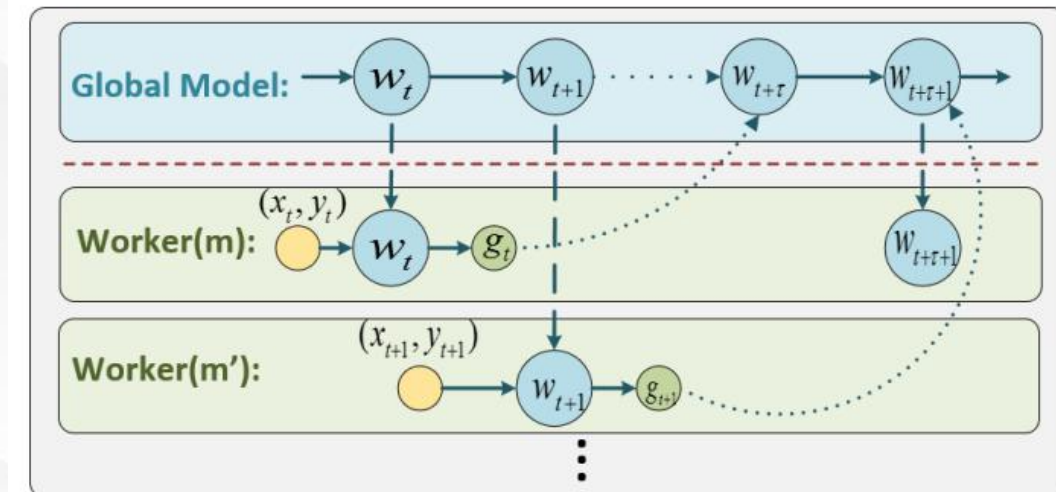
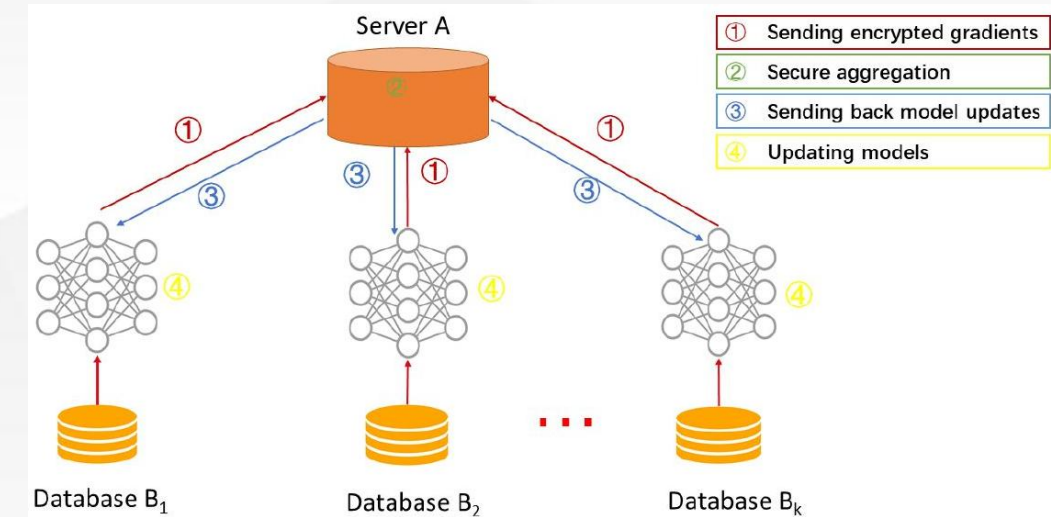




SageFlow

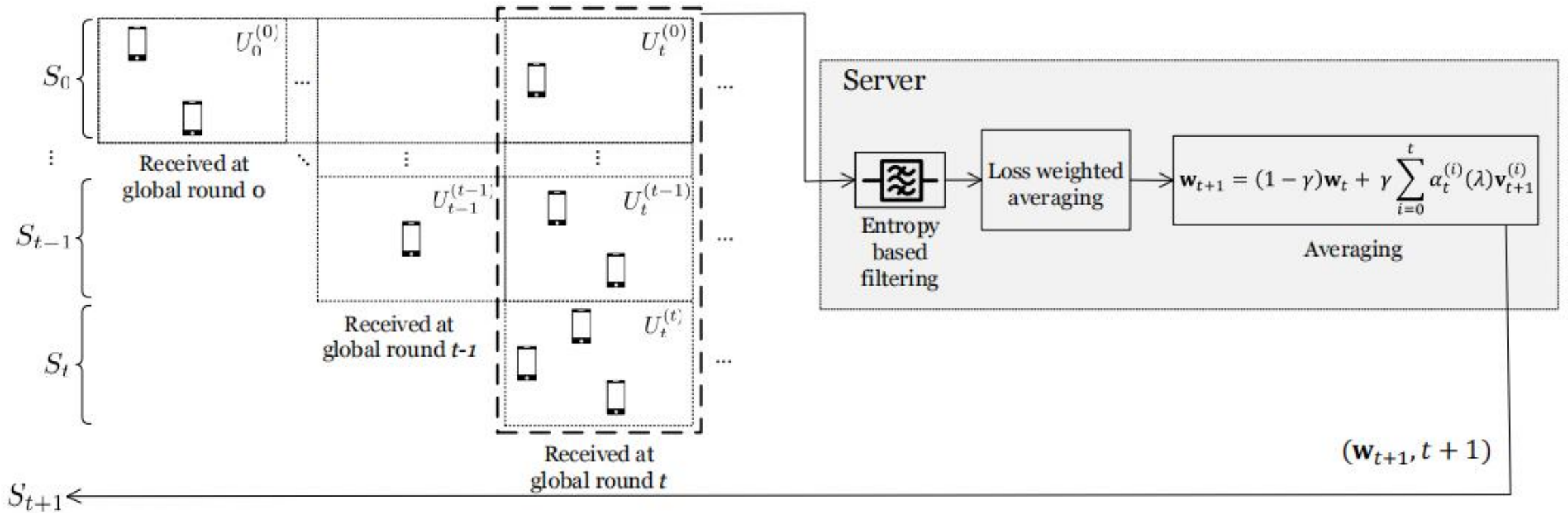
Sageflow algorithm

- ⊗ Stragglers: slow devices
 - **Keep waiting:** slow down the overall process
 - **Drop out:** important data missing
 - **Asynchronous**(staleness): + adversaries?
- ⊗ Attackers: malicious attacks launched by adversaries
 - **untargeted attacks:** model poisoning, data poisoning
 - **targeted/backdoor attacks:** misclassify the targeted subtasks
 - Robust Federated Averaging & Multi-Krum
- Large portion of adversaries
 - Straggler: increase attack ratio



Sageflow algorithm

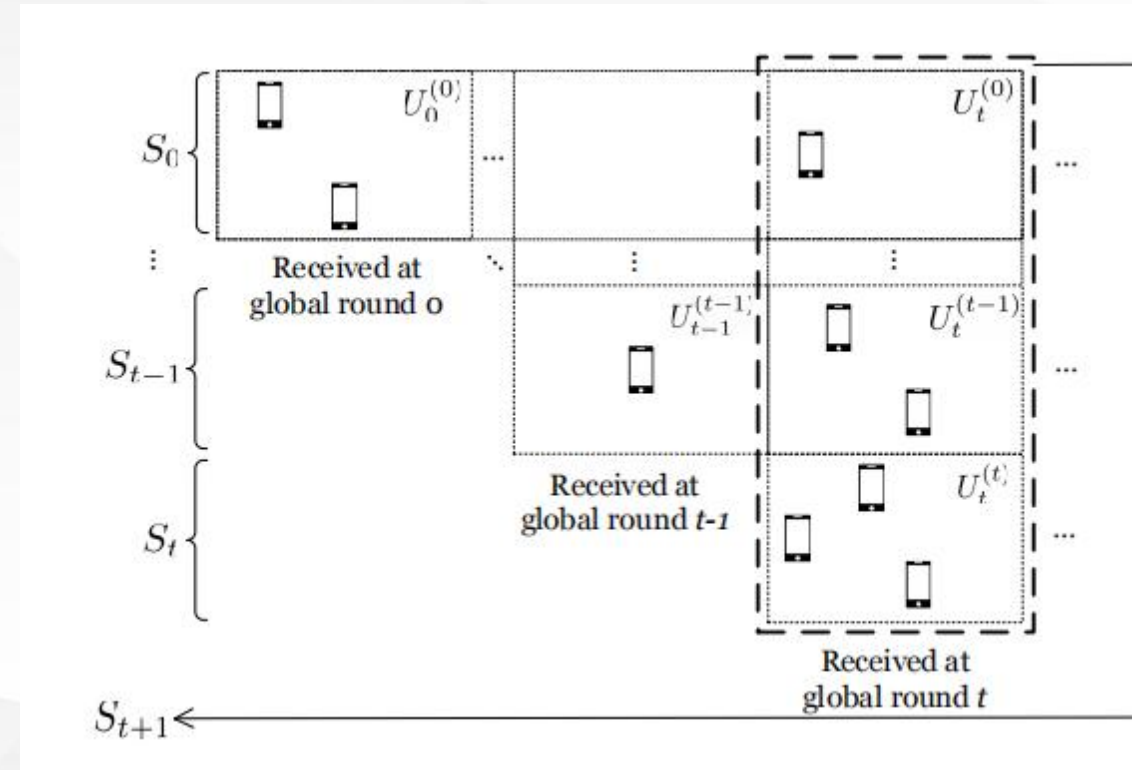
- Staleness-aware grouping
- Entropy-based filtering + Loss-weighted averaging



Sageflow algorithm

Staleness-aware grouping

- perform **periodic** global aggregation(fixed time deadline)
- allow stragglers to be aggregated in **later rounds**
- group with **same staleness** -> group representative model
- aggregate according to **staleness**



Staleness group Number of data samples

$$\mathbf{v}_{t+1}^{(i)} = \sum_{k \in U_t^{(i)}} \frac{m_k}{\sum_{k \in U_t^{(i)}} m_k} \mathbf{w}_i(k)$$

$$\mathbf{w}_{t+1} = (1 - \gamma) \mathbf{w}_t + \gamma \sum_{i=0}^t \alpha_t^{(i)}(\lambda) \mathbf{v}_{t+1}^{(i)}$$

$$\sum_{i=0}^t \alpha_t^{(i)}(\lambda) \mathbf{v}_{t+1}^{(i)}$$

$$\alpha_t^{(i)}(\lambda) \propto \frac{\sum_{k \in U_t^{(i)}} m_k}{(t-i+1)^\lambda}$$

Staleness function

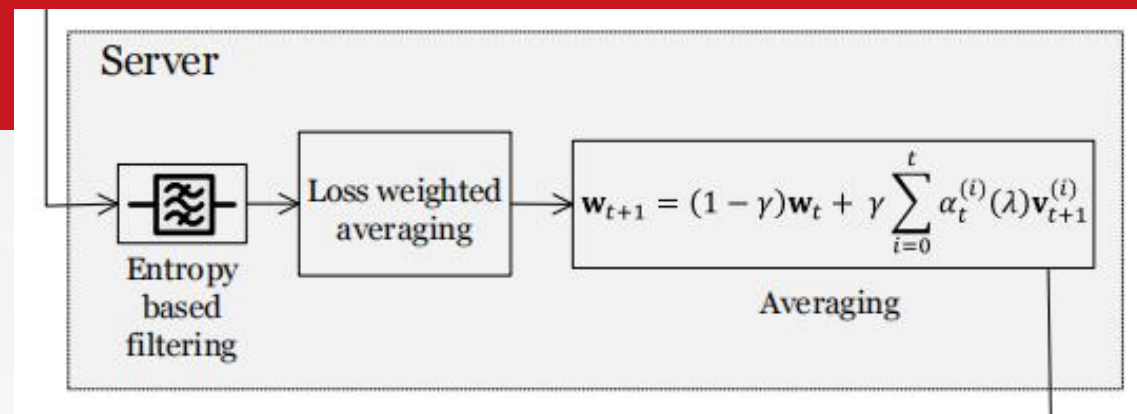




Sageflow algorithm

Entropy-based filtering

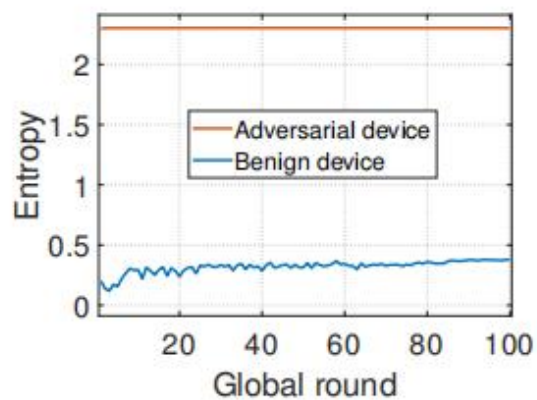
- **Public data** on server
- **Filter out high entropy** models (loss)
- For model poisoning



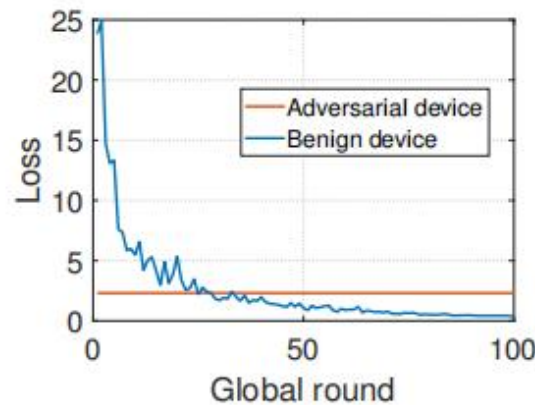
$$E_{x_{pub,j}}(k) = - \sum_{q=1}^Q P_{x_{pub,j}}^{(q)}(k) \log P_{x_{pub,j}}^{(q)}(k)$$

$$E(k) = \frac{1}{n_{pub}} \sum_{j=1}^{n_{pub}} E_{x_{pub,j}}(k)$$

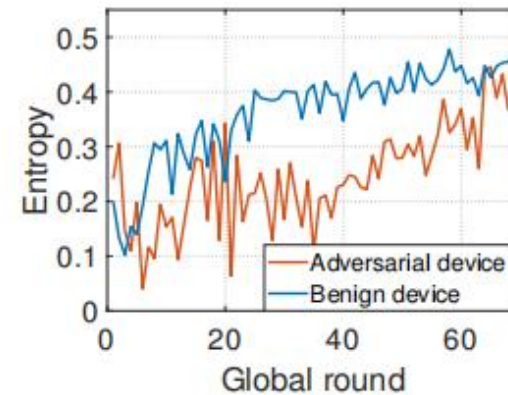
Shannon entropy



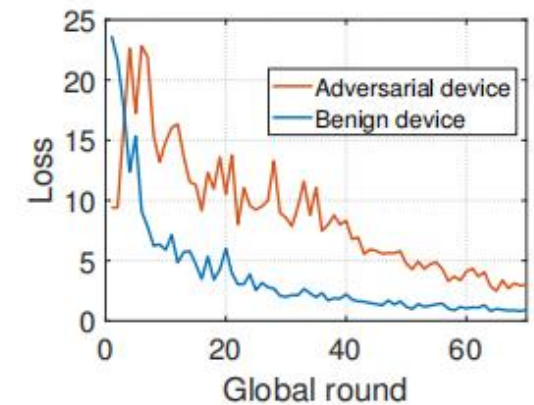
(a) Model poison, entropy



(b) Model poison, loss



(c) Data poison, entropy



(d) Data poison, loss

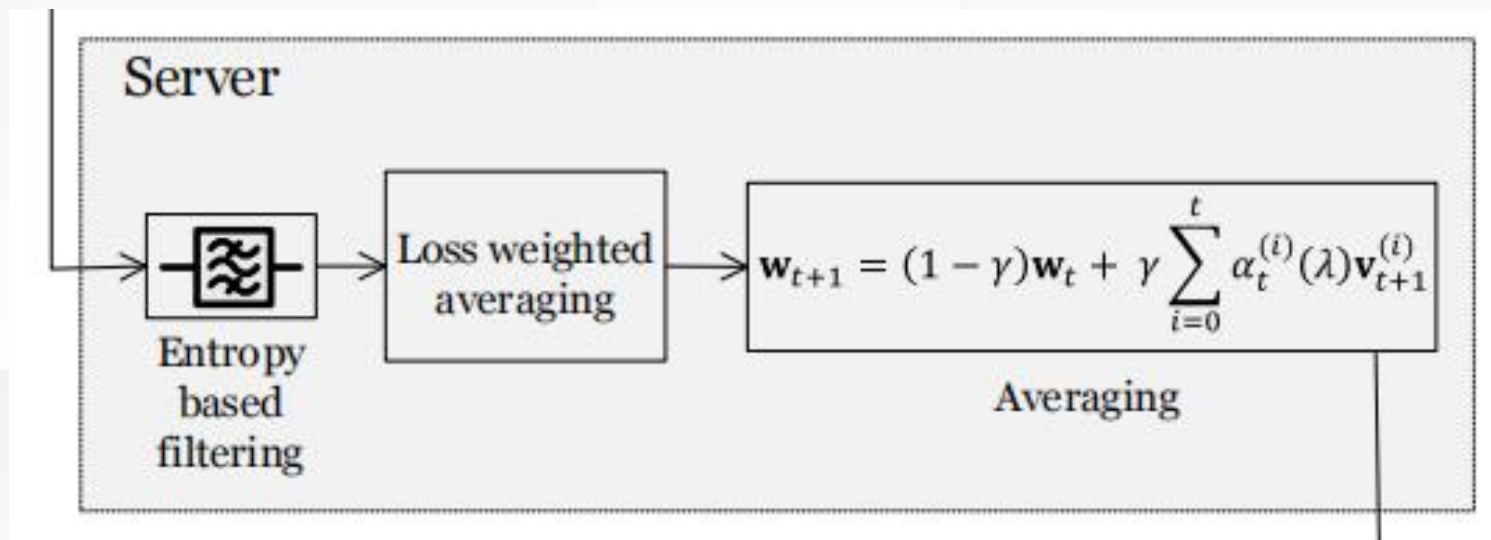


Sageflow algorithm

- Loss-weighted averaging
 - Aggregation weight according to local models' **measured qualities**
 - Measure by **loss** on public data
 - data-poisoned model -> small weight + less impact
 - For data poisoning & scaled backdoor

$$\beta_t^{(k)}(\delta) \propto \frac{m_k}{\{F_{pub}(\mathbf{w}_t(k))\}^\delta} \text{ and } \sum_{k \in S_t} \beta_t^{(k)}(\delta) = 1.$$

$$\mathbf{w}_{t+1} = \sum_{k \in S_t} \beta_t^{(k)}(\delta) \mathbf{w}_t(k)$$





Sageflow algorithm



Time complexity

Public
data

Model
parameters

Clients
number

$$\mathcal{O}(n_{pub}|w|K)$$

Algorithm 1 Proposed Sageflow Algorithm

Input: Initialized model $\mathbf{w}_0$, **Output:** Final global model $\mathbf{w}_T$

Process at the Server

- 1: **for** each global round $t = 0, 1, \dots, T - 1$ **do**
- 2: Choose S_t and send the current model and the global round $(\mathbf{w}_t, t)$ to the devices
- 3: Wait for T_d and then:
- 4: **for** $i = 0, 1, \dots, t$ **do**
- 5: $U_t^{(i)}(E_{th}) = \{k \in U_t^{(i)} | E(k) < E_{th}\}$ // Entropy-based filtering in each group
- 6: $\mathbf{v}_{t+1}^{(i)} = \sum_{k \in U_t^{(i)}(E_{th})} \beta_t^{(k)}(\delta) \mathbf{w}_i(k)$ // Loss-weighted averaging in each group (with same staleness)
- 7: **end for**
- 8: $\mathbf{w}_{t+1} = (1 - \gamma) \mathbf{w}_t + \gamma \sum_{i=0}^t \alpha_t^{(i)}(\lambda) \mathbf{v}_{t+1}^{(i)}$ // Averaging of representative models (with different staleness)
- 9: **end for**

Process at the Device: Device k receives $(\mathbf{w}_t, t)$ from the server and performs local updates to obtain $\mathbf{w}_t(k)$. Then each benign device k sends $(\mathbf{w}_t(k), t)$ to the server, while a malicious adversary sends a poisoned model depending on the type of attack.



Sageflow algorithm

Theoretical Analysis

Convergence analysis

- Assumption 1: μ -strongly convex + L-smooth
- Assumption 2: unbiased estimation

Theoretical bound

$$F(x) \leq F(y) + \nabla F(x)^T(x-y) - \frac{\mu}{2}\|x-y\|^2$$

$$F(x) \geq F(y) + \nabla F(x)^T(x-y) - \frac{L}{2}\|x-y\|^2$$

$$\mathbb{E}\|\nabla \bar{F}_k(\mathbf{w}_t^i(k), \xi_t^i(k)) - \nabla F(\mathbf{w}_t^i(k))\|^2 \leq \rho_1$$

$$\mathbb{E}[F(\mathbf{w}_T) - F(\mathbf{w}^*)] \leq \underbrace{\nu^T}_{\text{Convergence speed}} [F(\mathbf{w}_0) - F(\mathbf{w}^*)] + \underbrace{(1 - \nu^T)Z(\lambda, E_{th}, \delta)}_{\text{Error}}$$

Convergence speed

Error



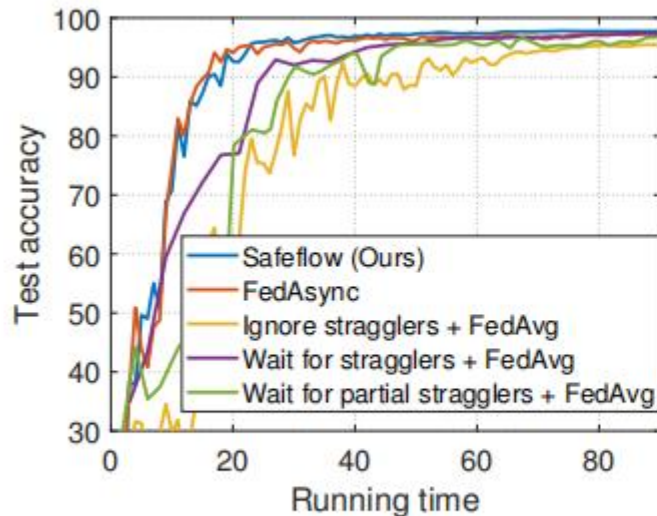
Sageflow algorithm



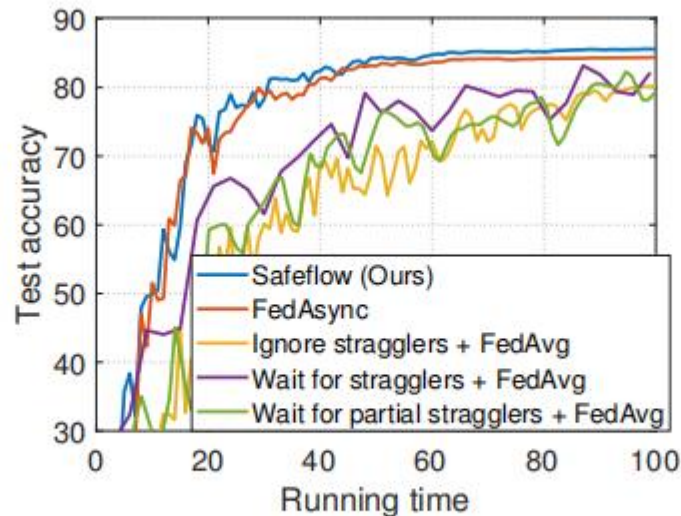
- ④ Datasets: MNIST, FMNIST, CIFAR10
 - 2% as public data
- ④ Models: CNN(2conv+2fc), CNN(2conv+1fc), CNN(VGG-11)
 - ignore batchnorm
- ④ FL setting: 100 clients, two classes for each client, 5 local epochs, batch size of 10

Sageflow algorithm

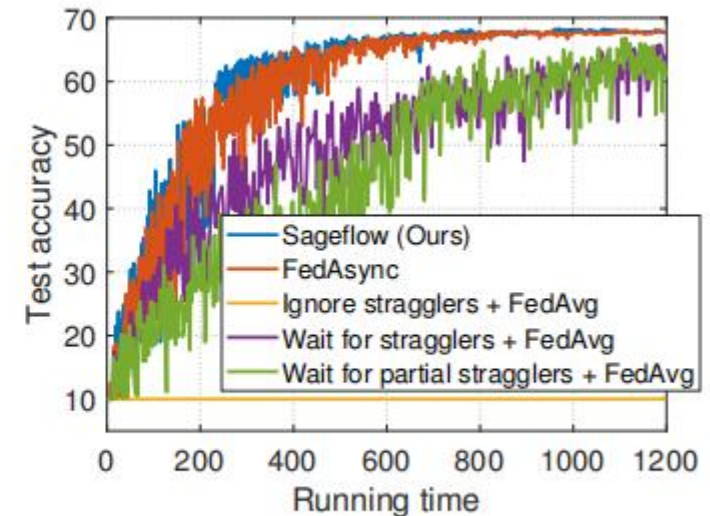
- Only stragglers: 10% participants
- Baselines: FedAvg(waiting, ignoring, waiting 50%), FedAsync
- Settings: uniform delay of [0,1,2] global rounds
- Ignoring lose significant data converges to a **suboptimal point**
- Waiting(all, 50%) requires the **largest running time** until convergence



(a) MNIST



(b) FMNIST

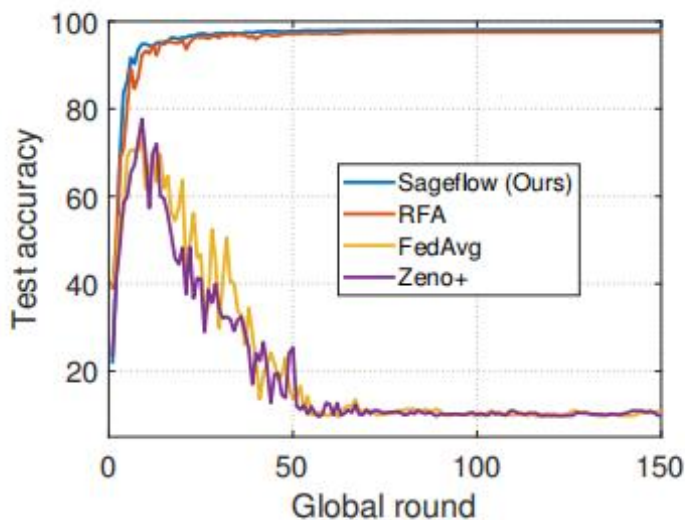


(c) CIFAR10

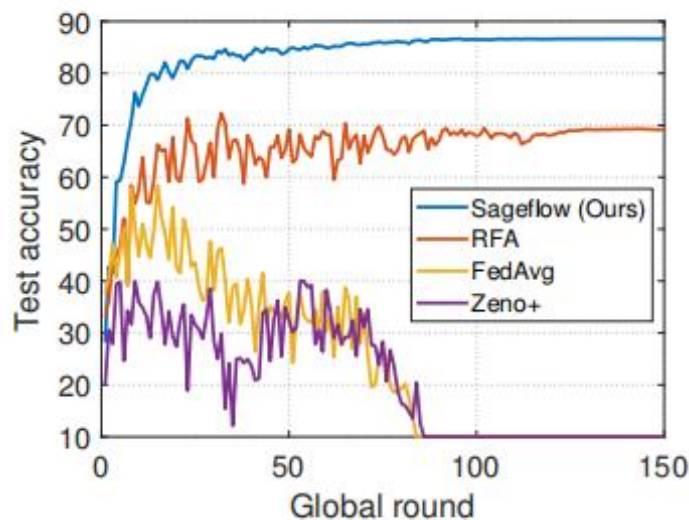


Sageflow algorithm

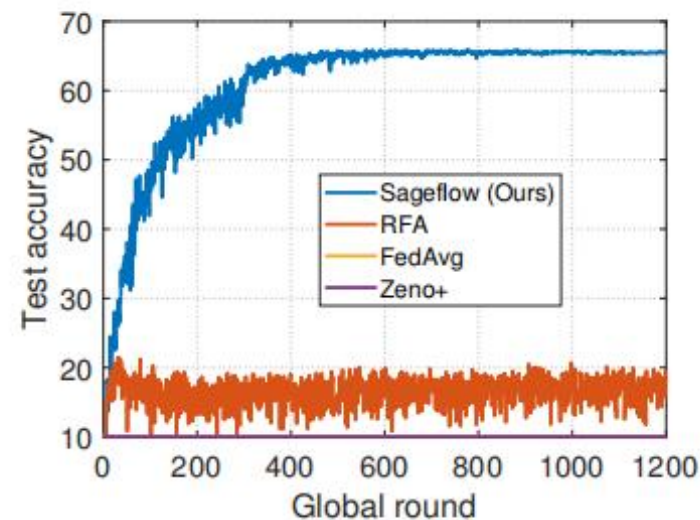
- Only adversaries: 20% participants
- Baselines: RFA, FedAvg, synchronized Zeno+, Multi-Krum
- Attacks: model(-0.1w), data(label-flipping), backdoor(model replacement, pixel-pattern attack)
- FedAvg **does not work well** on all datasets
- Zeno+: bad on poisoning **but** good for backdoor
- RFA: **complex model** led to worse performance
- Sageflow: **slow down** poisoning



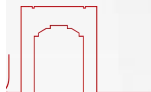
(a) MNIST



(b) FMNIST



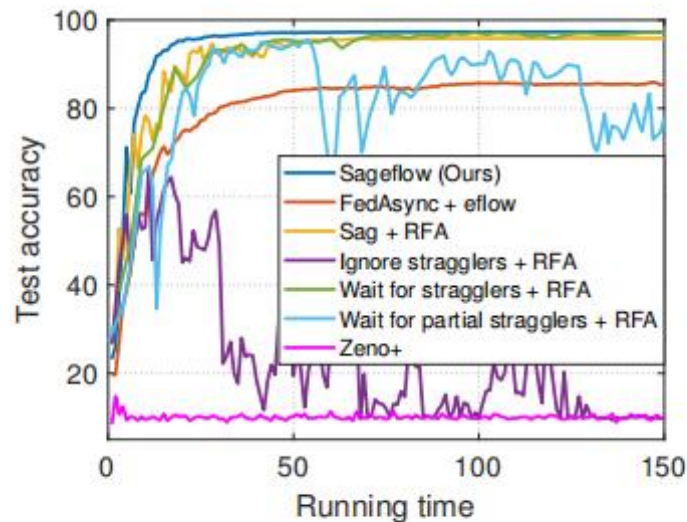
(c) CIFAR10



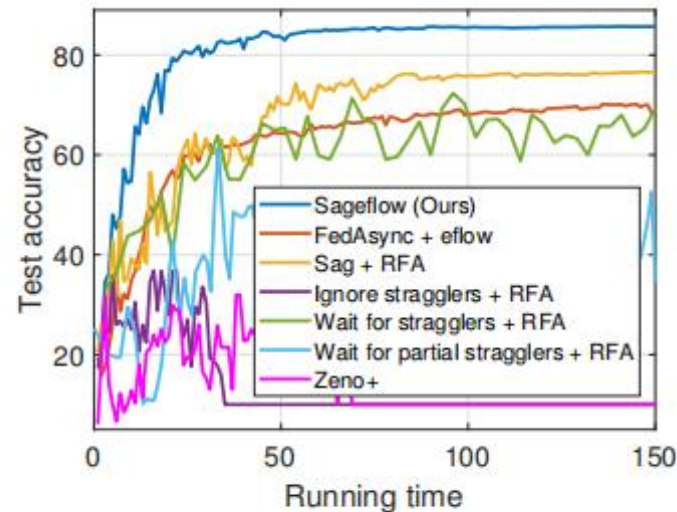


Sageflow algorithm

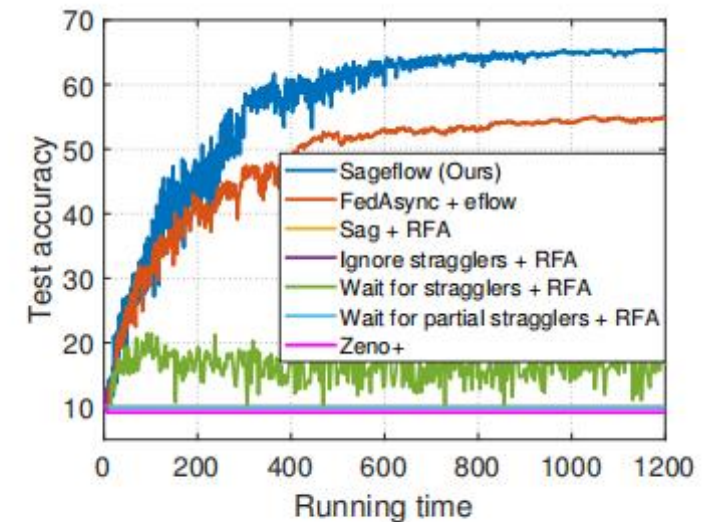
- ⊗ Stragglers + adversaries: 20%(model/data), 10%(backdoor) participants
- ⊗ Baselines: asynchronized Zeno+, Multi-Krum
- ⊗ Zeno+: **does not perform well**(ignore staleness & entropy)
- ⊗ Waiting + RFA: suffer from **straggler**
- ⊗ Ignoring/Sag + RFA: poor(**high attack ratio**) ⊗ eflow/RFA + FedAsync: poor(**one-by-one arrivals**)



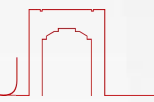
(a) MNIST



(b) FMNIST



(c) CIFAR10





Sageflow algorithm



① Sageflow: robust FL scheme handle both **stragglers and adversaries**

- staleness-aware grouping: stragglers
- entropy-based filtering: model poisoning
- loss-weighted averaging: data poisoning + backdoor

② Theoretical convergence analysis

③ Extensive experimental results

④ Future issues: Sageflow + secure aggregation

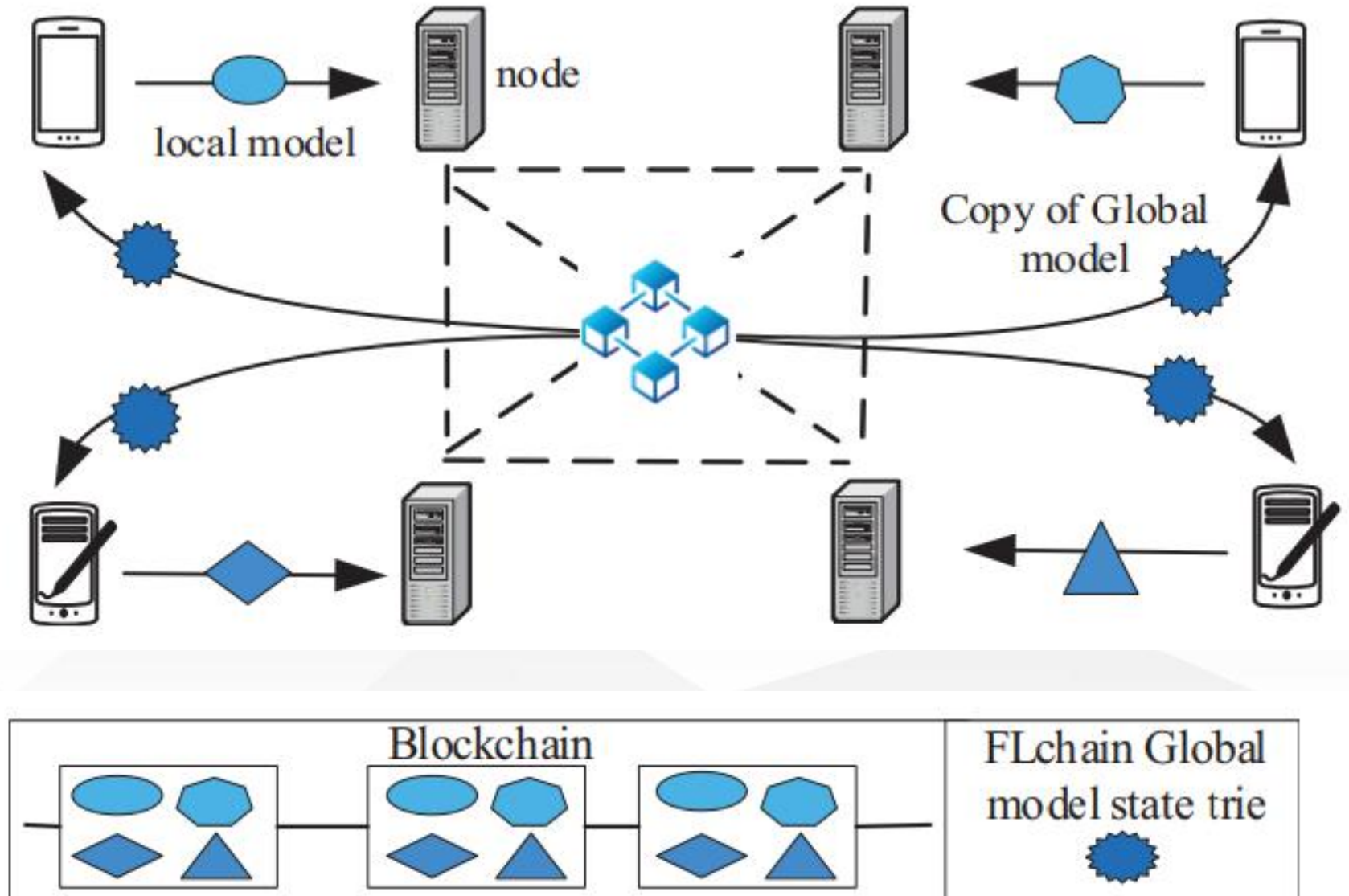


FLChain



FL + Blockchain

- complete dependency on the reliability of a central server for storage and computation of the global model update
- Any malicious activity leads to flawed global model update which is detrimental for accuracy of subsequent local model updates



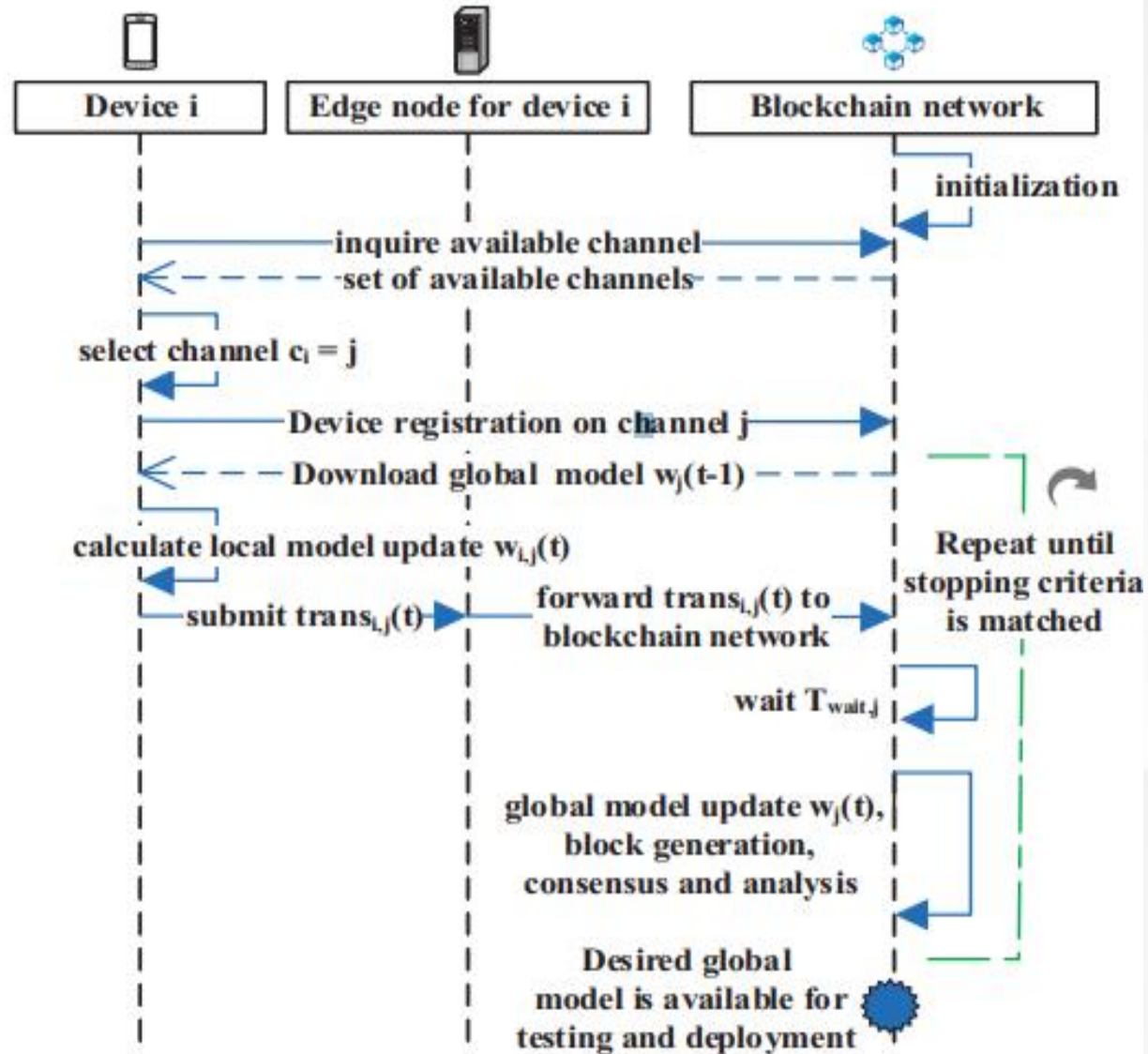


- ① Initialization
- ② Channel Inquiry
- ③ Channel Selection
- ④ Device Registration
- ⑤ Local Model Update
- ⑥ Transaction Pool
- ⑦ Global Model Update
- ⑧ Consensus Protocol
- ⑨ Analysis

Algorithm 1 : FLchain operation for channel j

```
1: Setup channel  $j$  for global model  $M_j$ 
2: initialization:  $t = 0$ ;  $w_j(0), w_{i,j}(0) \in [0, w_{j,max}]$ ;
3: for all  $i \in D_j$  do in parallel
4:   Inquire available channels
5:   Select channel  $c_i = j \in C \triangleq \{1, 2, 3, \dots, C_n\}$ 
6:   Register device  $i$  to channel  $j$ 
7: end for
8: while  $\|w_j(t) - w_j(t-1)\|_2 \leq \varepsilon_{threshold,j}$  do
9:   for all  $i \in D_j$  do in parallel
10:    Download  $w_j(t)$  from channel  $j$  to device  $i$ 
11:     $t \leftarrow t + 1$ ;  $w_{i,j}^0(t) = w_j(t)$ ;
12:    for  $v = 1, \dots, V$  do
13:       $w_{i,j}^v(t) = w_{i,j}^{v-1}(t) - \eta \nabla \Phi$ , and Eq. (6)
14:    end for
15:     $w_{i,j}(t) = w_{i,j}^V(t)$ , Generate  $trans_{i,j}(t)$  and forward to blockchain network
16:    Wait for notification from channel  $j$ 
17:  end for
18:  Calculate  $w_j(t)$  using Eq. (7)
19:  global model state trie updation, block generation and consensus
20: end while
```



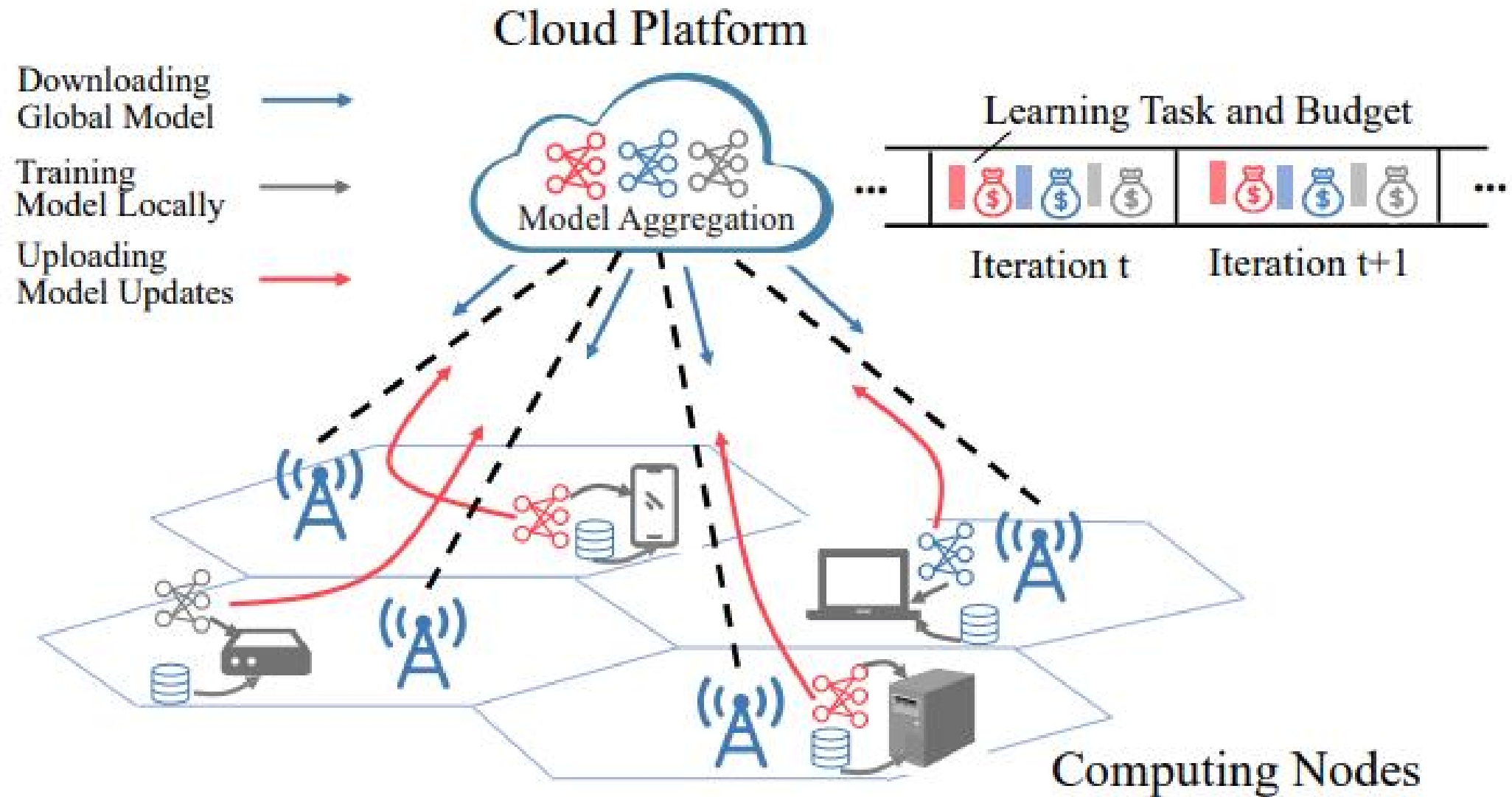




FAIR



Federated Learning





⊗ Problems

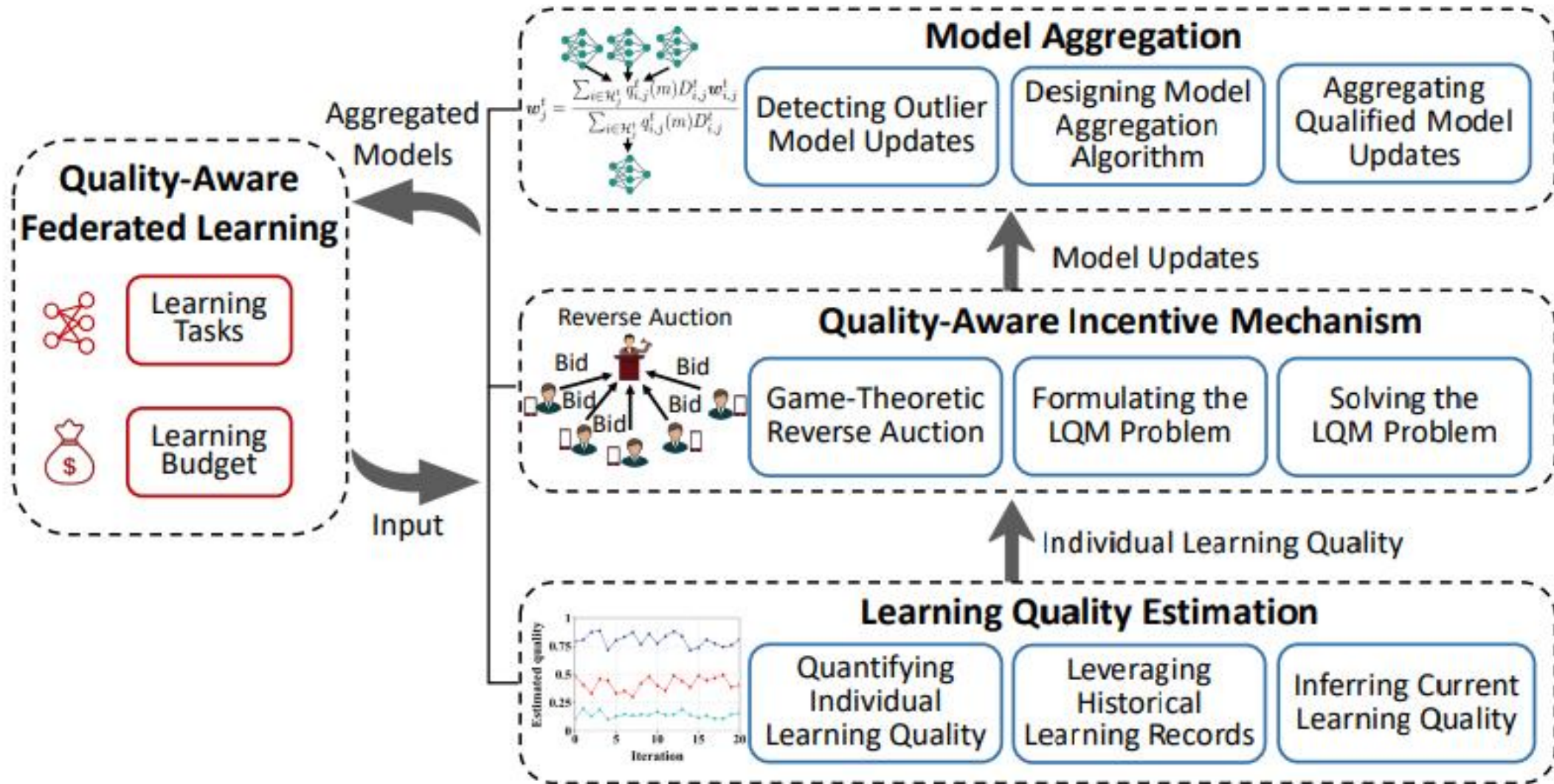
- mobile users do not participate in learning due to computation and energy consumption
- model update quality of mobile devices can vary dramatically, inclusively aggregating low-quality model updates can deteriorate the global model quality

⊗ Solution: Federated leArning with quality awaReness

- learning quality estimation: leverage historical learning records to estimate the user learning quality, the exponential forgetting function is utilized for weight assignment;
- quality-aware incentive mechanism: within the recruiting budget, model a reverse auction problem to encourage the participation of high-quality learning users;
- model aggregation: integrates the model quality into aggregation and filters out non-ideal model updates, to further optimize the global learning model



FAIR Architecture





Estimating Learning Quality

$$m_{i,j}^t = \text{loss}_j(t_s) - \text{loss}_{i,j}(t_e).$$

$$q_{i,j}^t = \begin{cases} m_{i,j}^t D_{i,j}^t, & \text{if } t_s < t_{i,j}^u \leq t_e; \\ 0, & \text{otherwise.} \end{cases}$$

$$\hat{q}_{i,j}^t = \frac{\sum_{k=0}^r \rho^{t_r - t_k} q_{i,j}^{t_k}}{\sum_{k=0}^r \rho^{t_r - t_k}}.$$

Quality-Aware Incentive Mechanism

Algorithm 1: Solving the LQM problem.

Input : (1) bid price $b_{i,j}^t$; (2) budget B_j^t ; (3) task set $\mathcal{L}_i^t$;
(4) quality estimation values $\hat{q}_{i,j}^t$.

Output : (1) the task allocation results $s_{i,j}^t$; (2) the payments $r_{i,j}^t$.

```

1 Initialize  $\mathcal{N}_j^t \leftarrow \emptyset$ ,  $p_j^t \leftarrow 0$  for each  $l_j^t \in \mathcal{L}^t$ ;
2 Initialize  $x_i^t \leftarrow 1$  for each  $i \in \mathcal{N}$ ;
3 Initialize  $r_{i,j}^t \leftarrow 0$ ,  $s_{i,j}^t \leftarrow 0$  for each  $i \in \mathcal{N}$ ,  $l_j^t \in \mathcal{L}^t$ ;
4 foreach  $i \in \mathcal{N}$  do
5   foreach  $l_j^t \in \mathcal{L}_i^t$  do
6      $\mathcal{N}_j^t \leftarrow \mathcal{N}_j^t + \{i\}$ ;
7   end
8 end
9 while  $\exists x_i^t = 0$  and  $\exists p_j^t = 0$  do
10   Initialize  $\mathcal{M}_j^t \leftarrow \emptyset$  for each  $l_j^t \in \mathcal{L}^t$ ;
11   foreach  $l_j^t \in \mathcal{L}^t$  do
12     if  $p_j^t = 0$  then
13       Sort all  $i \in \mathcal{N}_j^t$  in descending order of  $\frac{q_{i,j}^t}{b_{i,j}^t}$ ;
14       Find the smallest  $k$  such that
15          $\sum_{i=1}^k \frac{b_{k,j}^t}{q_{k,j}^t} q_{i,j}^t x_i^t > B_j^t$ ;
16       for  $i \leftarrow 1$  to  $k-1$  do
17          $\mathcal{M}_j^t \leftarrow \mathcal{M}_j^t + \{i\}$ ;
18          $r_{i,j}^t \leftarrow \frac{b_{k,j}^t}{q_{k,j}^t} q_{i,j}^t$ ;
19       end
20     end
21   end
22   Find the task  $l_k^t$  with maximum  $\sum_{i \in \mathcal{M}_k^t} q_{i,k}^t x_i^t$ ;
23   Set  $p_k^t \leftarrow 1$ ;
24   foreach  $i \in \mathcal{M}_k^t$  do
25     if  $x_i^t = 1$  then
26        $s_{i,k}^t \leftarrow 1$ ;
27        $x_i^t \leftarrow 0$ ;
28     end
29   end
30 return  $(s_{i,j}^t, r_{i,j}^t)$ ;

```

The defined LQM problem can be formulated as

$$\max_{\mathcal{M}_j^t, \mathcal{R}_j^t} \sum_{l_j^t \in \mathcal{L}^t} \sum_{i \in \mathcal{M}_j^t} \hat{q}_{i,j}^t, \quad (10)$$

$$\text{s.t.} \quad \sum_{i \in \mathcal{N}} r_{i,j}^t s_{i,j}^t \leq B_j^t, \quad \forall l_j^t \in \mathcal{L}^t, \quad (11)$$

$$r_{i,j}^t \geq b_{i,j}^t, \quad \forall l_j^t \in \mathcal{L}^t, \forall i \in \mathcal{M}_j^t, \quad (12)$$

$$s_{i,j}^t \in \{0, 1\}, \quad \forall i \in \mathcal{N}, \forall l_j^t \in \mathcal{L}^t, \quad (13)$$

$$s_{i,j}^t = 0, \quad \forall l_j^t \notin \mathcal{L}_i^t, \forall i \in \mathcal{N}, \quad (14)$$

$$\sum_{l_j^t \in \mathcal{L}^t} s_{i,j}^t \leq 1, \quad \forall i \in \mathcal{N}. \quad (15)$$

```

19   end
20 end
21 Find the task  $l_k^t$  with maximum  $\sum_{i \in \mathcal{M}_k^t} q_{i,k}^t x_i^t$ ;
22 Set  $p_k^t \leftarrow 1$ ;
23 foreach  $i \in \mathcal{M}_k^t$  do
24   if  $x_i^t = 1$  then
25      $s_{i,k}^t \leftarrow 1$ ;
26      $x_i^t \leftarrow 0$ ;
27   end
28 end
29 end
30 return  $(s_{i,j}^t, r_{i,j}^t)$ ;

```



Model Aggregation

$$w_j^t = \frac{\sum_{i=1}^N D_{i,j}^t w_{i,j}^t}{\sum_{i=1}^N D_{i,j}^t},$$

$$w_j^t = \frac{\sum_{i \in \mathcal{M}_j^t} m_{i,j}^t D_{i,j}^t w_{i,j}^t}{\sum_{i \in \mathcal{M}_j^t} m_{i,j}^t D_{i,j}^t},$$

Algorithm 2: Model aggregation algorithm.

Input : (1) winner node set $\mathcal{M}_j^t$; (2) local model parameters $w_{i,j}^t$; (3) data size used for training $D_{i,j}^t$; (4) data quality $m_{i,j}^t$.

Output : aggregated model parameters w_j^t .

- 1 Initialize high-quality node set $\mathcal{H}_j^t \leftarrow \{i : i \in \mathcal{M}_j^t\}$;
- 2 Set $D \leftarrow \sum_{i \in \mathcal{M}_j^t} m_{i,j}^t D_{i,j}^t$;
- 3 Set $w_j^t \leftarrow \sum_{i \in \mathcal{M}_j^t} \frac{m_{i,j}^t D_{i,j}^t}{D} w_{i,j}^t$;
- 4 **foreach** $i \in \mathcal{M}_j^t$ **do**
- 5 | Compute $d_{i,j}^t \leftarrow \text{similarity}(w_j^t, w_{i,j}^t)$;
- 6 **end**
- 7 Compute $\bar{\mu}_d, \hat{\mu}_d, \sigma_d$;
- 8 **if** $\bar{\mu}_d > \hat{\mu}_d$ **then**
- 9 | **foreach** $i \in \mathcal{H}_j^t$ **do**
- 10 | **if** $d_{i,j}^t > \hat{\mu}_d + \eta \sigma_d$ **then**
- 11 | $\mathcal{H}_j^t \leftarrow \mathcal{H}_j^t - \{i\}$;
- 12 | **end**
- 13 | **end**
- 14 **else**
- 15 | **foreach** $i \in \mathcal{H}_j^t$ **do**
- 16 | **if** $d_{i,j}^t < \hat{\mu}_d - \eta \sigma_d$ **then**
- 17 | $\mathcal{H}_j^t \leftarrow \mathcal{H}_j^t - \{i\}$;
- 18 | **end**
- 19 | **end**
- 20 **end**
- 21 Set $D \leftarrow \sum_{i \in \mathcal{H}_j^t} m_{i,j}^t D_{i,j}^t$;
- 22 Set $w_j^t \leftarrow \sum_{i \in \mathcal{H}_j^t} \frac{m_{i,j}^t D_{i,j}^t}{D} w_{i,j}^t$;
- 23 **return** w_j^t ;



Theoretical

- Truthful
- Individually rational
- Computational Efficiency

Truthfulness 每一个报价都必须是真的，等于自身的cost，如果不等于，则utility会变小

Individual Rationality 每一轮迭代的utility都大于等于0

Computational Efficiency 所有算法的执行时间为多项式时间



Settings

- Models: MLP, LeNet, MobileNet, VGG-11, EfficientNet-B0, ResNet-18
- Dataset: MNIST, Fashion-MNIST, CIFAR-10, Street View House Numbers
- Baselines: Theoretically optimal mechanism, Knapsack greedy mechanism, Bid price first mechanism
- Hyperparameters

Settings	$ \mathcal{N} $	$ \mathcal{L}^t $	$b_{i,j}^t$	$D_{i,j}^t$	N_e	R_e	B_j^t
I	20	1	[1,3]	[1000,3000]	5	[0.05,0.5]	5
II	20	1	[1,3]	5000	[0,20]	0.5	10
III	100	4	[1,10]	[1000,10000]	30	[0,1]	[10,30]

- **Theoretically optimal mechanism:** It adopts the depth-first search approach to find the theoretically optimal solution for the LQM problem, which however cannot guarantee the truthfulness of nodes.
- **Knapsack greedy mechanism:** It greedily selects winner nodes based on the amount of data used for training divided by the bid price, i.e., $D_{i,j}^t/b_{i,j}^t$, where the data quality and truthfulness of nodes are not considered.
- **Bid price first mechanism:** It preferentially selects nodes with the lowest bid price without guaranteeing the truthfulness of nodes.



Evaluation

④ Different incentive mechanisms

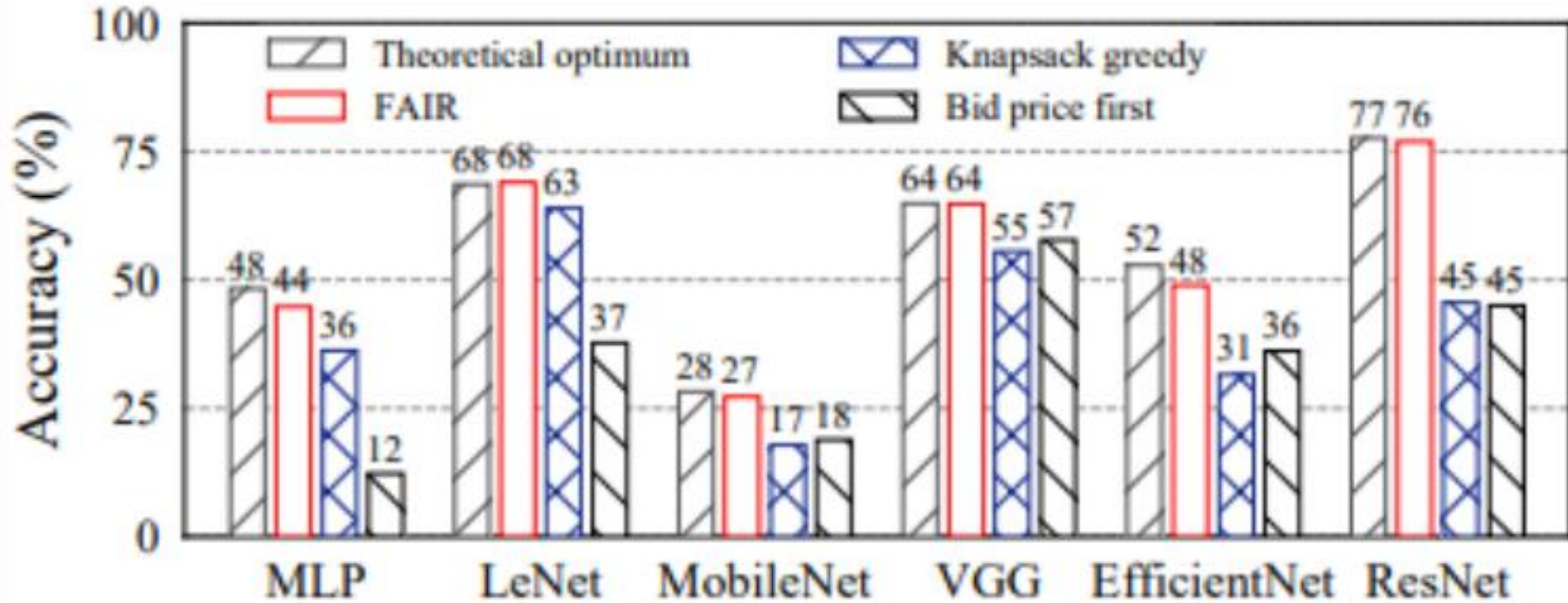


Fig. 4. The average model accuracy with different incentive mechanisms.



Attack scenarios

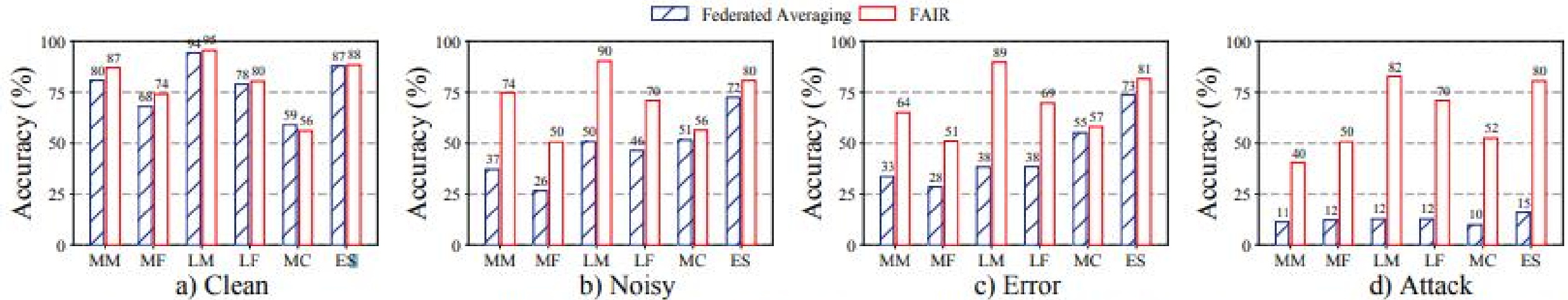
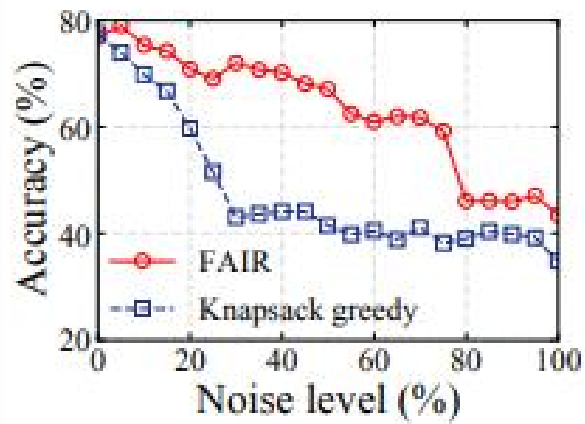


Fig. 3. The model aggregation performance of MLP-MNIST (MM), MLP-FMNIST (MF), LeNet-MNIST (LM), LeNet-FMNIST (LF), MobileNet-CIFAR10 (MC), EfficientNet-SVHN (ES) under different scenarios: a) Clean datasets; b) Noisy label datasets; c) Error label datasets; d) Attack datasets.

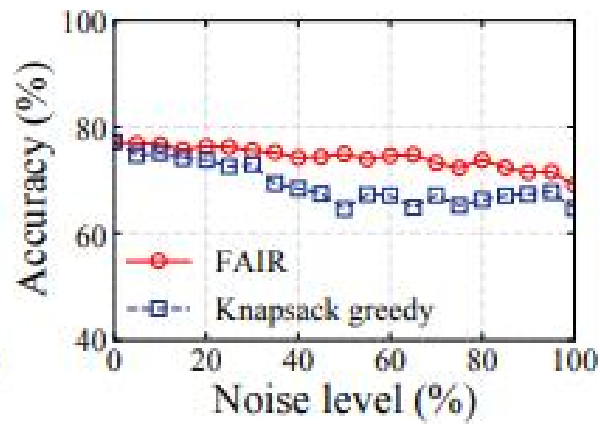


Evaluation

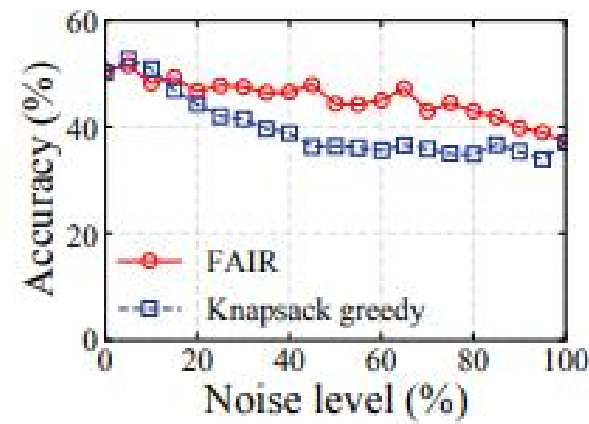
⊗ Noise levels



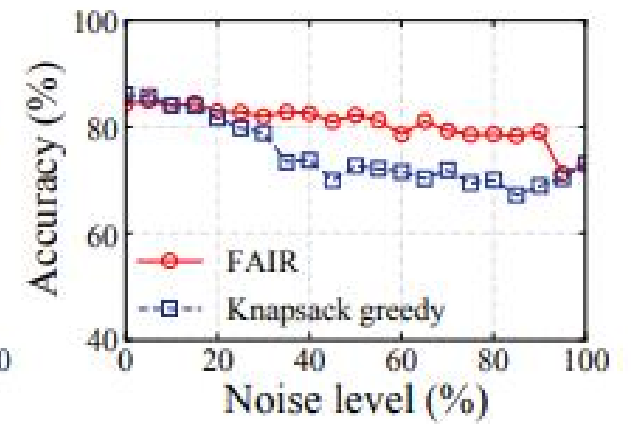
a) MLP MNIST



b) LeNet FMNIST



c) MobileNet CIFAR10



d) EfficientNet SVHN

Fig. 5. The model accuracy vs. noise levels.



Evaluation



Learning budgets

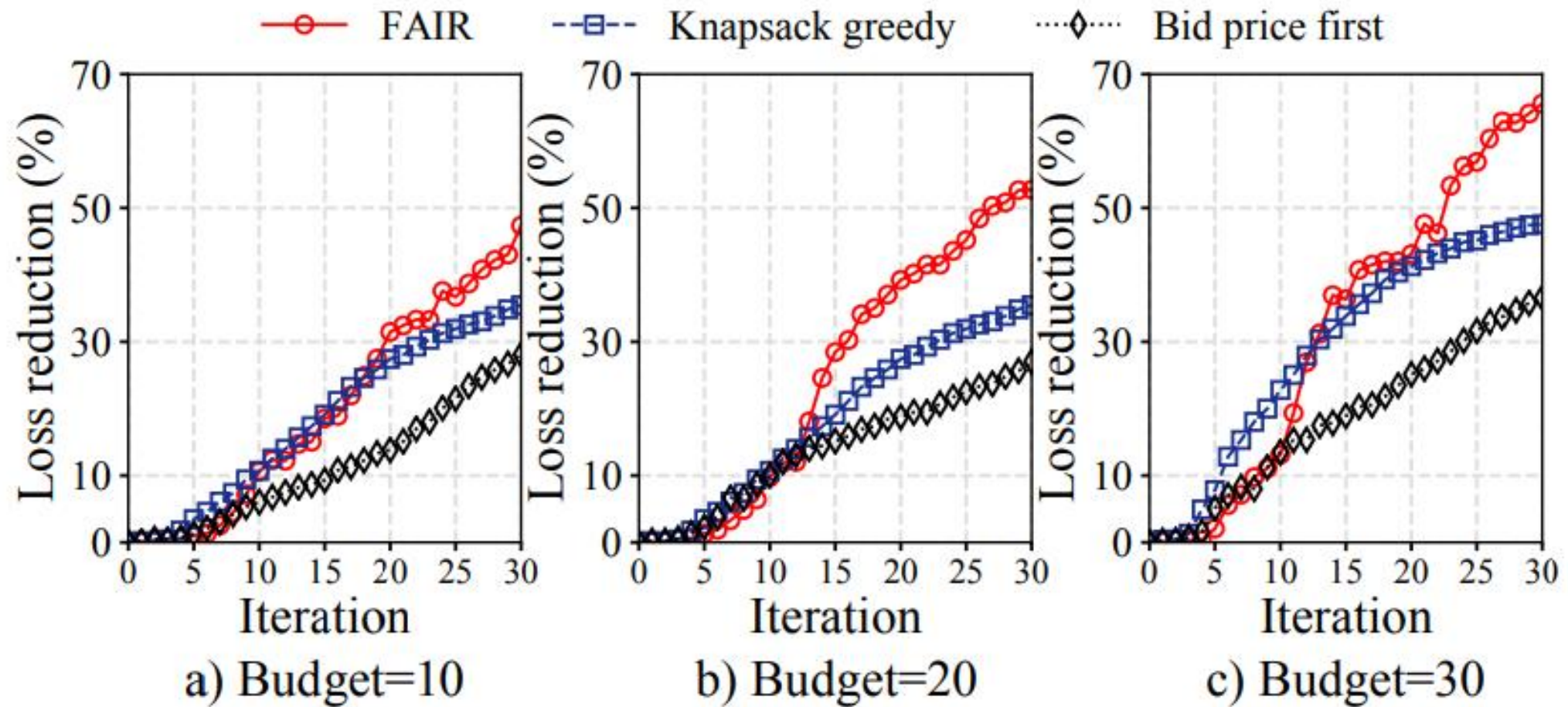


Fig. 6. The average test loss reduction of learning models vs. learning budgets.

04

Other algorithms

- 联邦学习效率
- 联邦学习安全
- 联邦学习异构性
- 其他论文



联邦学习效率

- 数据传输量大 - 模型压缩
- 网络环境复杂 - 传输调度



联邦学习效率-模型压缩

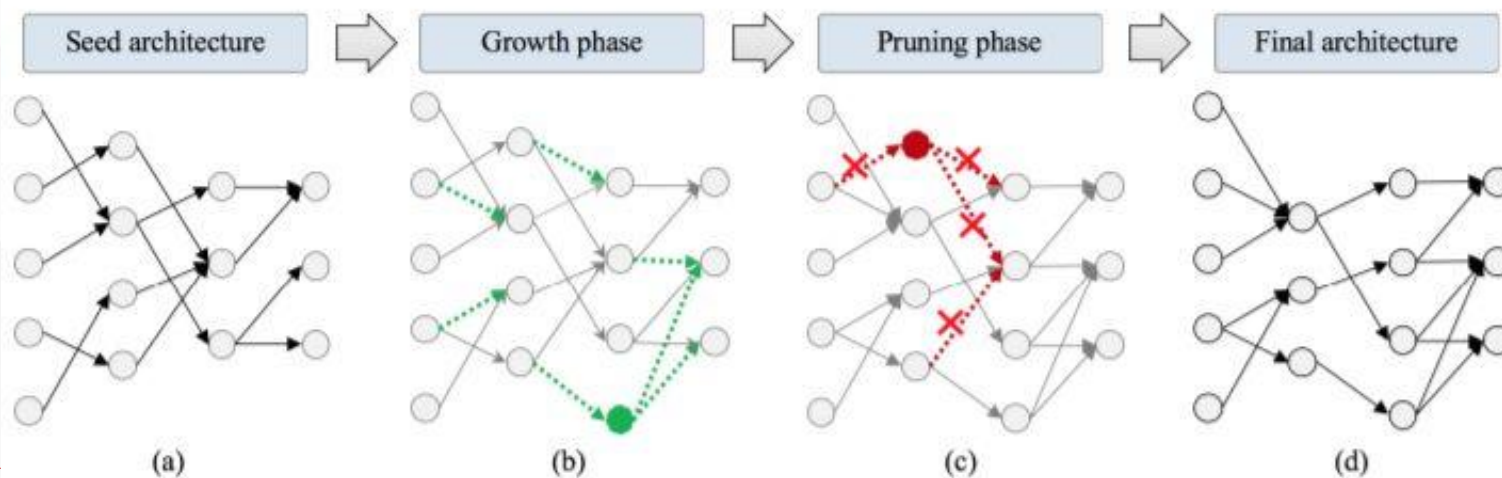
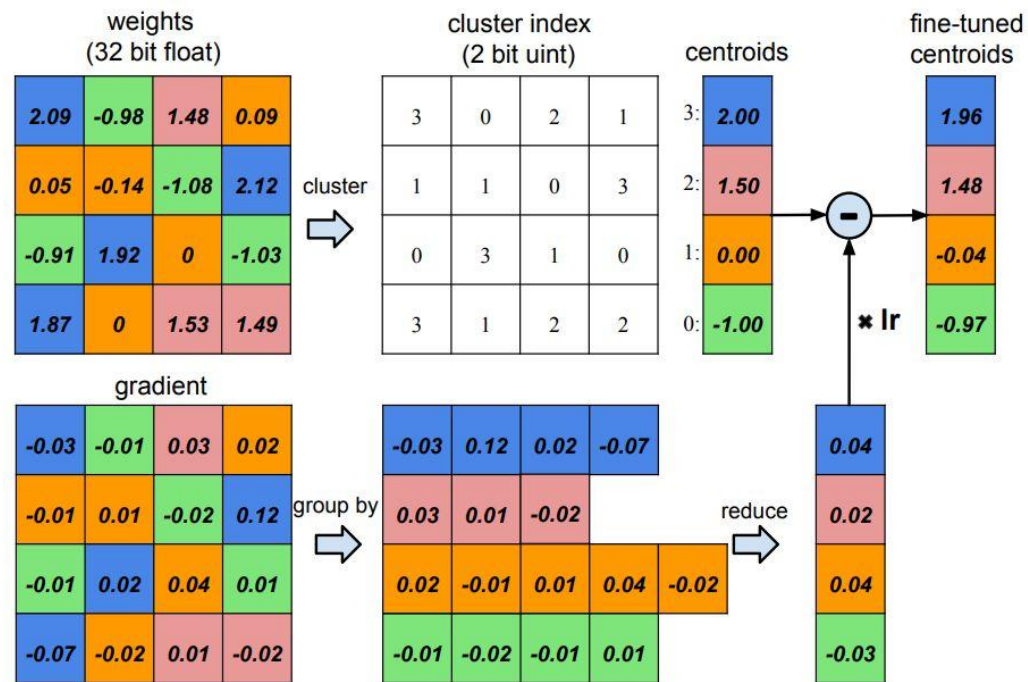
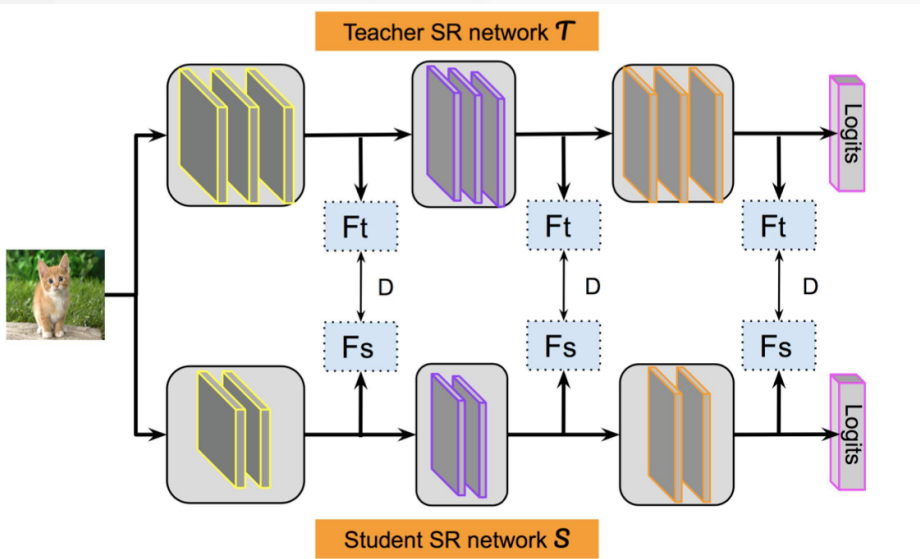


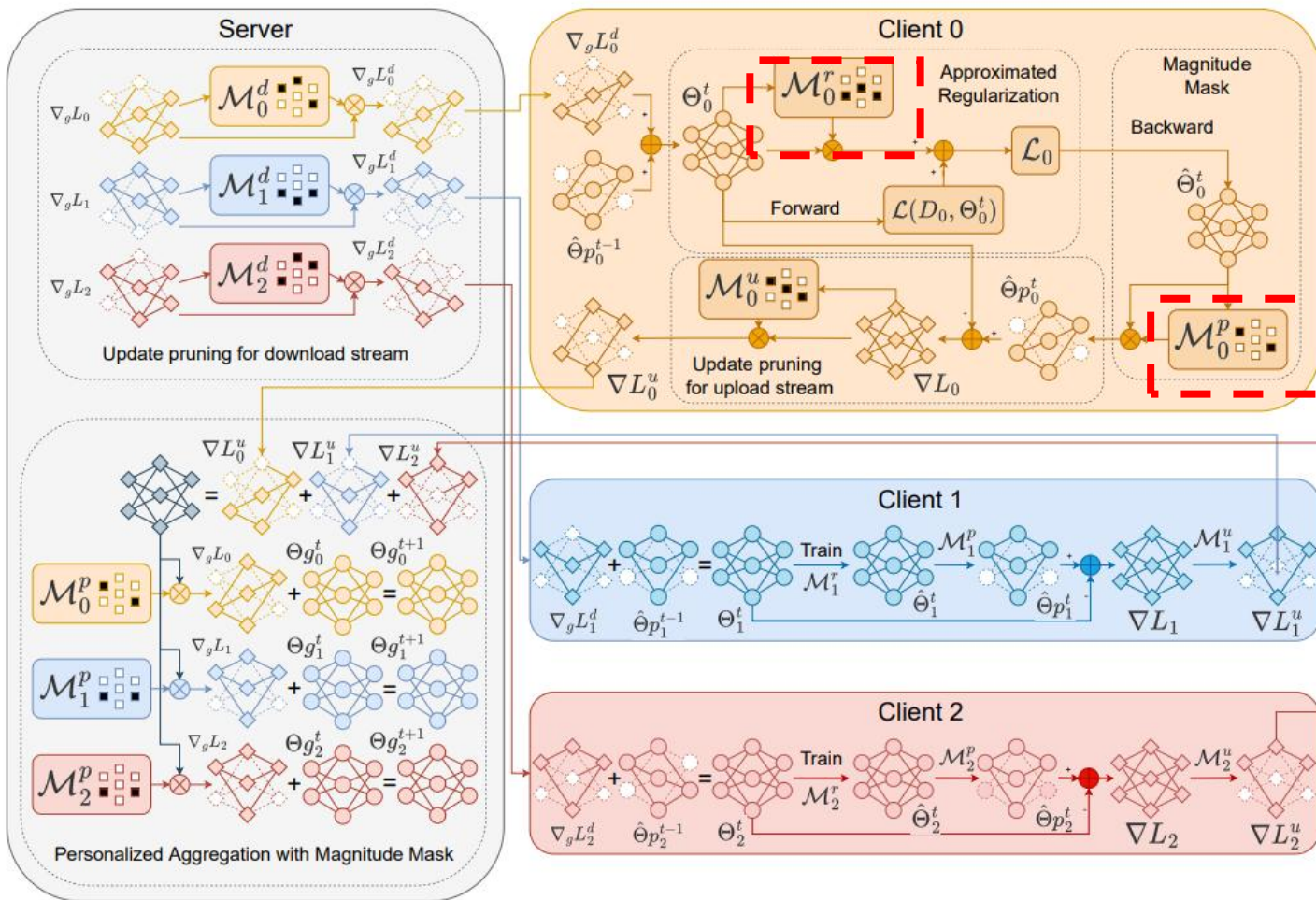
通信效率

- 智能算法本身通信量大，通信开销大

模型压缩：模型剪枝，模型量化，知识蒸馏

- 减少数据通信、计算量
- 构造准确但小的神经网络模型





绝对值剪枝

$$\mathcal{H}(w) = |w|.$$

损失函数正则化

定位对推理结果影响较低的参数

$$\mathcal{L}_i = l(D_i, \Theta_i^t) + \lambda_1 \cdot ||\Theta_i^t \odot \mathcal{M}_i^r||_1.$$

训练后模型剪枝

过滤对推理结果影响较低的参数

$$\hat{\Theta}_{p_i}^t \leftarrow \hat{\Theta}_i^t \odot \mathcal{M}_i^p.$$



重要性剪枝

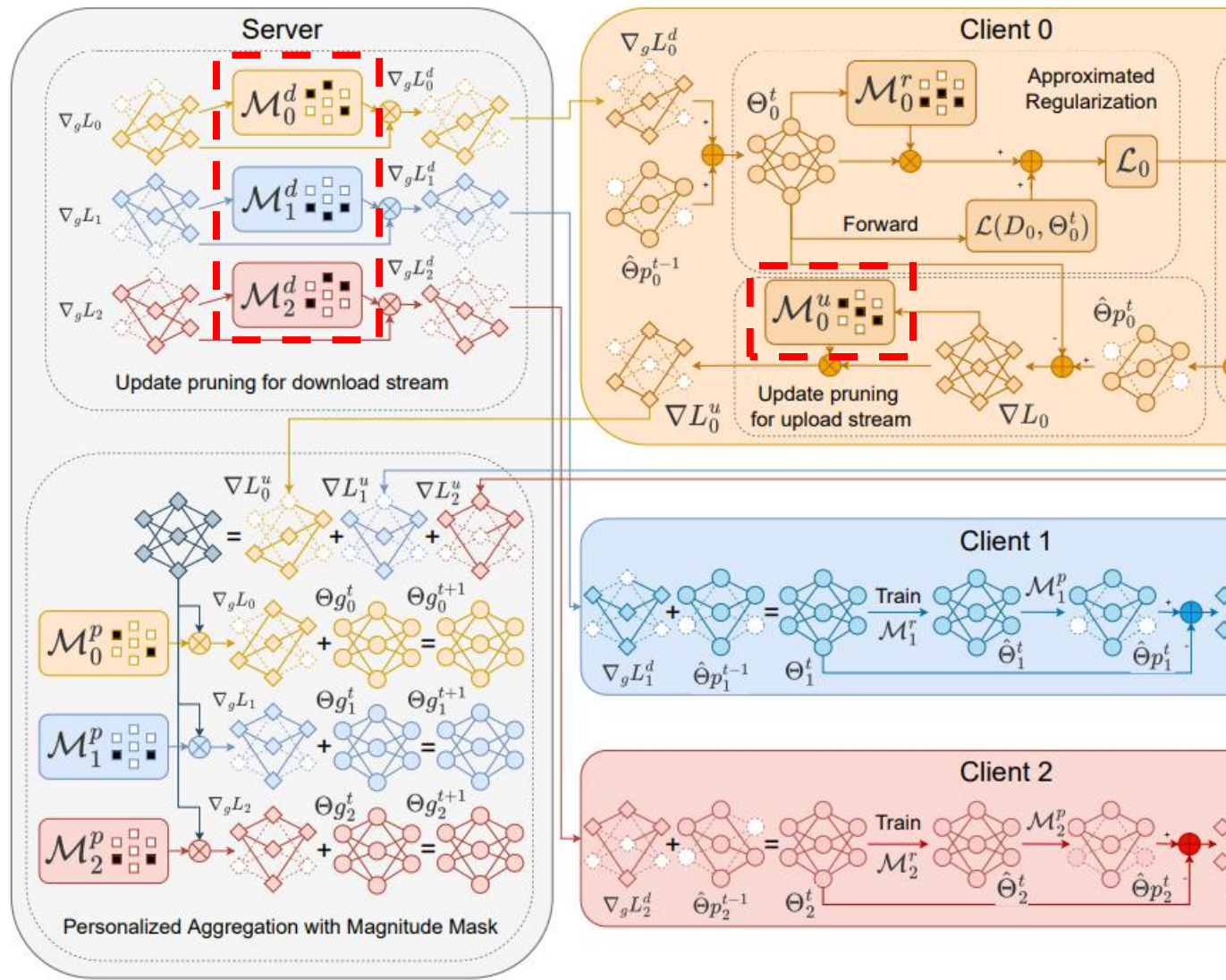
$$\begin{aligned}\mathcal{I}(w) &= |\mathcal{L}(D, \Theta_{w \rightarrow 0}) - \mathcal{L}(D, \Theta)| \\ &= \left| \frac{\partial \mathcal{L}(D, \Theta)}{\partial w} \cdot (0 - w) + o(w^2) \right| \\ &= \left| \frac{\partial \mathcal{L}(D, \Theta)}{\partial w} \cdot w \right|.\end{aligned}$$

上传/下载传输剪枝

过滤更新量较小的上传/下载模型更新

$$\nabla_g \mathcal{L}_i^u \leftarrow \nabla \mathcal{L}_i \odot \mathcal{M}_i^u.$$

$$\nabla \mathcal{L}_i^d = \left(\sum_{j=1}^M k_j \cdot \nabla_g \mathcal{L}_i \right) \odot \mathcal{M}_i^d$$



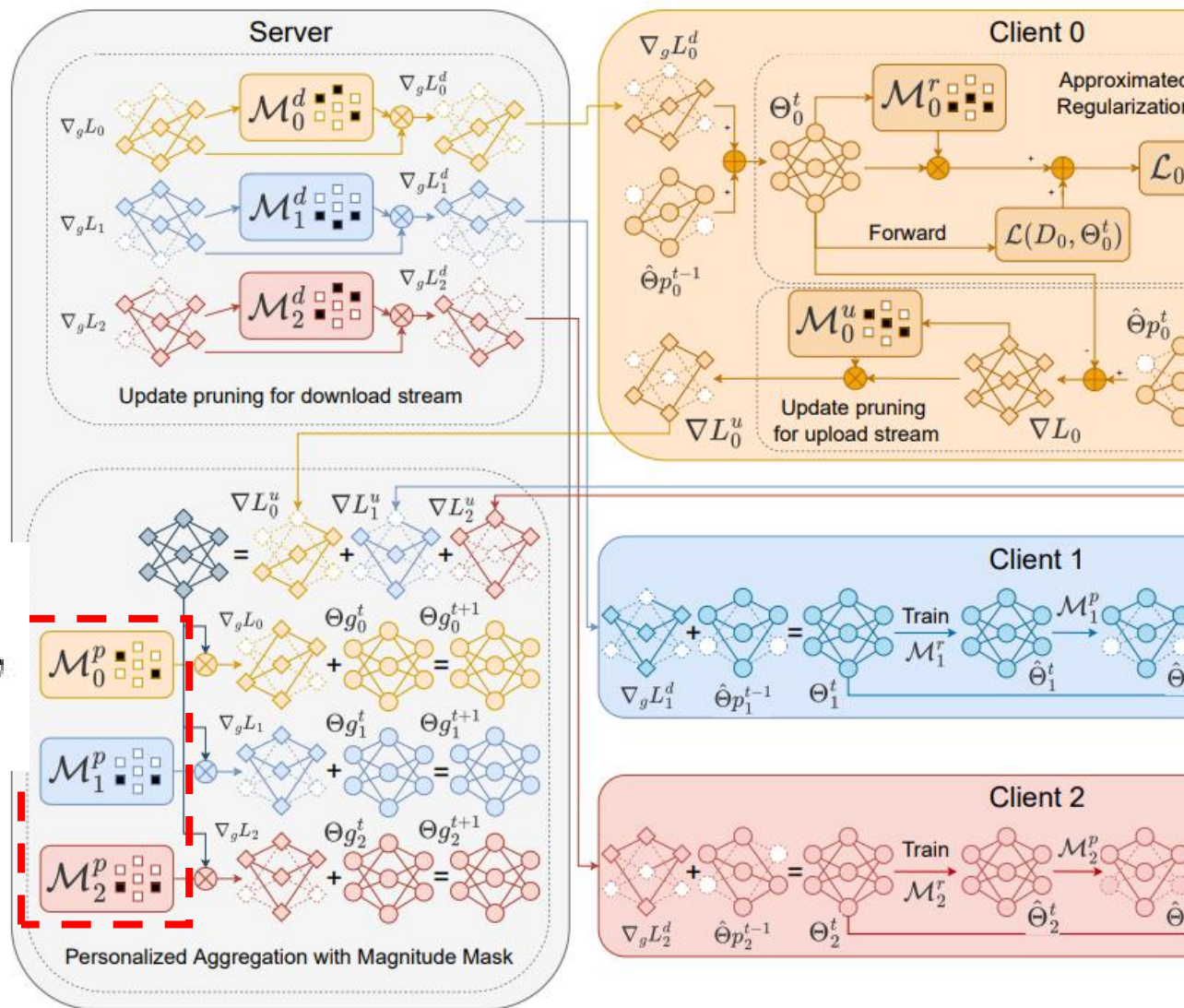


个性化模型聚合

只对有效更新进行聚合

$$\Theta g_i^{t+1} \leftarrow \lambda_2 \cdot \Theta g_i^t +$$

$$(1 - \lambda_2) \cdot \mathcal{M}_i^p \odot \sum_{j=1}^M k_j \cdot (\Theta g_j^t - \eta \cdot \nabla_g \mathcal{L}_j^u),$$





联邦学习效率-模型压缩



在大幅减少通信量
(~94%) 的情况下

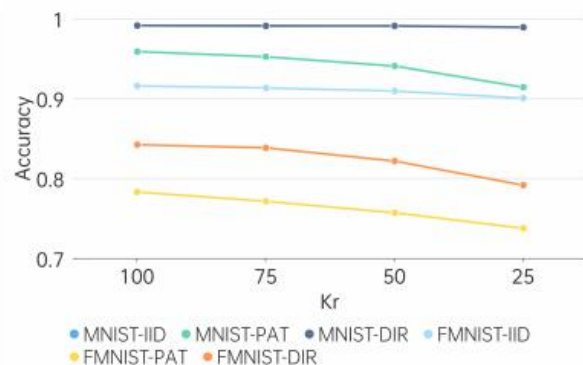
尽可能保障了训练准确率 (<6%)

			Dataset&Data distributions									
			MNIST					FMNIST				
			IID	DIR75	DIR50	DIR25	PAT	IID	DIR75	DIR50	DIR25	PAT
Metrics & Baselines	Acc.	FedAvg	0.9913	0.9907	0.9881	0.9833	0.9576	0.9142	0.9021	0.9029	0.8923	0.7795
		FedPer	0.9687	0.9569	0.9512	0.9291	0.8778	0.8490	0.8373	0.8439	0.8230	0.7334
		ADP-50	0.9900	0.9899	0.9882	0.9848	0.9593	0.9016	0.9022	0.9047	0.8920	0.8006
		Hermes-50	0.9902	0.9899	0.9883	0.9852	0.9596	0.9152	0.9030	0.9031	0.8930	0.7803
		SplitFed	0.9911	0.9909	0.9882	0.9834	0.9545	0.9134	0.9022	0.9050	0.8919	0.7680
		FCCL	0.9820	0.9808	0.9821	0.9866	0.9981	0.8831	0.9037	0.9122	0.9254	0.9949
		LotteryFL-75	0.9700	0.9700	0.9750	0.9800	0.9775	0.9152	0.9275	0.9125	0.9250	0.9825
		LotteryFL-50	0.9700	0.9725	0.9700	0.9775	0.9875	0.8625	0.9075	0.9075	0.9225	0.9875
		LotteryFL-25	0.9350	0.9400	0.9350	0.9525	0.9600	0.7625	0.7675	0.7075	0.7850	0.8600
		pFedEff-75	0.9908	0.9873	0.9888	0.9814	0.9729	0.9108	0.8887	0.8961	0.8989	0.9293
		pFedEff-50	0.9897	0.9843	0.9862	0.9786	0.9715	0.9086	0.8774	0.8867	0.8894	0.9585
		pFedEff-25	0.9863	0.9723	0.9745	0.9601	0.9745	0.9018	0.8427	0.8544	0.8557	0.9500
	Conv.	FedAvg	198	198	192	197	192	196	200	200	198	200
		FedPer	196	197	199	200	197	196	196	199	197	200
		ADP-50	148	187	179	162	197	174	197	198	200	192
		Hermes-50	90	160	193	199	198	178	184	198	177	198
		SplitFed	192	198	198	189	199	196	189	194	190	193
		FCCL	79	187	167	132	184	151	127	191	189	51
		LotteryFL-75	131	178	198	125	98	70	109	93	152	41
		LotteryFL-50	197	153	183	168	149	159	182	157	165	81
		LotteryFL-25	187	189	181	159	172	199	0	4	8	97
		pFedEff-75	197	193	196	194	2	200	187	195	198	4
		pFedEff-50	199	199	195	193	4	190	191	199	196	6
		pFedEff-25	195	200	200	200	19	184	200	199	199	16
	Comm. Vol.	FedAvg	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
		FedPer	0.0895	0.0895	0.0895	0.0895	0.0895	0.0895	0.0895	0.0895	0.0895	0.0895
		ADP-50	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
		Hermes-50	1.0037	1.0037	1.0037	1.0037	1.0037	1.0037	1.0037	1.0037	1.0037	1.0037
		SplitFed	0.1071	0.1071	0.1071	0.1071	0.1071	0.1071	0.1071	0.1071	0.1071	0.1071
		FCCL	0.1031	0.1031	0.1031	0.1031	0.1031	0.1031	0.1031	0.1031	0.1031	0.1031
		LotteryFL-75	0.7350	0.7351	0.7351	0.7351	0.7350	0.7350	0.7351	0.7351	0.7351	0.7350
		LotteryFL-50	0.4876	0.4877	0.4877	0.4877	0.4876	0.4876	0.4877	0.4877	0.4877	0.4876
		LotteryFL-25	0.2377	0.2378	0.2378	0.2378	0.2377	0.3362	0.2430	0.2575	0.2523	0.2377
		pFedEff-75	0.5277	0.5625	0.5625	0.5625	0.5149	0.5548	0.5625	0.5625	0.5625	0.5623
		pFedEff-50	0.2398	0.2500	0.2500	0.2500	0.2407	0.2500	0.2500	0.2500	0.2500	0.2500
		pFedEff-25	0.0596	0.0625	0.0625	0.0625	0.0625	0.0625	0.0625	0.0625	0.0625	0.0625

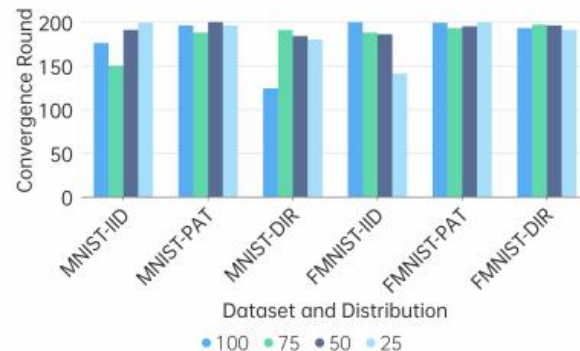


不同绝对值
mask下不同压
缩率结果

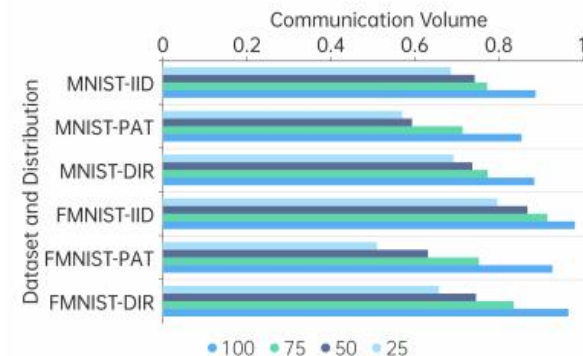
pre-training
mask和post-
training mask
能够有效减少
通信量，但是
对准确率影响
较大



(a)

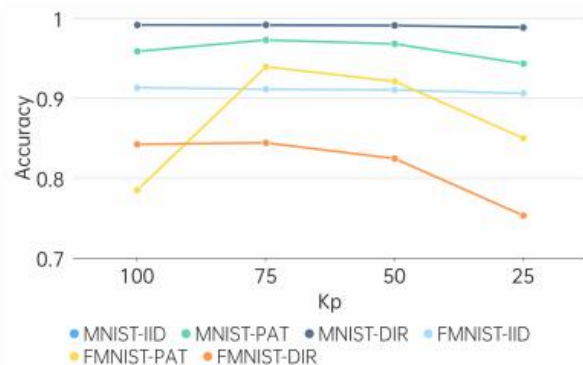


(b)

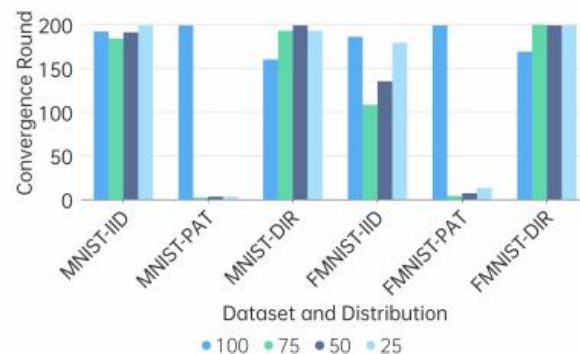


(c)

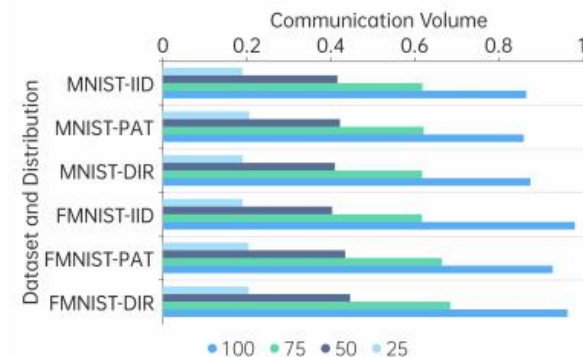
Fig. 5. The metrics when using different K_i^T for different distributions on different datasets. (a) Accuracy; (b) Convergence rate; (c) Communication Volume. We can observe that K_i^T can decrease the communication volume but introduce minor accuracy loss.



(a)



(b)



(c)

Fig. 6. The metrics when using different K_i^P for different distributions on different datasets. (a) Accuracy; (b) Convergence rate; (c) Communication Volume. We can observe that K_i^P significantly reduces the communication volume but introduce relatively large accuracy loss.



不同重要性

mask下不同压
缩率结果



upload mask

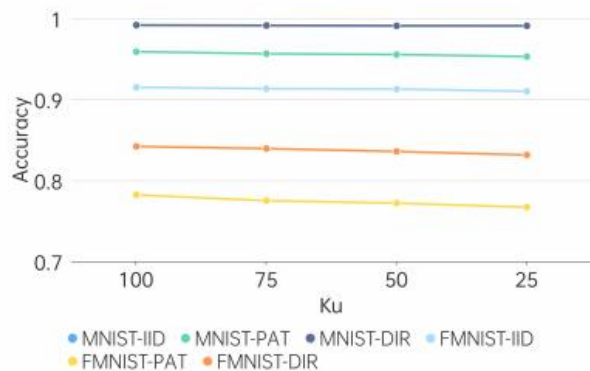
和download

mask虽然通信

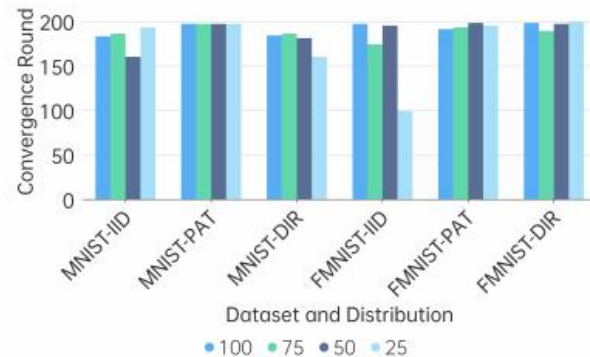
量减少有限,

但是能够有效

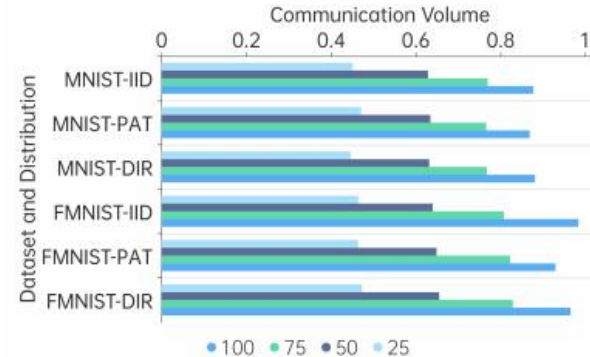
维持准确率



(a)

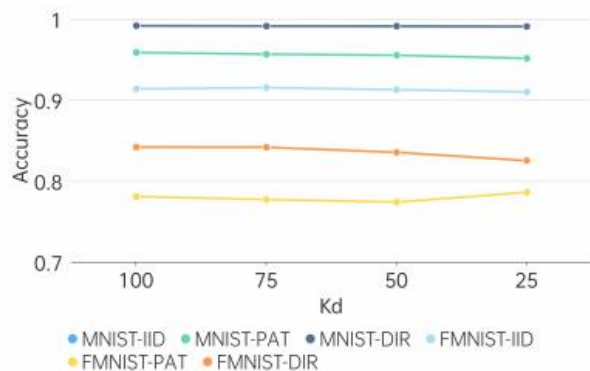


(b)

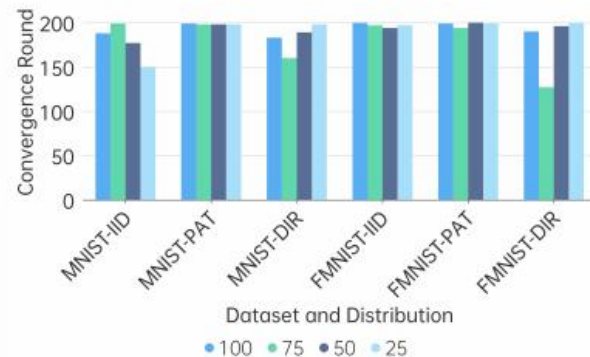


(c)

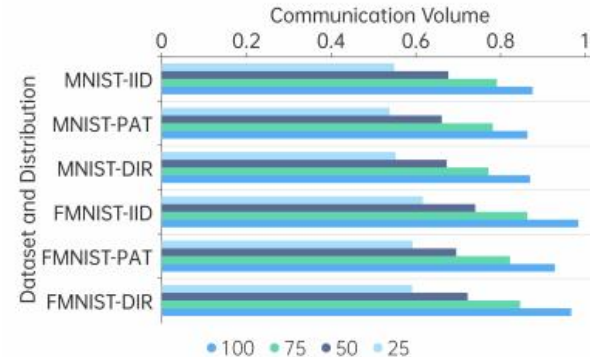
Fig. 7. The metrics when using different K_i^u for different distributions on different datasets. (a) Accuracy; (b) Convergence rate; (c) Communication Volume. We can observe that K_i^u can keep the training accuracy while decreasing the communication volume.



(a)



(b)



(c)

Fig. 8. The metrics when using different K_i^d for different distributions on different datasets. (a) Accuracy; (b) Convergence rate; (c) Communication Volume. We can observe that K_i^d can keep the training accuracy while decreasing the communication volume.



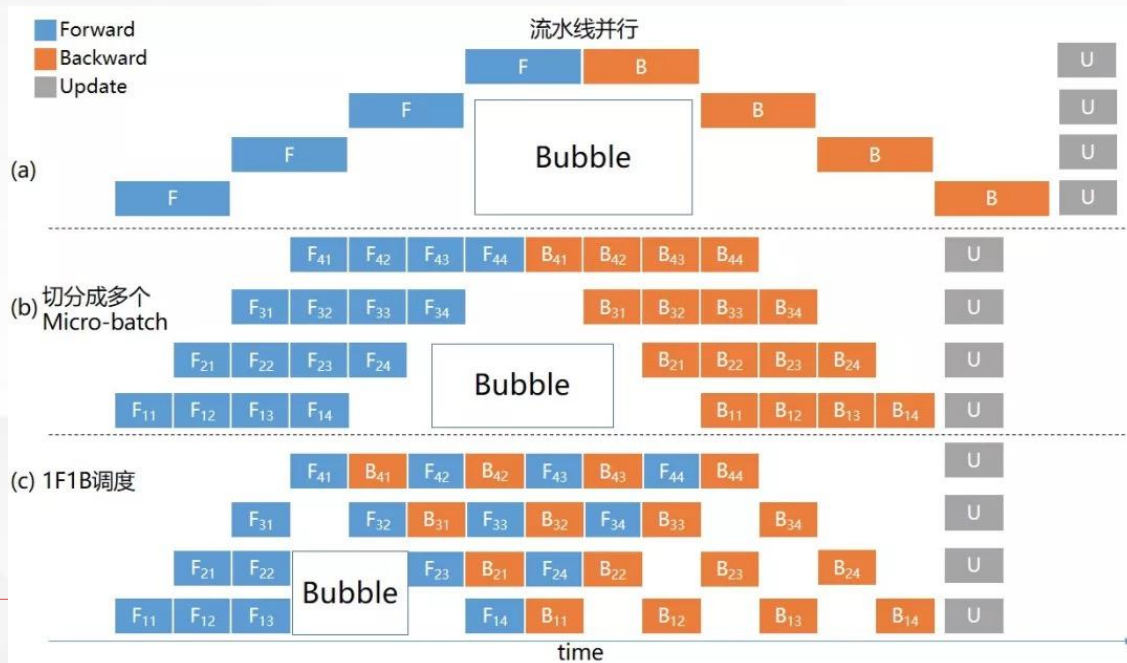
联邦学习效率-传输调度

任务调度

- 任务复杂，资源丰富，需要调度计算子任务

并行算法:

- 计算and计算、计算and通信、通信and通信
- 数据并行、模型并行、算子并行、优化器并行



Parallel Strategy	Affiliation
Pangu[2]	Huawei
Pipedream[3]	Microsoft
Gpipe[4]	Google
Fairscale[5]	Facebook
Megatron-LM[6]	NVIDIA
DeepSpeed[7]	Microsoft
Colossal-AI[8]	HPC-AI
FlexFlow[9]	Stanford
OneFlow[10]	OneFlow



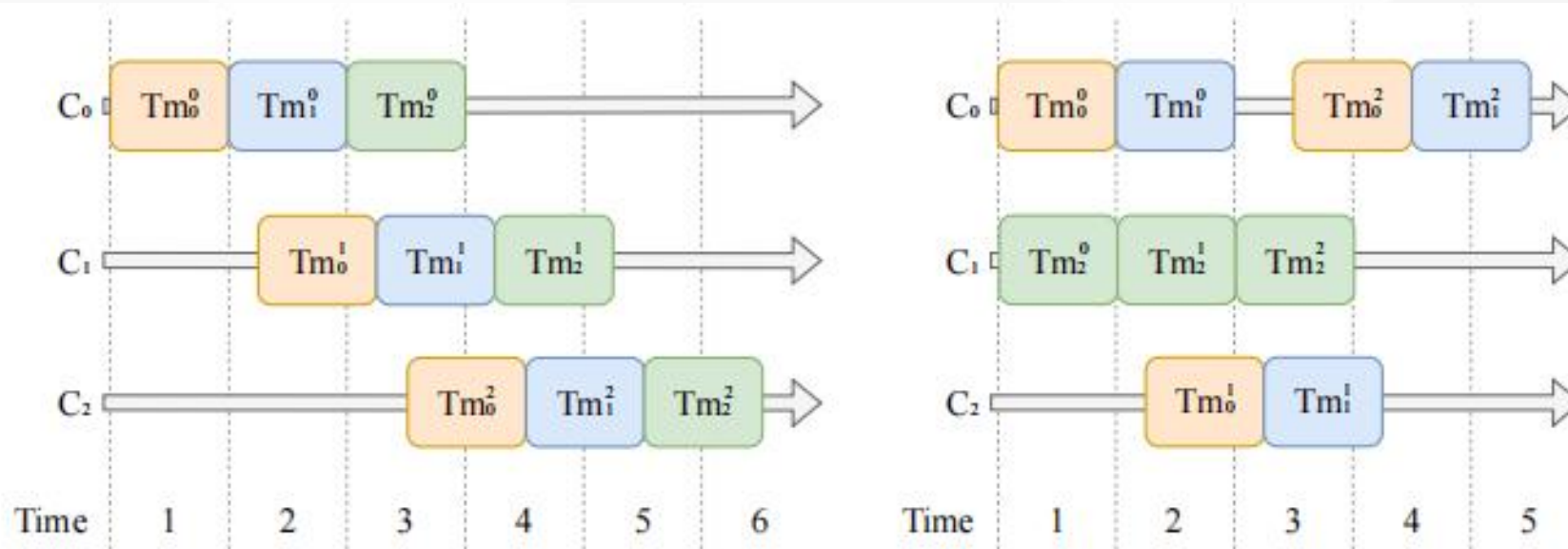
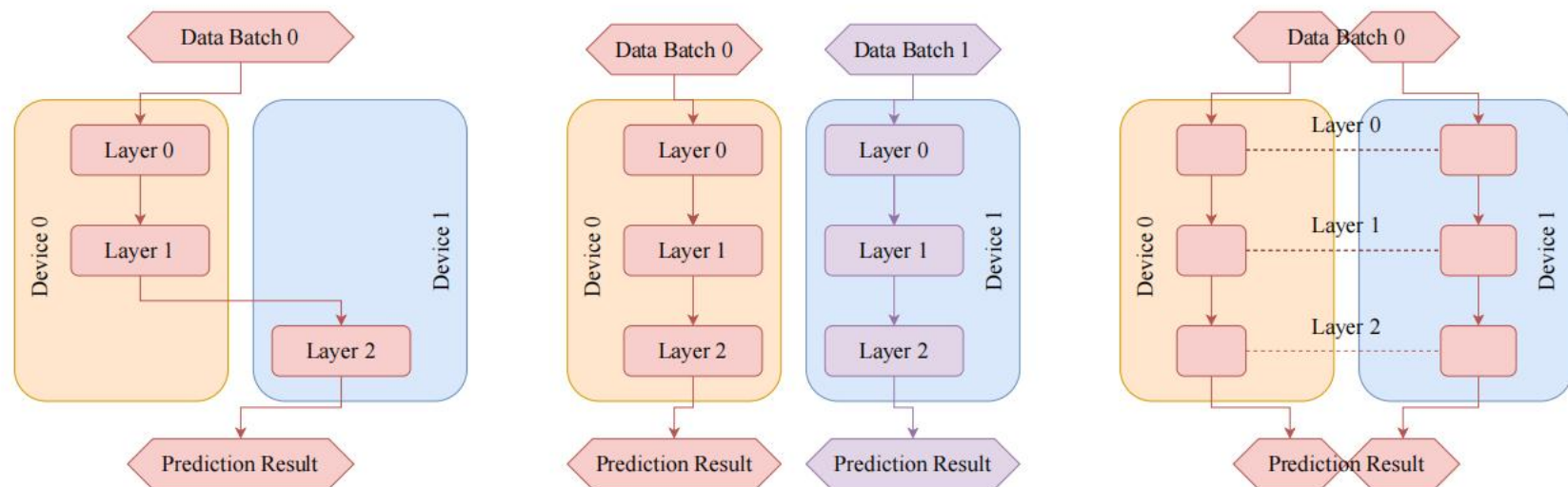
联邦学习效率-传输调度

并行算法

- **模型**: 减少空间占用
- **数据**: 减少处理延迟
- **算子**: 提高处理速度

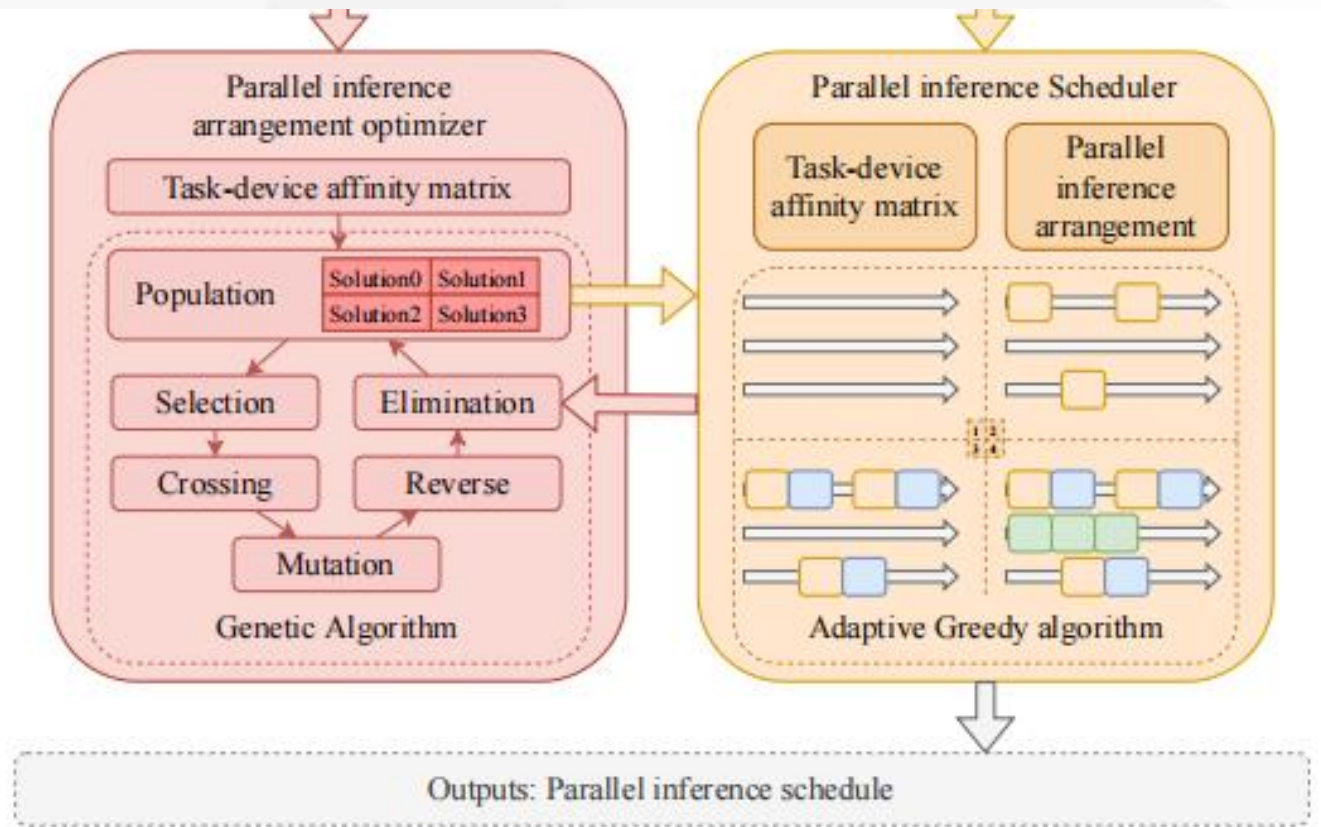
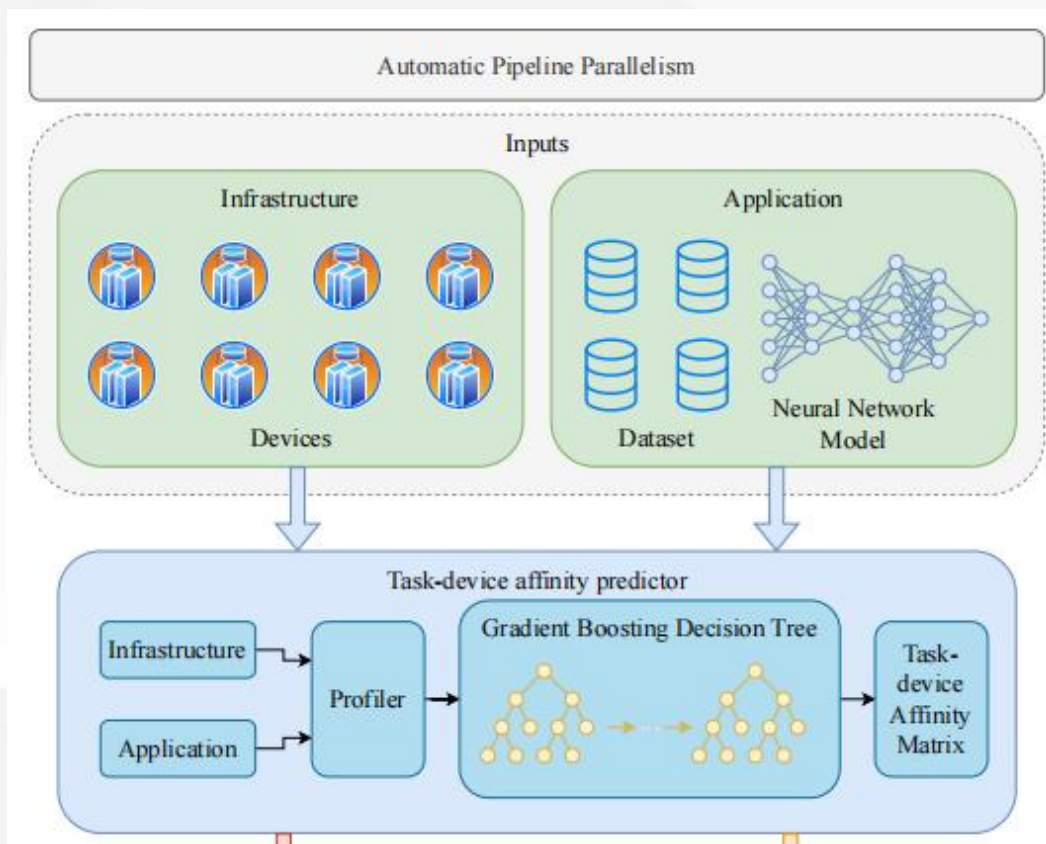
存在的问题

- 单一并行算法效率有限, 存在大量的资源浪费, 可用性无保障
- 多并行算法需要专业设计



联邦学习效率-传输调度

- ① 先采用**GBDT预测**各任务在各设备上的运行时间，来减少profiling的用时
- ② 而后采用**遗传算法**决定任务位置，确认各个任务所在设备，以减少任务处理延迟
- ③ 遗传算法中会用**贪心算法**排列各设备上的任务，得到适应度函数，并得到实际可运行的任务时间表





④ 整体算法能够提高设备利用率

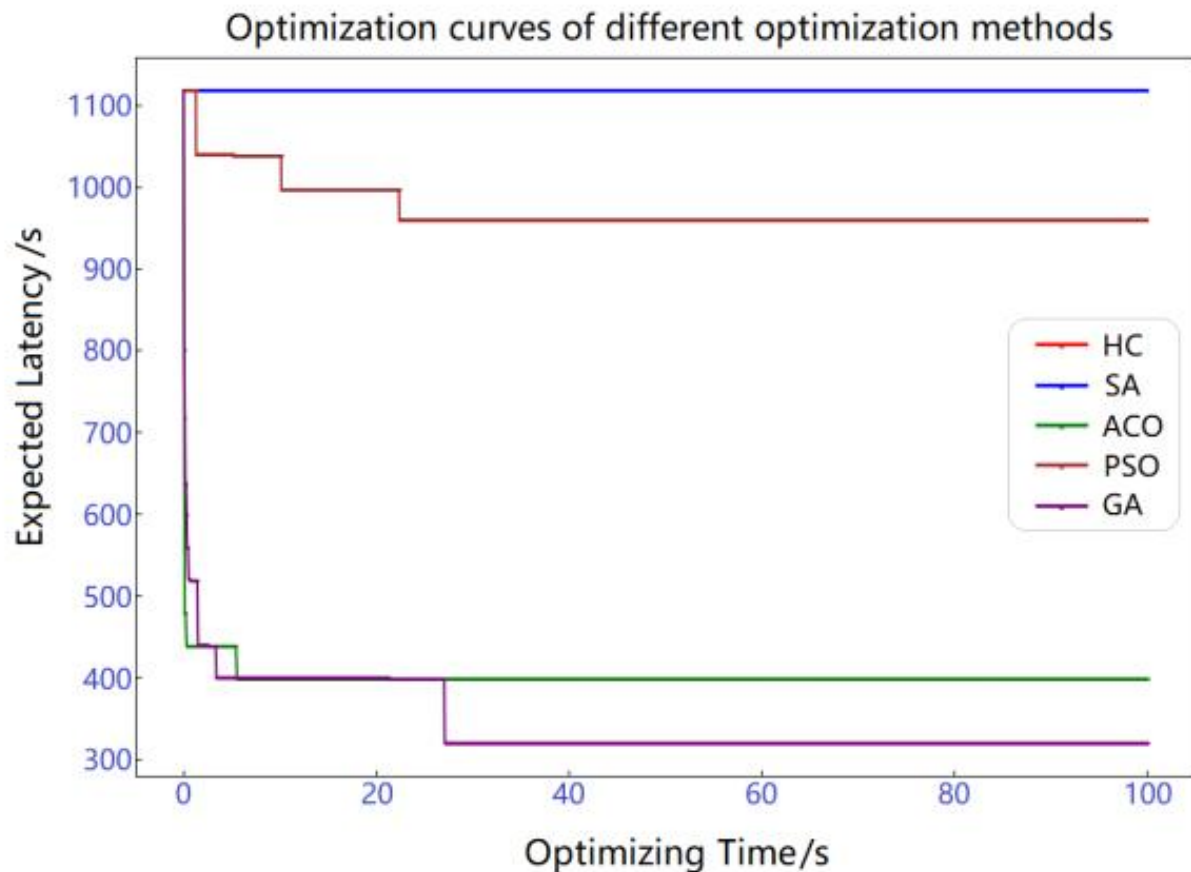
④ 减少延迟提高吞吐量

④ 同时能提高可用性

④ 采用遗传算法的效果和收敛性较好

Method	Latency(s)	Throu./(s)	Dev. Util.(%)	Rel.(%)
Standalone	6.1053	1310.34	12.2831	0
Model#8	9.6717	827.15	8.3009	0
Data	0.9468	8449.12	83.9462	100
Pipeline#8	2.9603	2702.43	27.0113	0
AP ² #5	1.3739	5822.89	49.9417	100

Method	Latency(s)	Throu./(s)	Dev. Util.(%)	Rel.(%)
Standalone	6.9814	1432.37	9.8450	0
Model#5	11.9720	835.29	6.2150	0
Data	0.9734	10273.53	82.3580	100
Pipeline#5	1.7692	5652.16	39.2073	89
AP ² #5	1.7178	5821.57	45.4050	100





- GBDT预测方法准确率明显由于其他预测方法
- 并且由于是线下训练，训练时间不会对线上推理产生负面影响
- 主要是由于GBDT对于表格数据和feature较少的数据有着十分优越的表现

Method	Test Accuracy	Train Accuracy	Training time(ms)
GBDT	0.7176±0.0001	0.8058±0.0000	107.48±0.64
FCNN-leakyrelu	0.6583±0.0003	0.7670±0.0000	6620.46±108.63
FCNN-relu [78]	0.6719±0.0002	0.7664±0.0000	6699.71±114.25
FCNN-tanh	0.6860±0.0000	0.7674±0.0000	6674.88±95.96
FCNN-sigmoid	0.7039±0.0000	0.7624±0.0000	6574.73±106.43
ABDT [79]	0.7114±0.0000	0.8001±0.0000	70.20±1.06
RF [80]	0.7096±0.0011	0.7738±0.0004	9.56±0.57
SVR [81]	0.7166±0.0000	0.7399±0.0000	7.97±0.52

n_estimators	Train Accuracy	Test Accuracy	Training Time(ms)
100	0.7410±0.0000	0.7172±0.0000	48.39±0.72
120	0.7634±0.0000	0.7251±0.0000	58.17±0.57
140	0.7783±0.0000	0.7266±0.0000	67.46±0.79
160	0.7882±0.0000	0.7238±0.0000	76.44±0.68
180	0.7978±0.0000	0.7209±0.0000	86.48±1.20
200	0.8058±0.0000	0.7176±0.0001	95.27±0.77
220	0.8120±0.0000	0.7132±0.0001	106.06±4.01
240	0.8219±0.0025	0.7064±0.0029	115.31±3.25
260	0.8273±0.0020	0.7020±0.0025	125.07±4.59
280	0.8309±0.0000	0.6992±0.0036	132.34±1.07



联邦学习安全

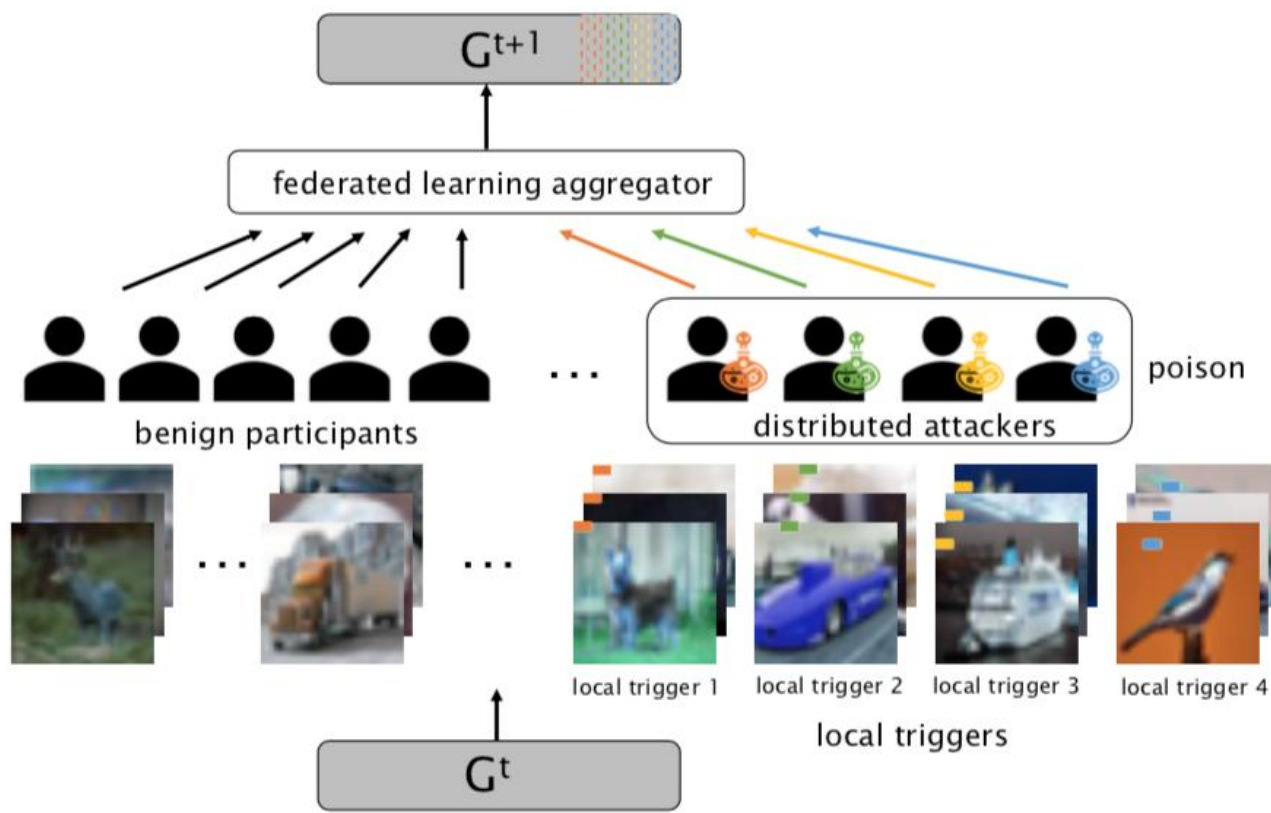
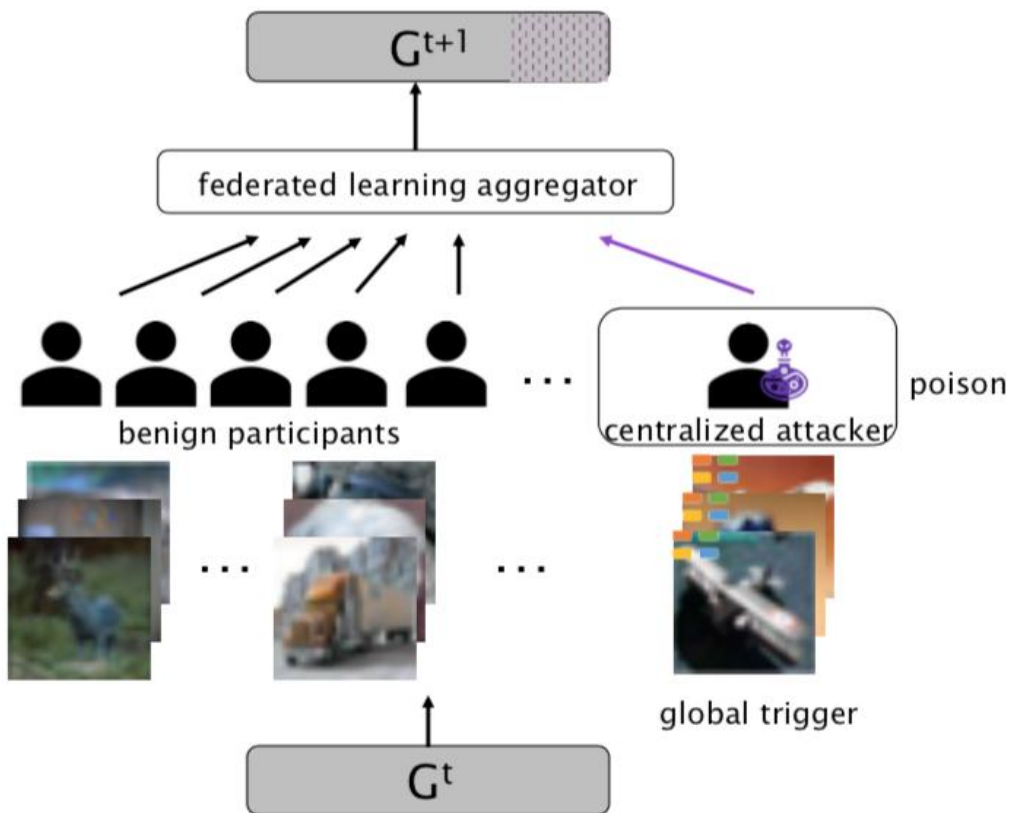
- 拜占庭攻击 - 双端协同预警
- 不可信第三方 - 区块链辅助聚合

计算安全

- 拜占庭攻击，系统中存在恶意节点通过传输错误数据破坏训练模型

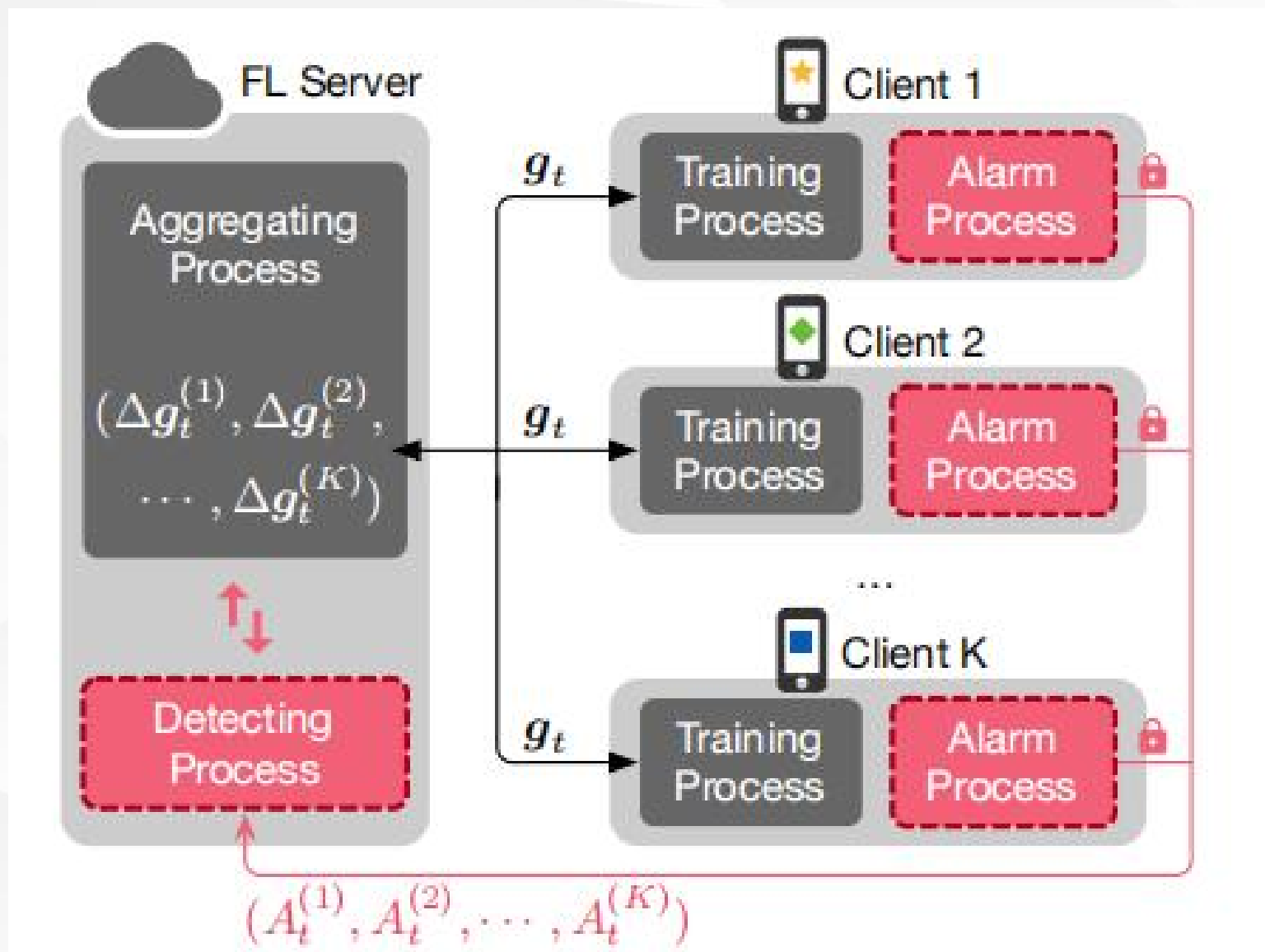
异常更新判断:

- 判断模型更新中的离群点
- 剔除离群点或是抑制离群点贡献





- ④ 利用客户端信息辅助排除恶意节点
- ④ 客户端侧拥有**示警机制**，可以和服务器侧的决策流程互相配合
- ④ 服务器侧使用基于准确度筛查和权重分析的**决策流程**
- ④ 系统提供两种辅助机制，分别为**惩罚机制**和**奖励机制**





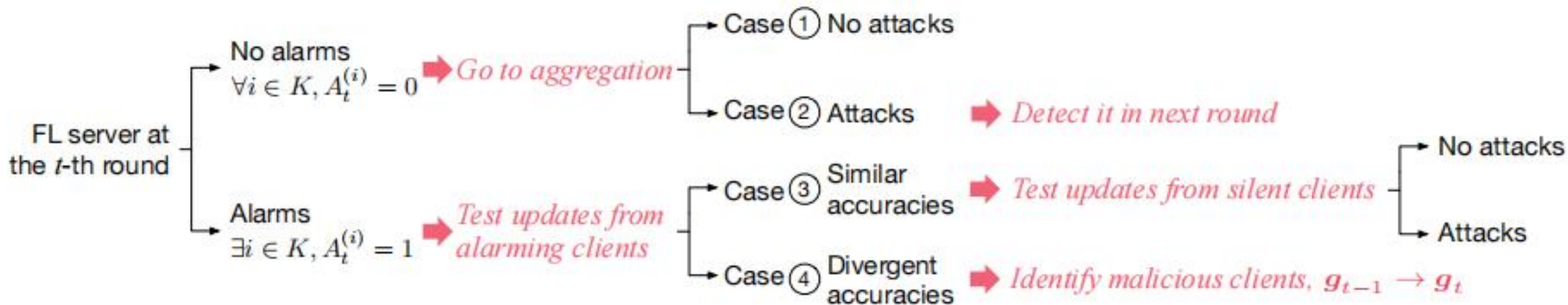
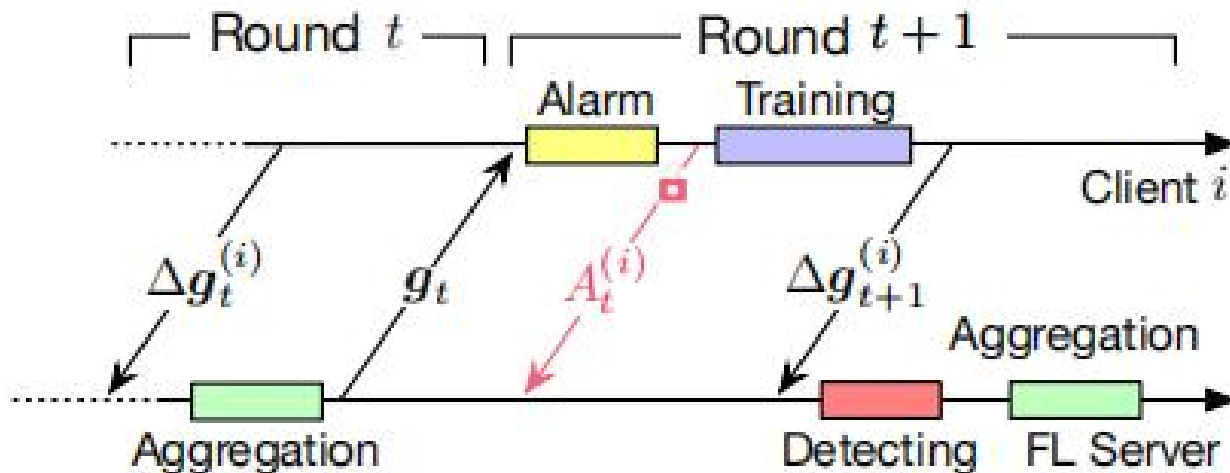
联邦学习安全---拜占庭攻击

客户端侧结构与方法流程

- 根据全局模型和本地模型**区别**进行判断

服务器侧结构与方法流程

- 根据客户端**告警**判断是否存在恶意攻击





在sign-flipping, label-flipping和targeted model poisoning攻击下可以有效收敛到一个较优准确率

也可以看出针对targeted model poisoning时, Siren稳定性较好

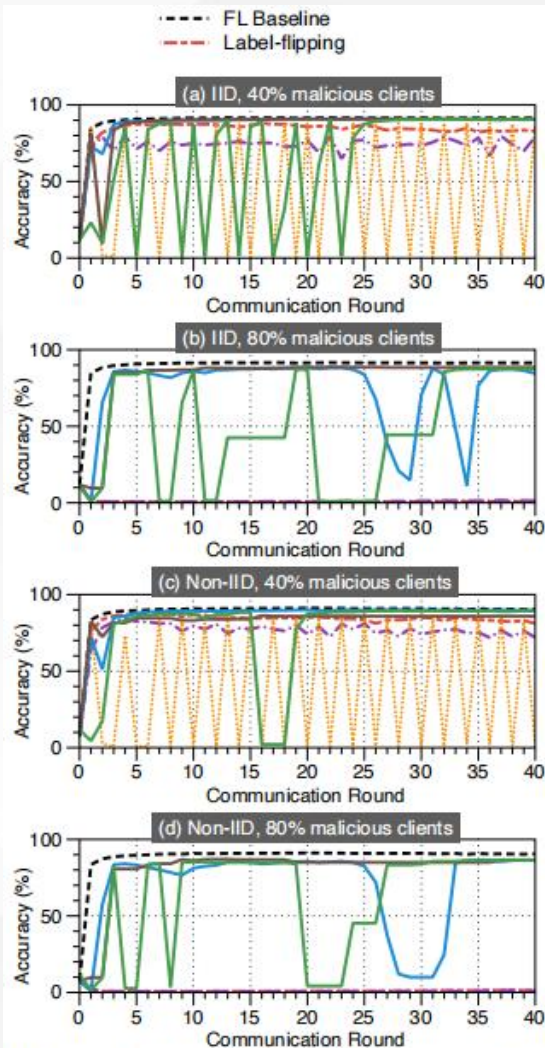


Figure 5: Training efficiency under label-flipping attack when $|K| = 10$.

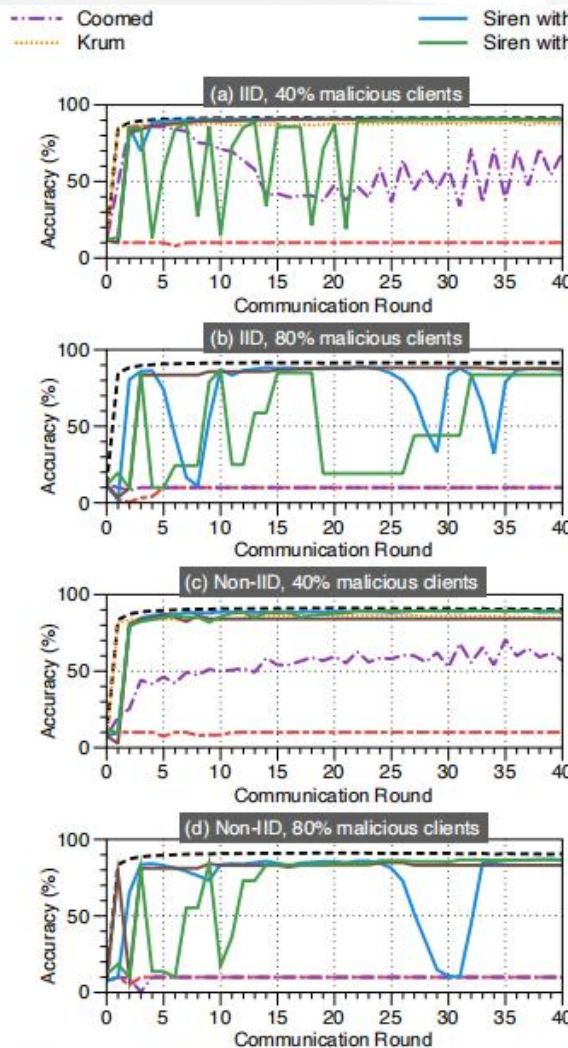


Figure 6: Training efficiency under sign-flipping attack when $|K| = 10$.

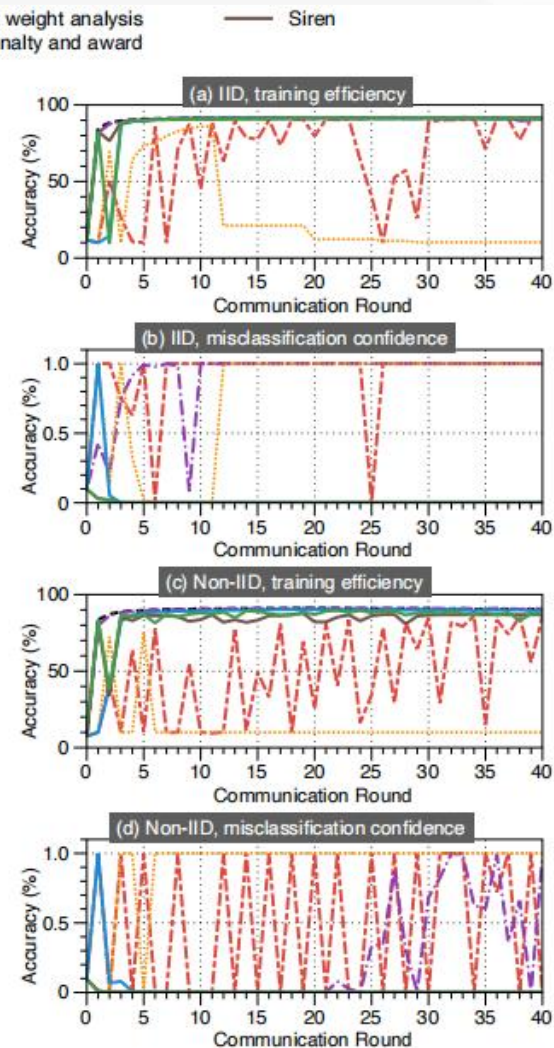
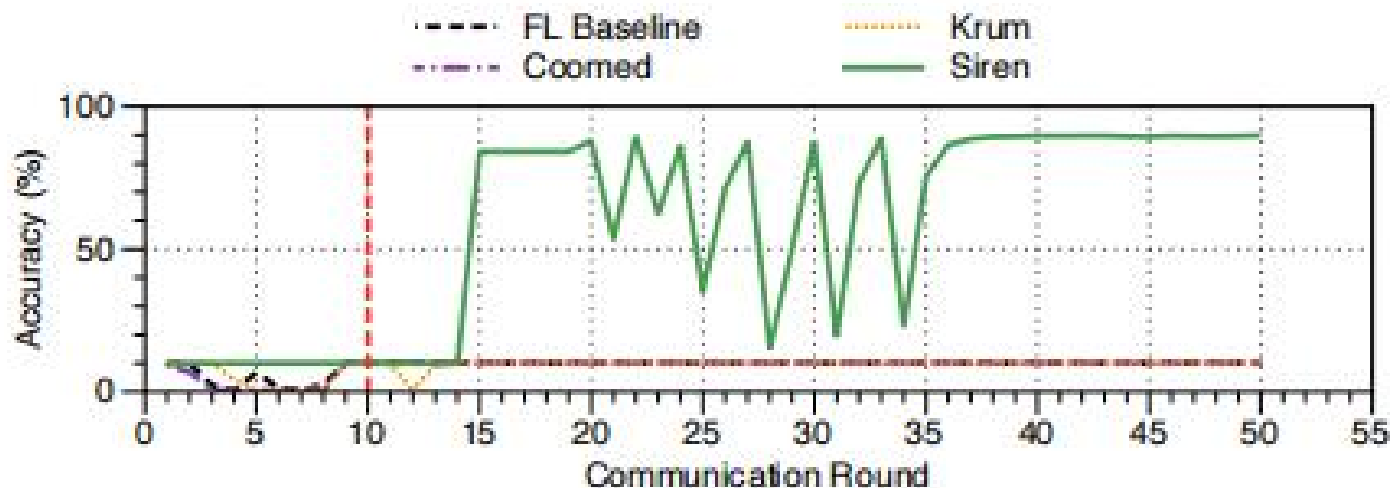


Figure 7: Training efficiency and misclassification confidence under targeted model poisoning, $|K| = 10$.



🔴 Siren可以在大比例恶意客户端的情况下
(80%) 拥有较强的防御能力

🔴 Siren可以在系统已经遭受攻击的情况下恢复
原有系统的训练准确率



Attack Type	Defense Methods	Malicious Proportion	Accuracy
Sign-flipping	None	0%	55.33%
	None	40%	10.01%
		80%	10.01%
	Krum ¹	40%	41.53%
	Coomed	40%	34.25%
		80%	9.99%
	SIREN	40%	51.58%
		80%	45.50%
Label-flipping	None	40%	34.70%
		80%	11.68%
	Krum ¹	40%	44.72%
	Coomed	40%	37.21%
		80%	9.60%
	SIREN	40%	49.82%
		80%	43.52%

¹ Krum cannot work properly when malicious clients' proportion reaches 80%.



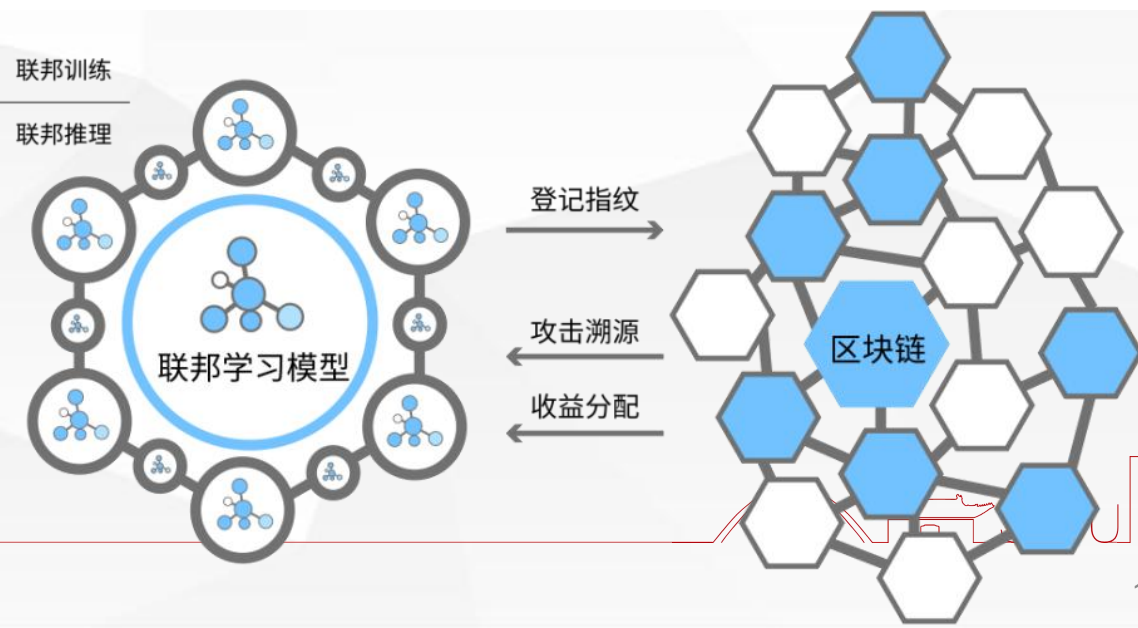
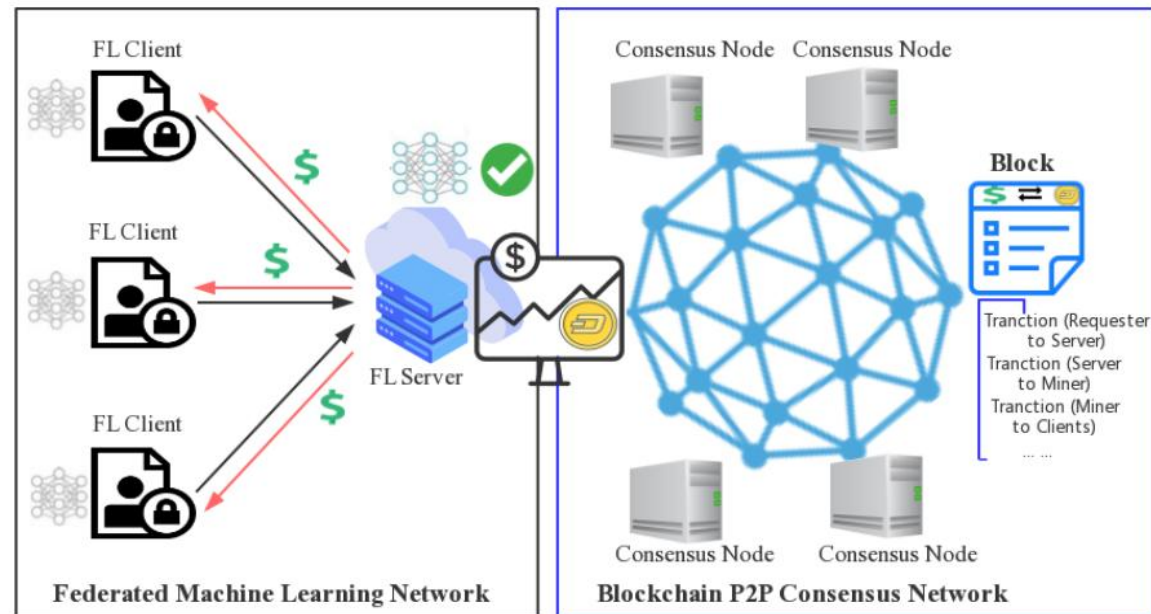
联邦学习安全---不可信第三方

通信安全

- 无可信第三方服务器，易受攻击

联邦学习+区块链

- 用区块链替换原有server保存全局模型，可以提高全局模型的防篡改能力
- 区块链本身的共识机制和联邦学习的聚合权重有很多相似之处





联邦学习异构性

- 统计异构性 - 非聚合联邦学习
- 系统异构性 - 适应性训练调整



联邦学习异构性---统计异构性

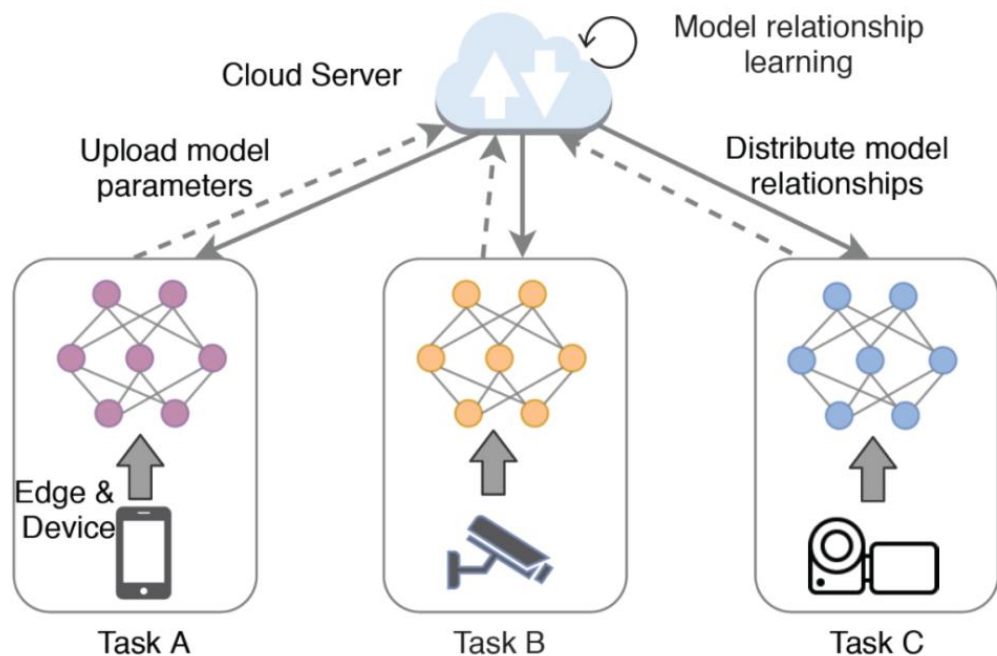


统计异构

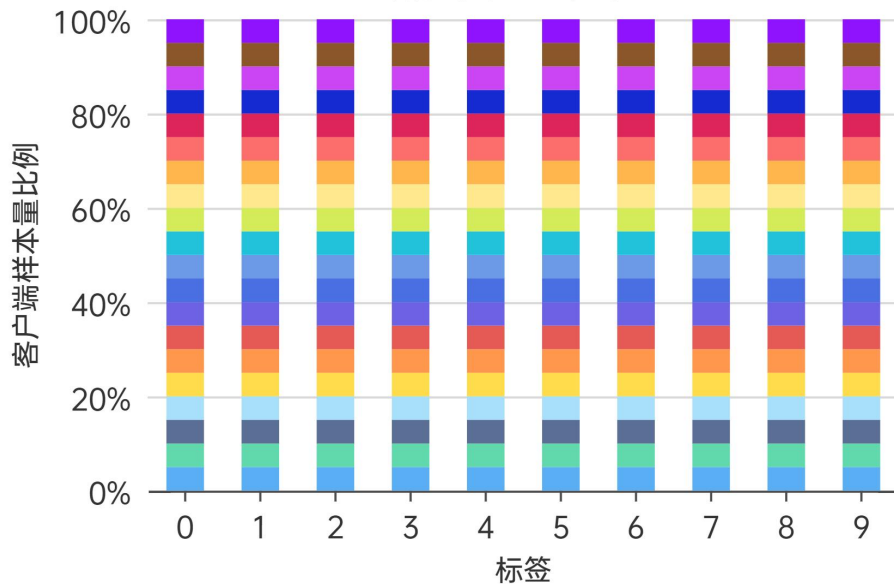
- 各客户端数据分布不同，模型不准确

跨域联邦学习：

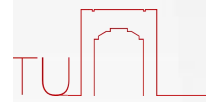
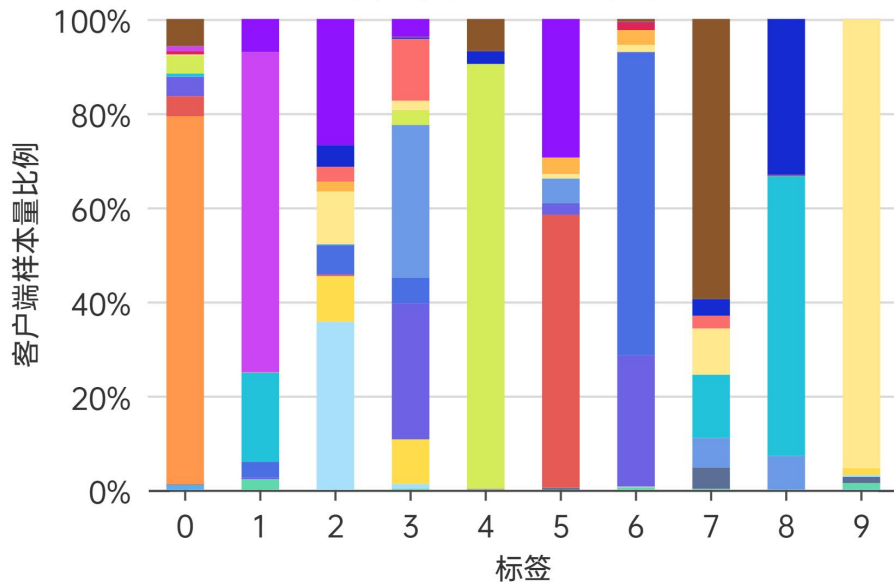
- 模型之间交换知识，不单纯进行聚合
- 模型在本地测试集上的效果会更好



FMNIST数据集下的IID分布



FMNIST数据集下的Dirichlet分布



联邦学习异构性---统计异构性

引入个性化mask做element-wise的更新

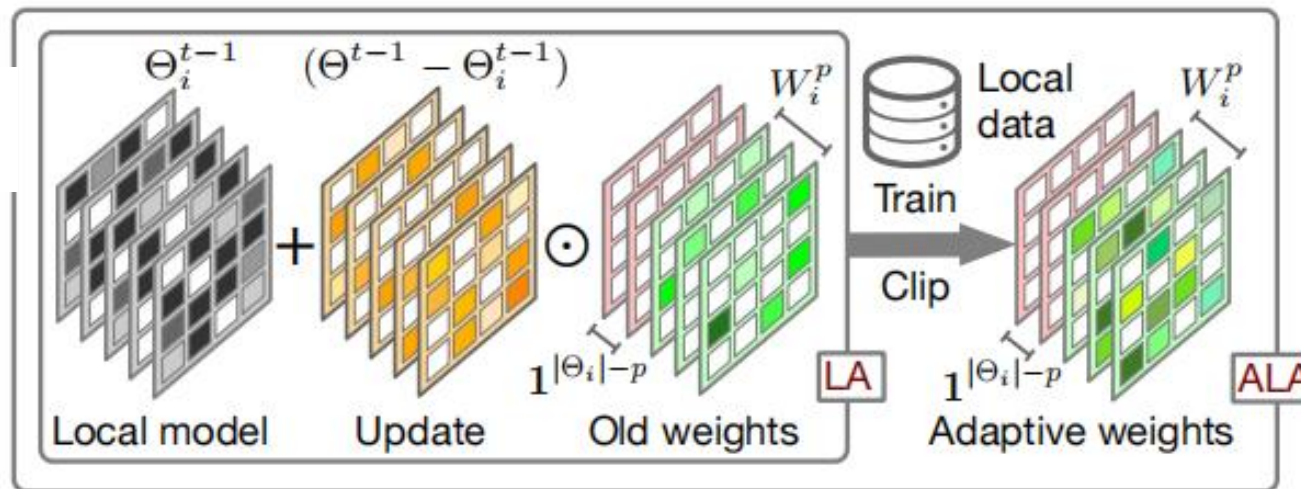
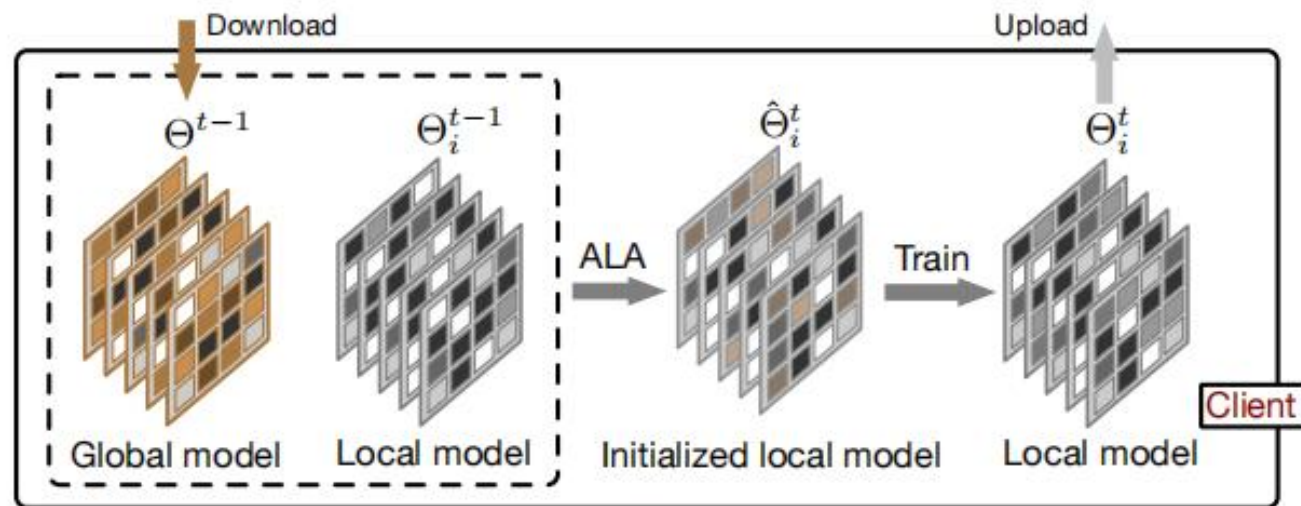
$$\hat{\Theta}_i^t := \Theta_i^{t-1} \odot W_{i,1} + \Theta^{t-1} \odot W_{i,2},$$

$$s.t. \quad w_1^q + w_2^q = 1, \forall \text{ valid } q$$

$$\hat{\Theta}_i^t := \Theta_i^{t-1} + (\Theta^{t-1} - \Theta_i^{t-1}) \odot W_i,$$

$$\hat{\Theta}_i^t := \Theta_i^{t-1} + (\Theta^{t-1} - \Theta_i^{t-1}) \odot [\mathbf{1}^{|\Theta_i|-p}; W_i^p],$$

$$W_i^p \leftarrow W_i^p - \eta \nabla_{W_i^p} \mathcal{L}(\hat{\Theta}_i^t, D_i^{s,t}; \Theta^{t-1}),$$





联邦学习异构性---统计异构性

在多个数据集多种设定上均取得了最高的准确率

Table 2: The test accuracy (%) in the pathological heterogeneous setting and practical heterogeneous setting.

Settings	Pathological heterogeneous setting			Practical heterogeneous setting				
Methods	MNIST	Cifar10	Cifar100	Cifar10	Cifar100	TINY	TINY*	AG News
FedAvg	97.93±0.05	55.09±0.83	25.98±0.13	59.16±0.47	31.89±0.47	19.46±0.20	19.45±0.13	79.57±0.17
FedProx	98.01±0.09	55.06±0.75	25.94±0.16	59.21±0.40	31.99±0.41	19.37±0.22	19.27±0.23	79.35±0.23
FedAvg-C	99.79±0.00	92.13±0.03	66.17±0.03	90.34±0.01	51.80±0.02	30.67±0.08	36.94±0.10	95.89±0.25
FedProx-C	99.80±0.04	92.12±0.03	66.07±0.08	90.33±0.01	51.84±0.07	30.77±0.13	38.78±0.52	96.10±0.22
Per-FedAvg	99.63±0.02	89.63±0.23	56.80±0.26	87.74±0.19	44.28±0.33	25.07±0.07	21.81±0.54	93.27±0.25
FedRep	99.77±0.03	91.93±0.14	67.56±0.31	90.40±0.24	52.39±0.35	37.27±0.20	39.95±0.61	96.28±0.14
pFedMe	99.75±0.02	90.11±0.10	58.20±0.14	88.09±0.32	47.34±0.46	26.93±0.19	33.44±0.33	91.41±0.22
Ditto	99.81±0.00	92.39±0.06	67.23±0.07	90.59±0.01	52.87±0.64	32.15±0.04	35.92±0.43	95.45±0.17
FedAMP	99.76±0.02	90.79±0.16	64.34±0.37	88.70±0.18	47.69±0.49	27.99±0.11	29.11±0.15	94.18±0.09
FedPHP	99.73±0.00	90.01±0.00	63.09±0.04	88.92±0.02	50.52±0.16	35.69±3.26	29.90±0.51	94.38±0.12
FedFomo	99.83±0.00	91.85±0.02	62.49±0.22	88.06±0.02	45.39±0.45	26.33±0.22	26.84±0.11	95.84±0.15
APPLE	99.75±0.01	90.97±0.05	65.80±0.08	89.37±0.11	53.22±0.20	35.04±0.47	39.93±0.52	95.63±0.21
PartialFed	99.86±0.01	89.60±0.13	61.39±0.12	87.38±0.08	48.81±0.20	35.26±0.18	37.50±0.16	85.20±0.16
FedALA	99.88±0.01	92.44±0.02	67.83±0.06	90.67±0.03	55.92±0.03	40.54±0.02	41.94±0.05	96.52±0.08



联邦学习异构性---统计异构性

FedALA对在不同统计异构性和扩展性的情况都展开了测试，在多种设定下其效果均比传统方法要好。

	Heterogeneity			Scalability		Applicability of ALA			
Datasets	Tiny-ImageNet		AG News	Cifar100		Tiny-ImageNet		Cifar100	
Methods	<i>Dir</i> (0.01)	<i>Dir</i> (0.5)	<i>Dir</i> (1)	50 clients	100 clients	Acc.	Imps.	Acc.	Imps.
FedAvg	15.70±0.46	21.14±0.47	87.12±0.19	31.90±0.27	31.95±0.37	40.54±0.17	21.08	55.92±0.15	24.03
FedProx	15.66±0.36	21.22±0.47	87.21±0.13	31.94±0.30	31.97±0.24	40.53±0.26	21.16	56.18±0.65	24.19
FedAvg-C	49.88±0.11	16.21±0.05	91.38±0.21	49.82±0.11	47.90±0.12	—	—	—	—
FedProx-C	49.84±0.02	16.36±0.19	92.03±0.19	49.79±0.14	48.02±0.02	—	—	—	—
Per-FedAvg	39.39±0.30	16.36±0.13	87.08±0.26	44.31±0.20	36.07±0.24	30.90±0.28	5.83	48.68±0.36	4.40
FedRep	55.43±0.15	16.74±0.09	92.25±0.20	47.41±0.18	44.61±0.20	37.89±0.31	0.62	53.02±0.11	0.63
pFedMe	41.45±0.14	17.48±0.61	87.08±0.18	48.36±0.64	46.45±0.18	27.30±0.24	0.37	47.91±0.21	0.57
Ditto	50.62±0.02	18.98±0.05	91.89±0.17	54.22±0.04	52.89±0.22	40.75±0.06	8.60	56.33±0.07	3.46
FedAMP	48.42±0.06	12.48±0.21	83.35±0.05	44.39±0.35	40.43±0.17	28.18±0.20	0.19	48.03±0.23	0.34
FedPHP	48.63±0.02	21.09±0.07	90.52±0.19	52.44±0.16	49.70±0.31	40.16±0.24	4.47	54.28±0.21	3.76
FedFomo	46.36±0.54	11.59±0.11	91.20±0.18	42.56±0.33	38.91±0.08	—	—	—	—
APPLE	48.04±0.10	24.28±0.21	84.10±0.18	55.06±0.20	52.81±0.29	—	—	—	—
PartialFed	49.38±0.02	24.20±0.10	91.01±0.28	48.95±0.07	39.31±0.01	35.40±0.02	0.14	48.99±0.05	0.18
FedALA	55.75±0.02	27.85±0.06	92.45±0.10	55.61±0.02	54.68±0.57	—	—	—	—



联邦学习异构性---统计异构性

同时算法相比其他baseline，减少了一定的时间，提高了整体的计算效率和训练效率

其通信量和FedAvg相当，并没有引入额外的传输

	Computation		Communication
	Total time	Time/iter.	Param./iter.
FedAvg	365 min	1.59 min	$2 * \Sigma$
FedProx	325 min	1.99 min	$2 * \Sigma$
FedAvg-C	607 min	24.28 min	$2 * \Sigma$
FedProx-C	711 min	28.44 min	$2 * \Sigma$
Per-FedAvg	121 min	3.56 min	$2 * \Sigma$
FedRep	471 min	4.09 min	$2 * \alpha_f * \Sigma$
pFedMe	1157 min	10.24 min	$2 * \Sigma$
Ditto	318 min	11.78 min	$2 * \Sigma$
FedAMP	92 min	1.53 min	$2 * \Sigma$
FedPHP	264 min	4.06 min	$2 * \Sigma$
FedFomo	193 min	2.72 min	$(1 + M) * \Sigma$
APPLE	132 min	2.93 min	$(1 + M) * \Sigma$
PartialFed	693 min	2.13 min	$2 * \Sigma$
FedALA	7+116 min	1.93 min	$2 * \Sigma$

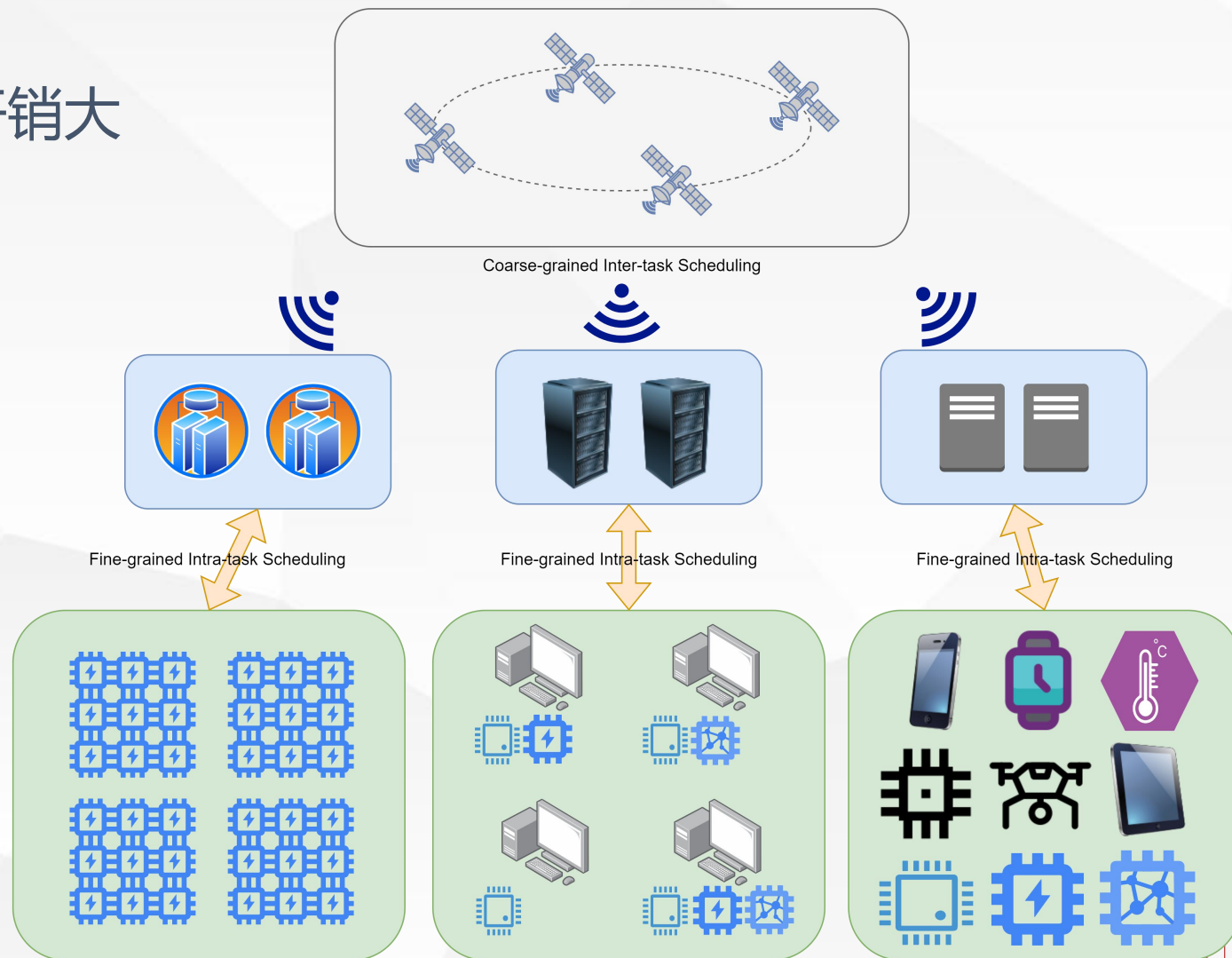
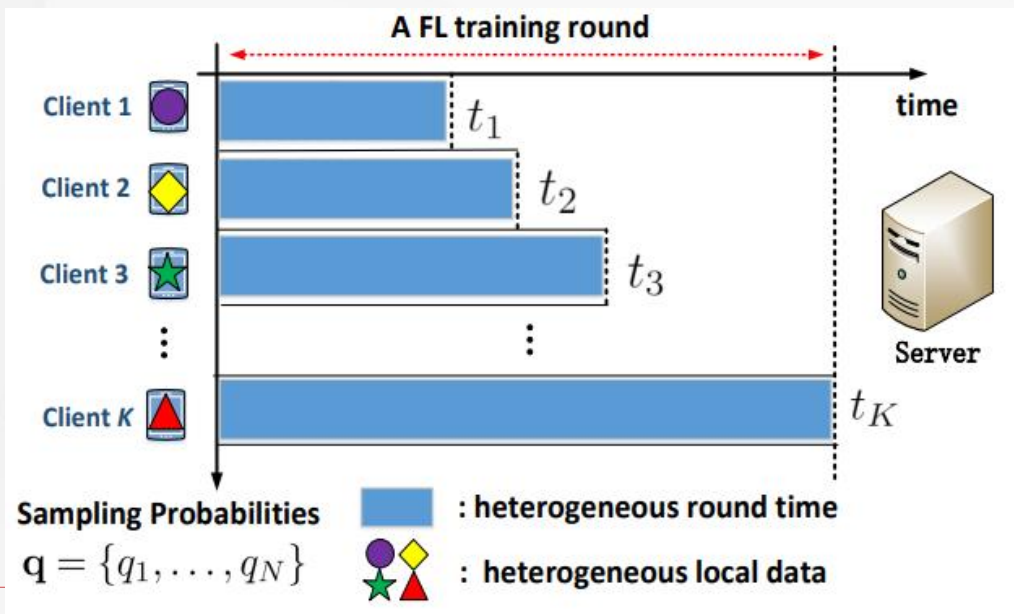
联邦学习异构性---系统异构性

系统异构

- 各客户端计算能力不同，通信开销大

适应性训练调整

- 采用不同模型不同训练参数
- 采用异步或者回合制聚合





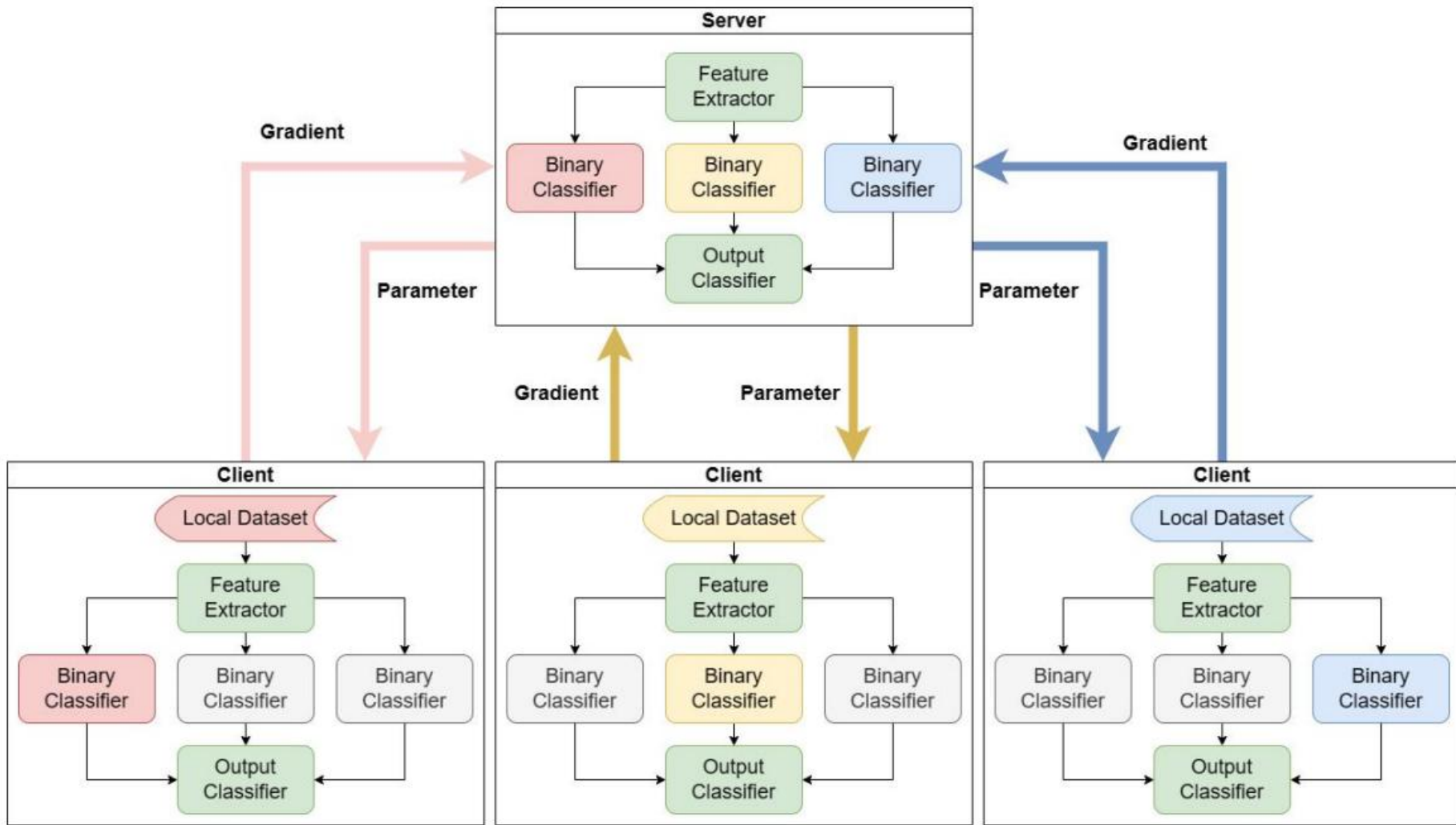
联邦学习异构性---系统异构性

将多分类模型分解成多个二分类模型

训练时只对本地标签对应的分支进行训练

传输时只传输经过训练的模型分支

聚合时只聚合传输上来的对应分支

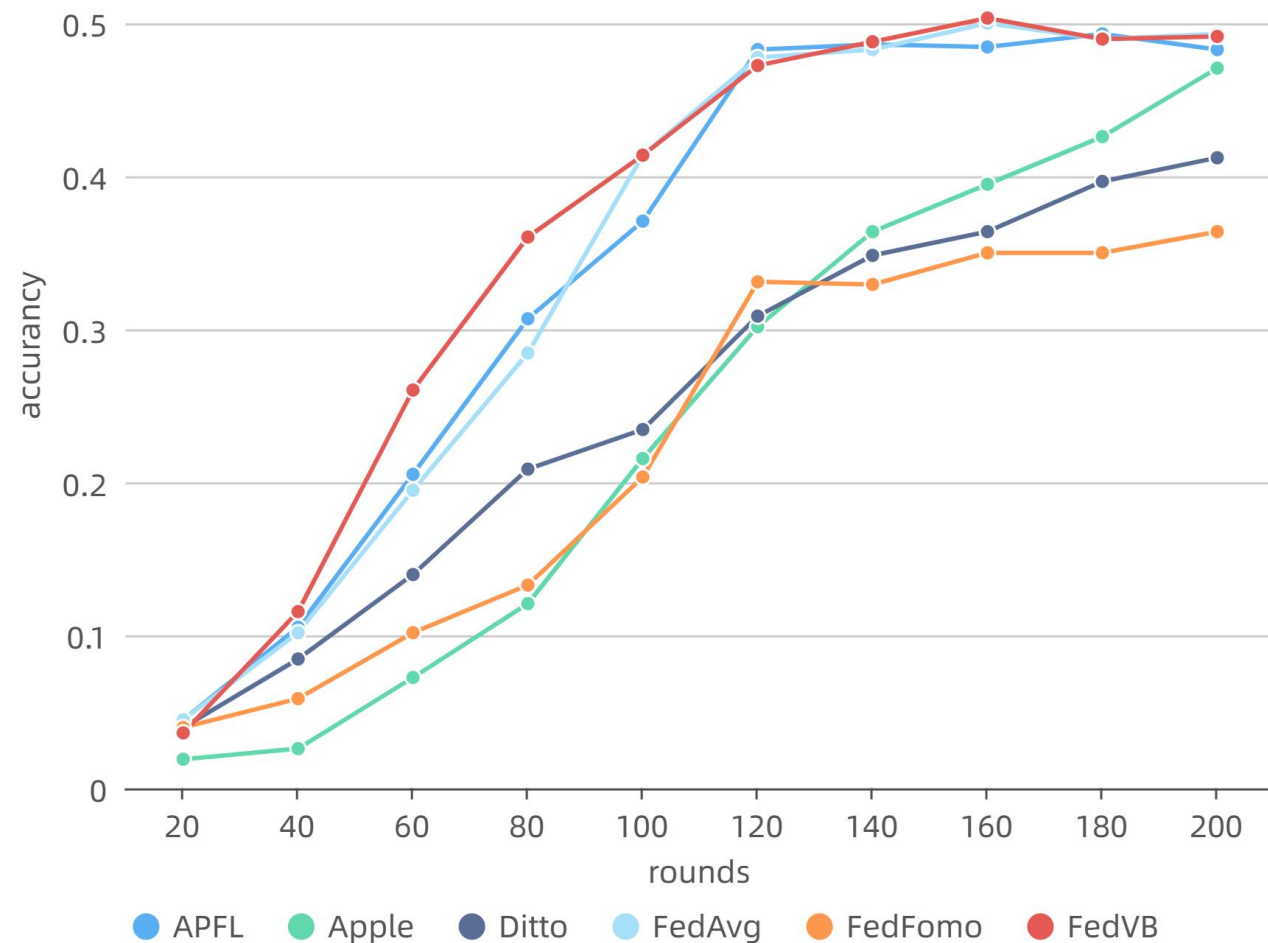




FedVB相比其他baseline收敛更快，准确率更高

多分支模型相比传统模型能够在准确率上有所提升

Algorithm	Simple classifier	Multi-branch Network
FedAvg	0.3983	0.5000
APFL	0.4276	0.5000
Apple	0.3983	0.4707
Ditto	0.3137	0.4259
FedBN	0.3983	0.5017
FedMTL	0.3224	0.5017
FedProx	0.4000	0.5017
pFedMe	0.0259	0.0379
FedFomo	0.8897	0.8500
LOCAL	0.8638	0.8621
PerAvg	0.6000	0.7483





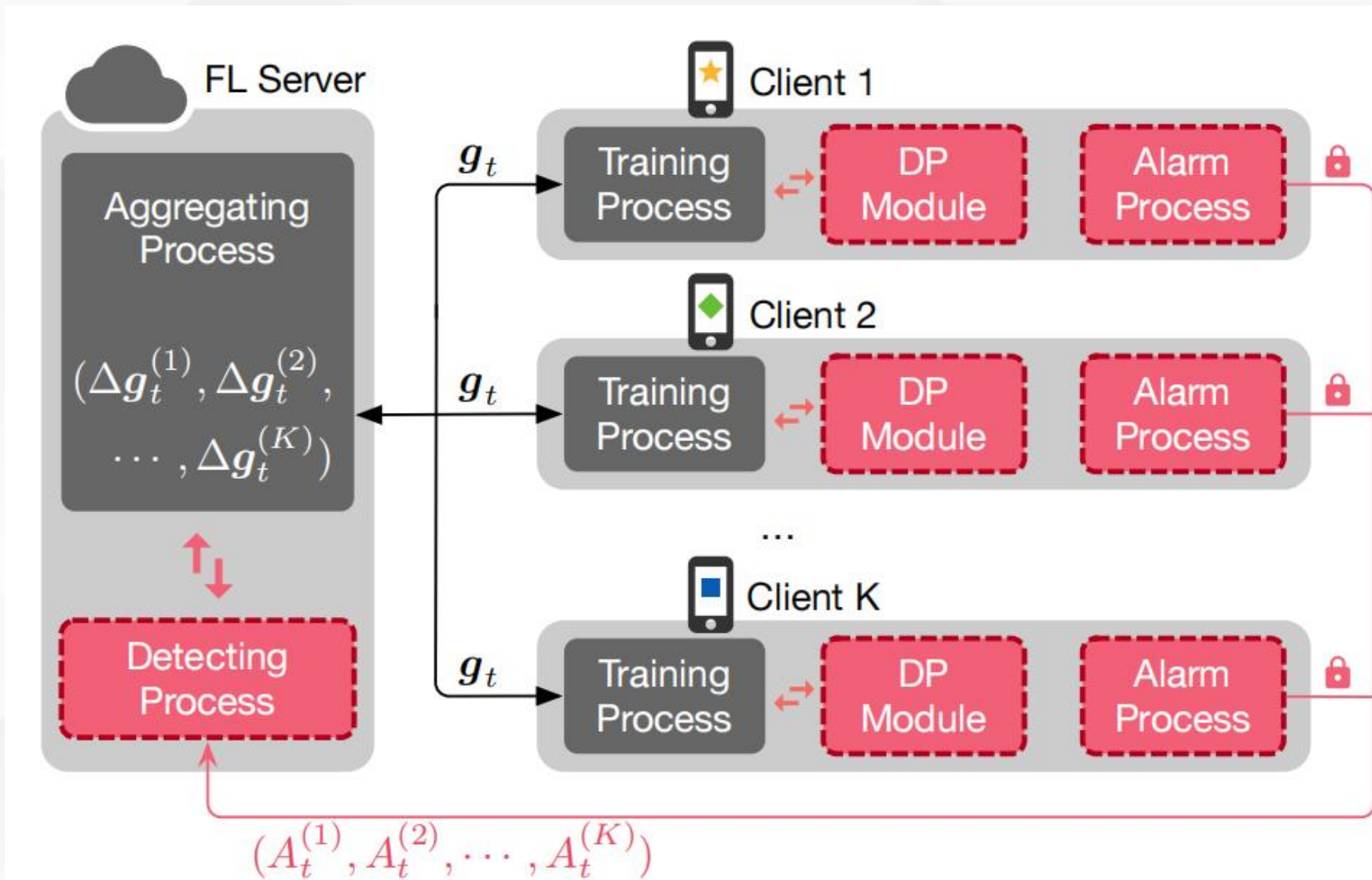
其他论文

- 效率 + 安全
- 效率 + 异构性
- 安全 + 异构性
- 效率 + 安全 + 异构性



联邦学习安全

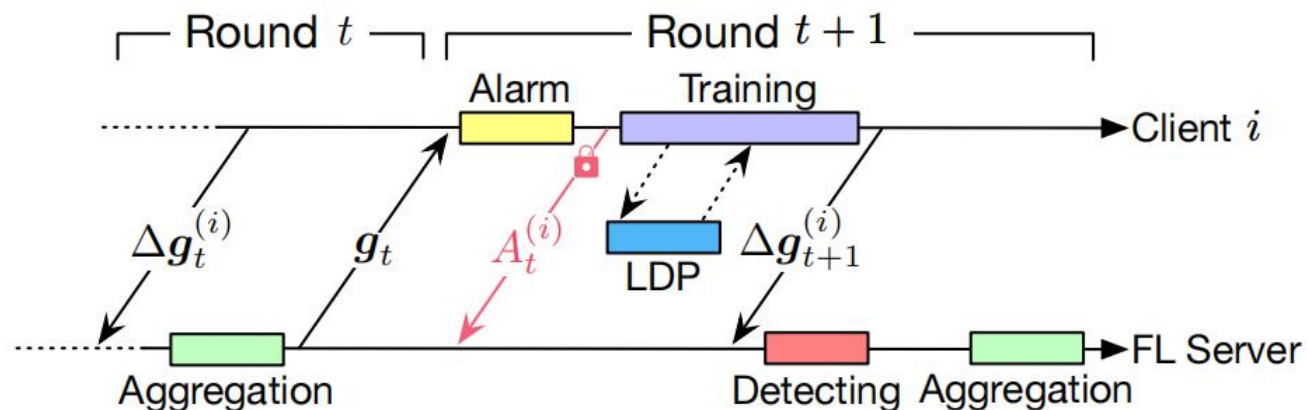
在原本Siren的基础上融入和差分隐私以进一步保障数据隐私





在原本Siren基础上将客户端部分加入了差分隐私

```
// training process
10 function ClientUpdate (i, gt)
11   At(i) ← Alarm(gt, gt(i));
12   if At(i) is 0 then
13     g(i) ← gt;
14   else
15     g(i) ← gt(i);
16   for each epoch e = 1, ..., E do
17     if not use DP then
18       // normal local training
19       train the model g(i) with optimizer on the
20       local dataset D(i), and obtain gt+1(i);
21     else
22       // local training with DP
23       train the model g(i) with LDP on the local
24       dataset D(i), and obtain gt+1(i);
25   // calculate model updates
26   Δgt+1(i) ← gt+1(i) - gt;
27   return Δgt+1(i);
```



```
// alarming process
1 function Alarm(gt, gt(i))
2   ωt ← testing gt on the local test dataset D0(i);
3   ωt(i) ← testing gt(i) on the local test dataset D0(i);
4   if ωt ≥ ωt(i) · (1 - Cc) then
5     // the global model is normal
6     At(i) ← 0;
7   else
8     // the global model is abnormal
9     At(i) ← 1;
10  send At(i) in a secure tunnel to the FL server;
11  return At(i);
```



与原本Siren性能相当

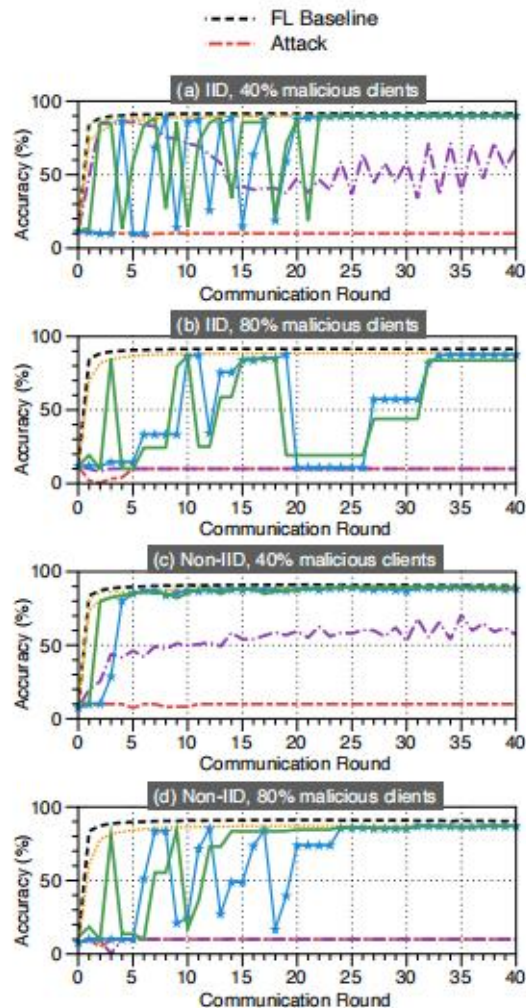


Fig. 5: Training efficiency under sign-flipping attack when $|K| = 10$.

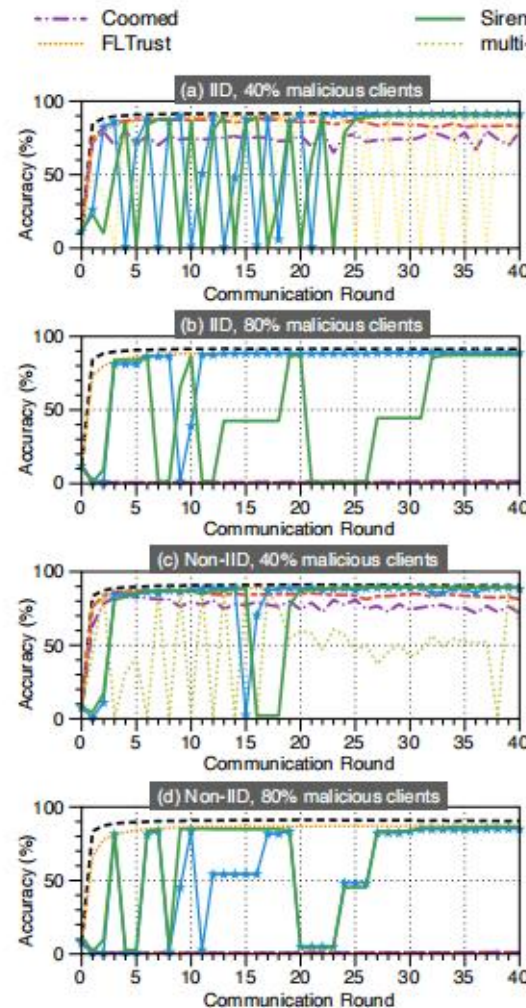


Fig. 6: Training efficiency under label-flipping attack when $|K| = 10$.

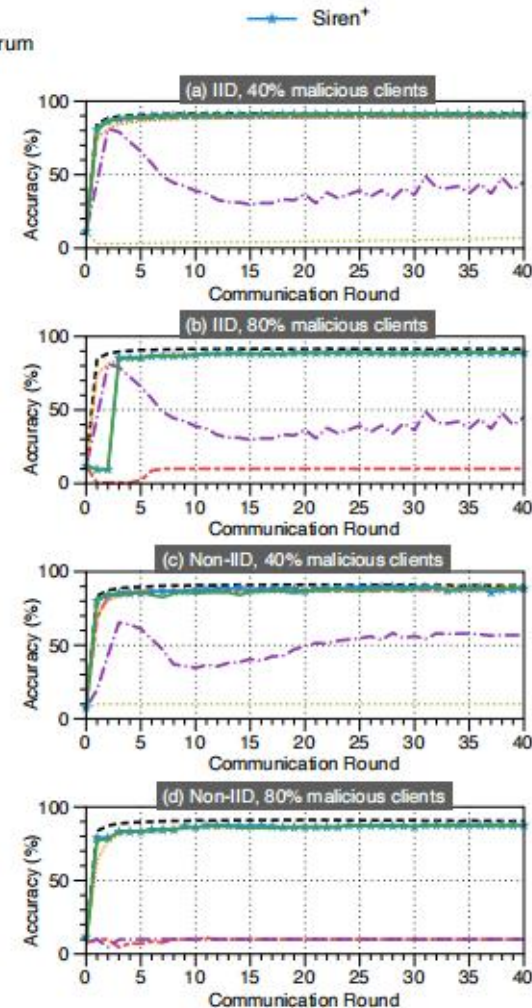
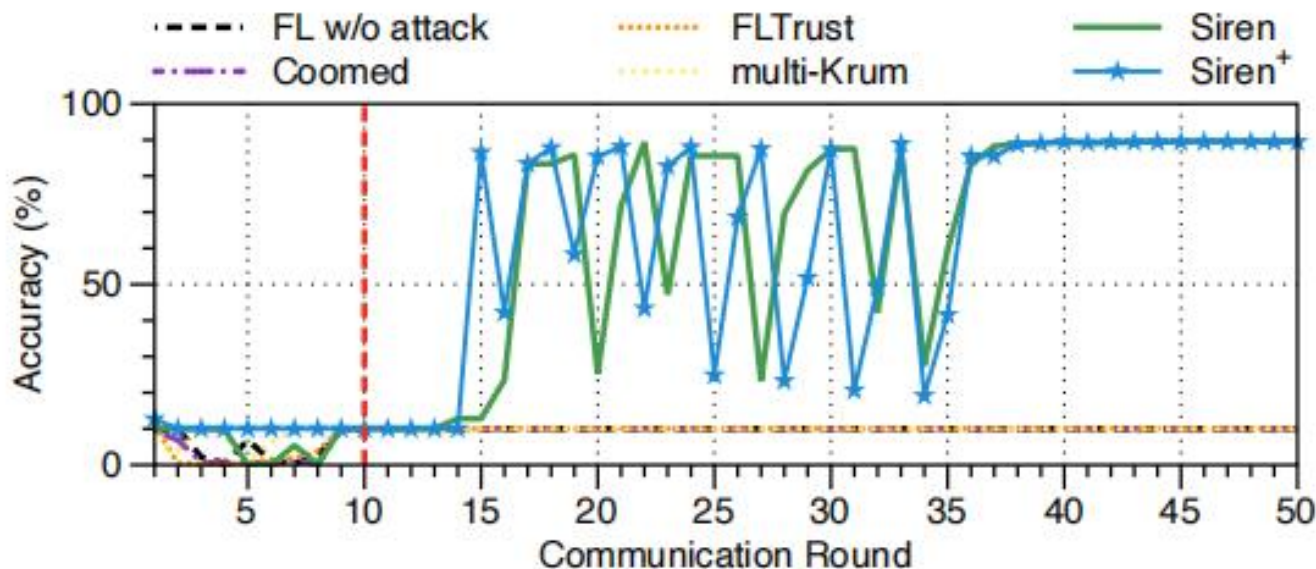


Fig. 7: Training efficiency under adaptive attack when $|K| = 10$.



🔴 Siren+延续了Siren的训练恢复能力

🔴 Siren+特别对Adaptive Attack进行了测试，依旧有着较强的防御效果

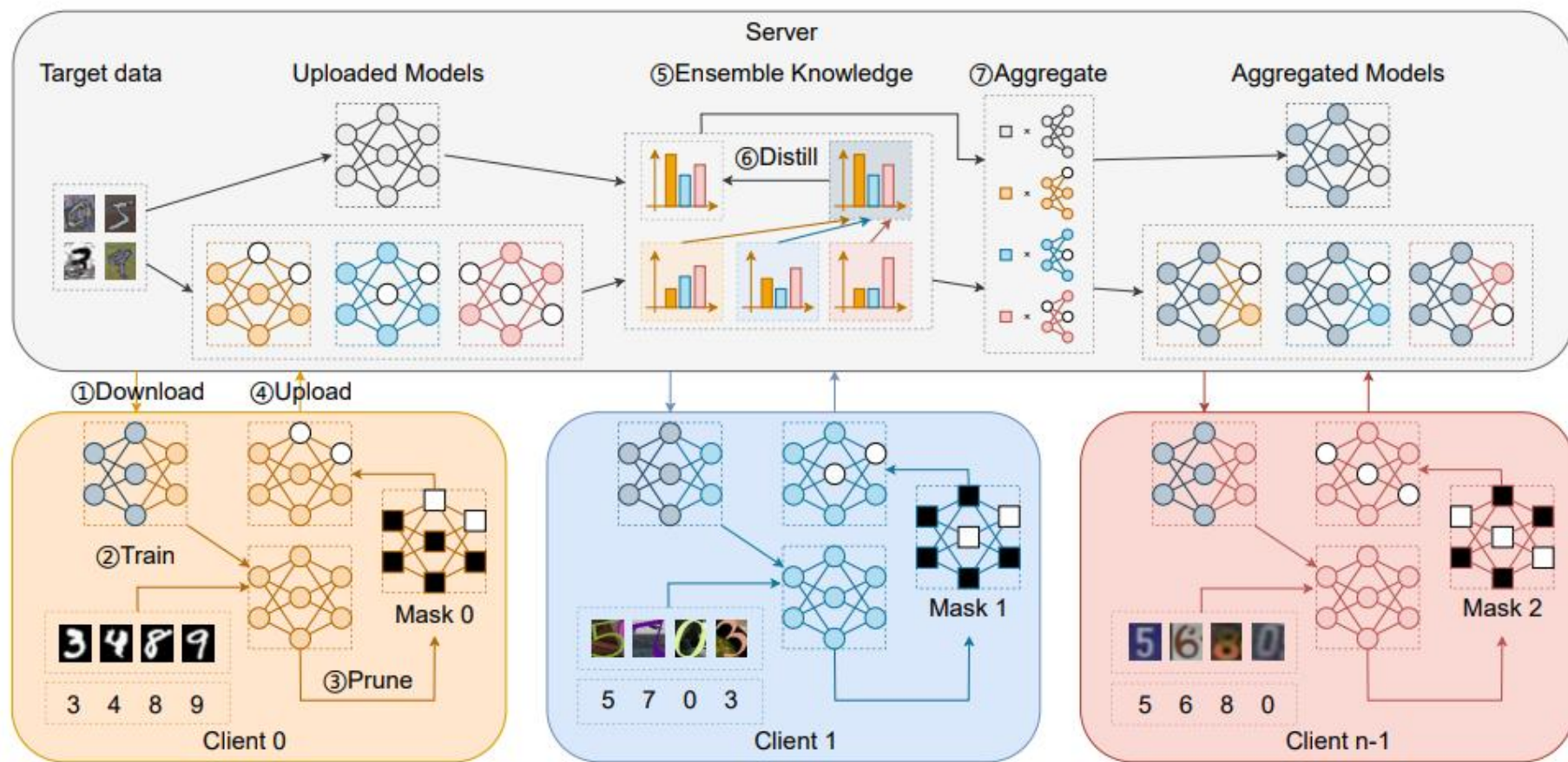


Attack Type	Malicious Client Proportion	Accuracy
None	0%	80.23%
Sign-flipping	20%	76.32%
	40%	74.4%
	80%	63.81%
Label-flipping	20%	76.94%
	40%	75.42%
	80%	64.53%
Adaptive Attack	20%	78.41%
	40%	75.79%
	80%	61.57%

联邦学习效率 + 异构性

跨域联邦学习中，源域模型不一定适用于目标域

标签蒸馏：聚合源域标签，用于训练目标域模型





联邦学习效率 + 异构性

适应性训练批次调整

- 解决系统异构性

减少客户端训练时间差异减少等待时间

$$B_i^{t+1} = B_i^t * \frac{\tau^{t+1}}{\tau_i^t} \quad \tau^{t+1} = \frac{1}{N} \sum_{i=0}^{n-1} \tau_i^t$$

减少更新步长提高收敛效果

$$B^{t+1} = B^t * \frac{t}{t+1}$$

$$B_i^{t+1} = B_i^t * \frac{\frac{1}{N} \sum_{i=0}^{n-1} \tau_i^t}{\tau_i^t} * \frac{t_{max} - t}{t_{max}}$$

模型剪枝

- 减少网络传输量

$$m_{ik}^t = I(|\hat{\Theta}_{ik}^t| \geq h_i^t) \longrightarrow \Theta_i'^t = \hat{\Theta}_i^t \odot M_i^t$$

基于比例的剪枝阈值设计

$$h_i^t = \min_k(|\hat{\Theta}_{ik}^t|) + p * [\max_k(|\hat{\Theta}_{ik}^t|) - \min_k(|\hat{\Theta}_{ik}^t|)]$$

基于分位数的剪枝阈值设计

$$h_i^t = \text{quantile}(|\hat{\Theta}_{ik}^t|)$$

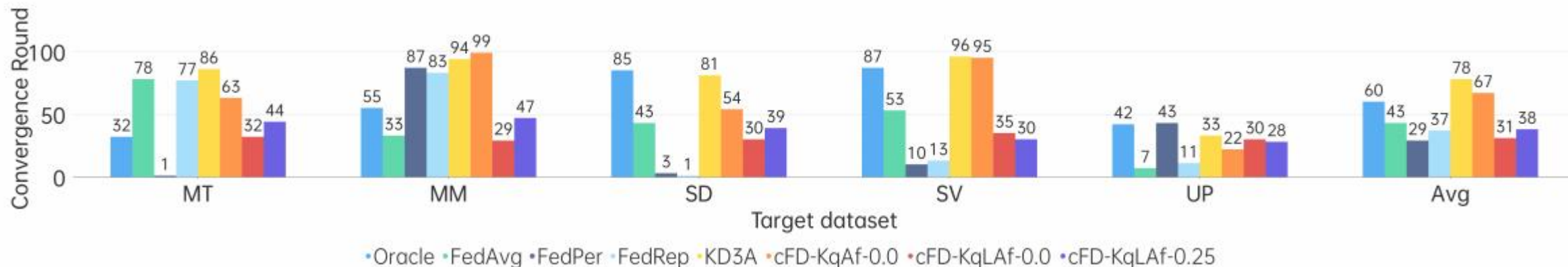


联邦学习效率 + 异构性

- ☉ cFedDT能够在跨域联邦学习的场景下从源域学习到更准确的知识，为目标域构造更加准确的pesudo-label
- ☉ 同时其收敛效率在大部分情况下比其他SOTA算法高

TABLE II
THE TARGET ACCURACIES FOR SOTA ALGORITHMS.

Algorithm	Target Dataset					Avg.
	MT	MM	SD	SV	UP	
Oracle	0.996	0.980	0.990	0.927	0.994	0.977
FedAvg	0.993	0.739	0.902	0.822	0.969	0.885
FedPer	0.119	0.116	0.095	0.098	0.061	0.098
FedRep	0.074	0.122	0.095	0.104	0.201	0.119
KD3A	0.993	0.880	0.913	0.885	0.978	0.932
cFD-KqAf-0.0	0.994	0.886	0.915	0.888	0.977	0.932
cFD-KqLAf-0.0	0.991	0.978	0.911	0.884	0.974	0.948
cFD-KqLAf-0.25	0.993	0.958	0.916	0.876	0.974	0.943





联邦学习效率 + 异构性

- 批次量调整时，减少客户端用时差距和批次量递减均对准确率有正面效果
- 合并使用两种批次调整方案能够有效的提高收敛效率，单一任意一种都没有办法做到

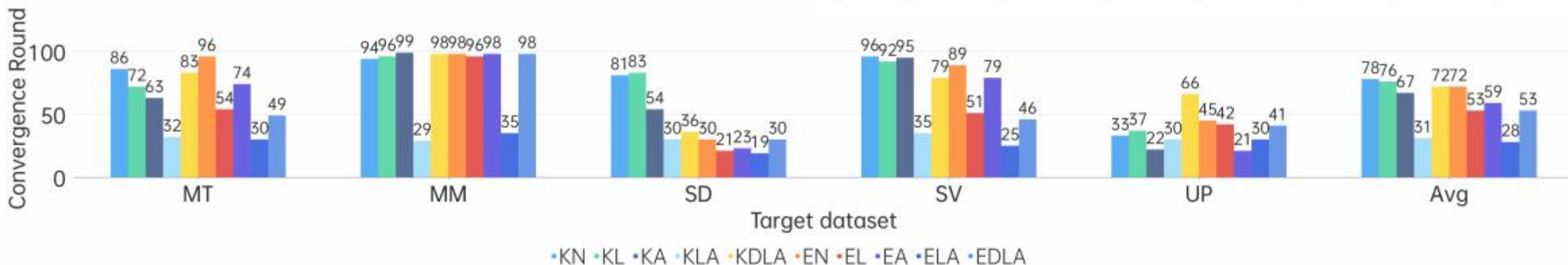


TABLE III
THE TARGET ACCURACIES FOR DIFFERENT BATCH ADJUSTMENT METHODS.

KE	BA	Target Dataset					Avg.
		MT	MM	SD	SV	UP	
K	N	0.993	0.880	0.913	0.885	0.978	0.930
	L	0.994	0.914	0.912	0.883	0.980	0.937
	A	0.994	0.886	0.915	0.888	0.977	0.932
	LA	0.991	0.978	0.911	0.884	0.974	0.948
	DLA	0.993	0.914	0.913	0.880	0.980	0.936
E	N	0.992	0.895	0.884	0.864	0.978	0.923
	L	0.989	0.913	0.880	0.860	0.981	0.925
	A	0.992	0.901	0.883	0.860	0.979	0.923
	LA	0.989	0.944	0.870	0.844	0.973	0.924
	DLA	0.992	0.914	0.876	0.857	0.977	0.923



联邦学习效率 + 异构性

- 采用基于分位数的剪枝阈值设计方案相对而言对准确率的负面影响较小
- 压缩率不同对收敛速度基本没有影响
- 但能够将原本模型参数量压缩至0.05%

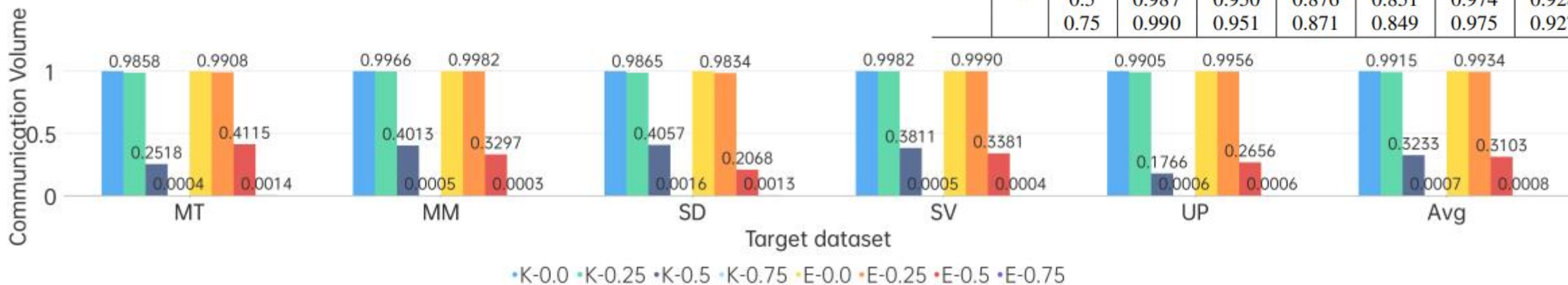


TABLE IV
THE TARGET ACCURACIES FOR DIFFERENT PRUNING TYPES AND PRUNING RATIOS.

KE	PT	p	Target Dataset					Avg.
			MT	MM	SD	SV	UP	
K	r	0.0	0.991	0.959	0.909	0.878	0.972	0.942
		0.25	0.992	0.961	0.914	0.878	0.977	0.944
		0.5	0.991	0.949	0.912	0.879	0.977	0.942
		0.75	0.993	0.954	0.912	0.881	0.975	0.943
	q	0.0	0.991	0.978	0.884	0.911	0.974	0.948
		0.25	0.993	0.958	0.916	0.876	0.974	0.943
		0.5	0.992	0.956	0.912	0.875	0.973	0.942
		0.75	0.991	0.951	0.905	0.876	0.973	0.939
E	r	0.0	0.988	0.941	0.875	0.852	0.977	0.927
		0.25	0.990	0.946	0.875	0.850	0.974	0.927
		0.5	0.986	0.933	0.866	0.850	0.974	0.922
		0.75	0.990	0.951	0.873	0.846	0.971	0.926
	q	0.0	0.989	0.944	0.870	0.844	0.973	0.924
		0.25	0.988	0.946	0.875	0.839	0.970	0.924
		0.5	0.987	0.950	0.876	0.851	0.974	0.928
		0.75	0.990	0.951	0.871	0.849	0.975	0.927



联邦学习效率 + 异构性

- 聚合部分参数（特征提取or分类器）并不能提高收敛后的准确率
- 并且对整体的收敛速度并没有太多影响

TABLE V
THE TARGET ACCURACIES FOR DIFFERENT AGGREGATION TYPES.

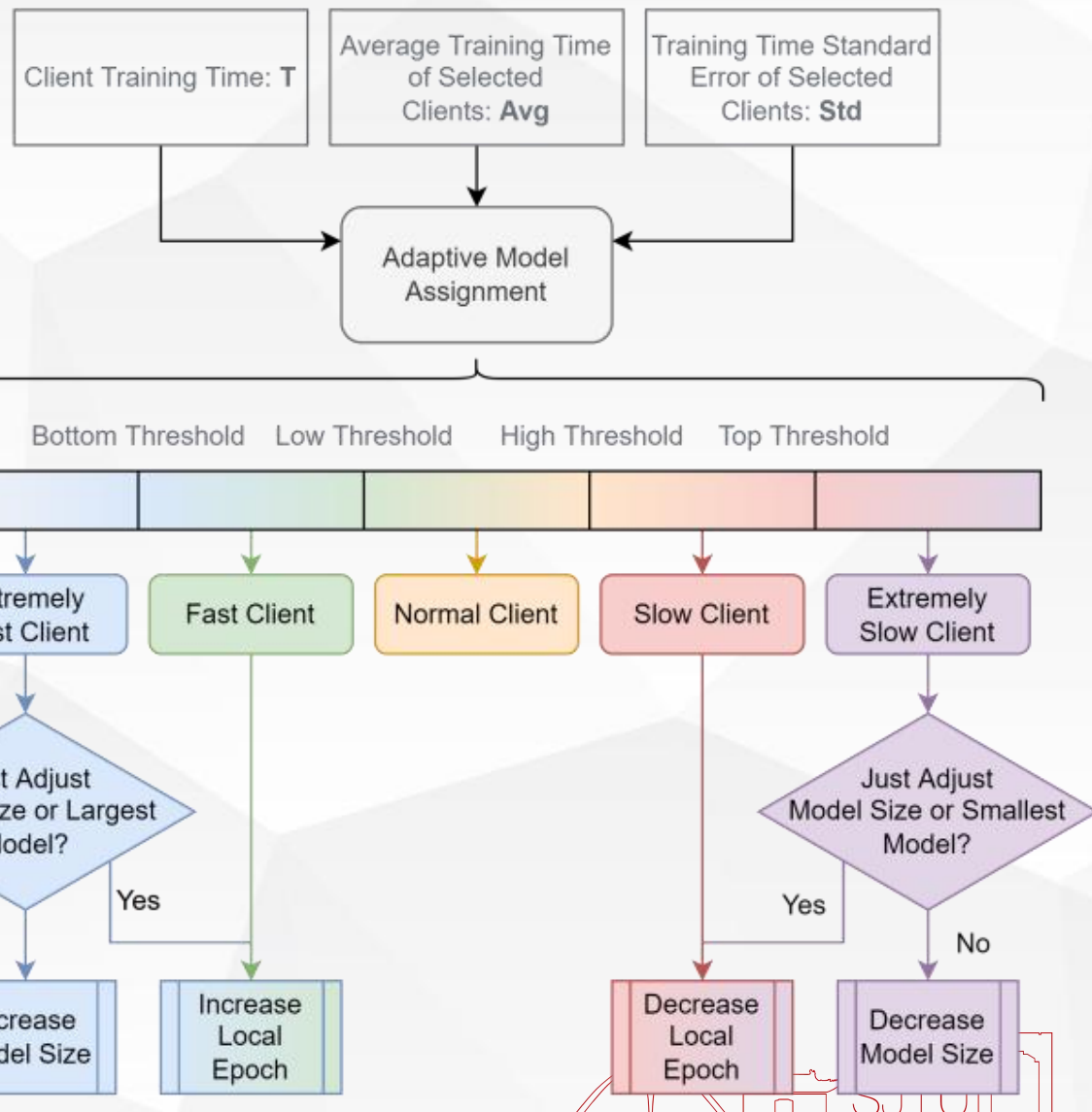
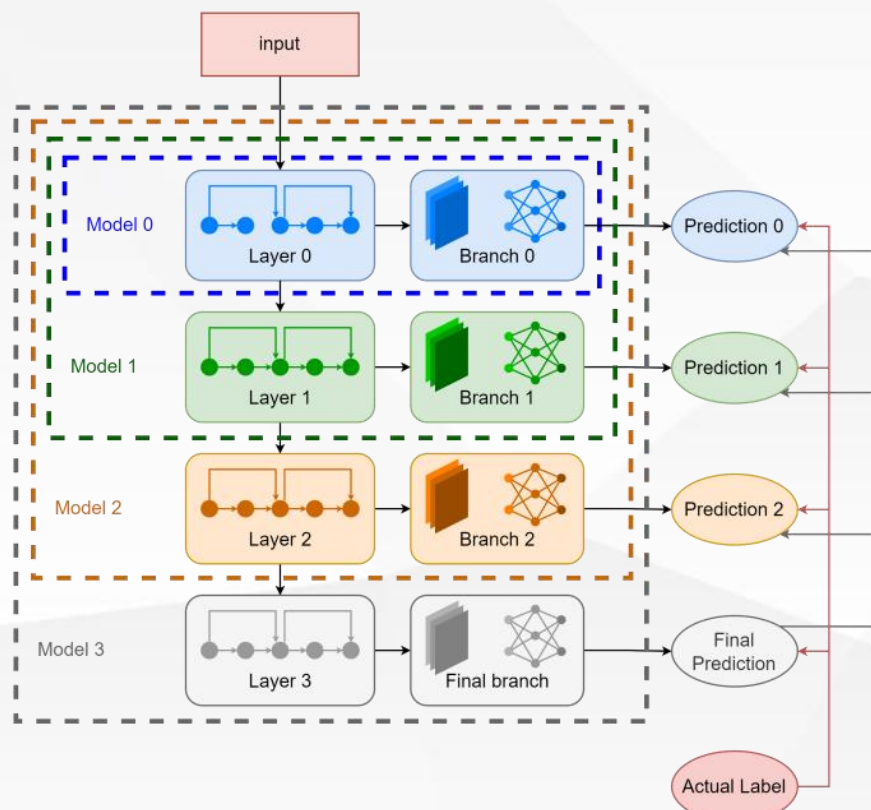
KE	AT	Target Dataset					Avg.
		MT	MM	SD	SV	UP	
K	f	0.991	0.959	0.909	0.878	0.972	0.942
	c	0.993	0.950	0.910	0.884	0.973	0.942
	fc	0.992	0.950	0.914	0.882	0.976	0.943
E	f	0.988	0.941	0.875	0.852	0.977	0.927
	c	0.988	0.951	0.871	0.849	0.966	0.925
	fc	0.986	0.942	0.877	0.851	0.969	0.925





联邦学习效率 + 异构性

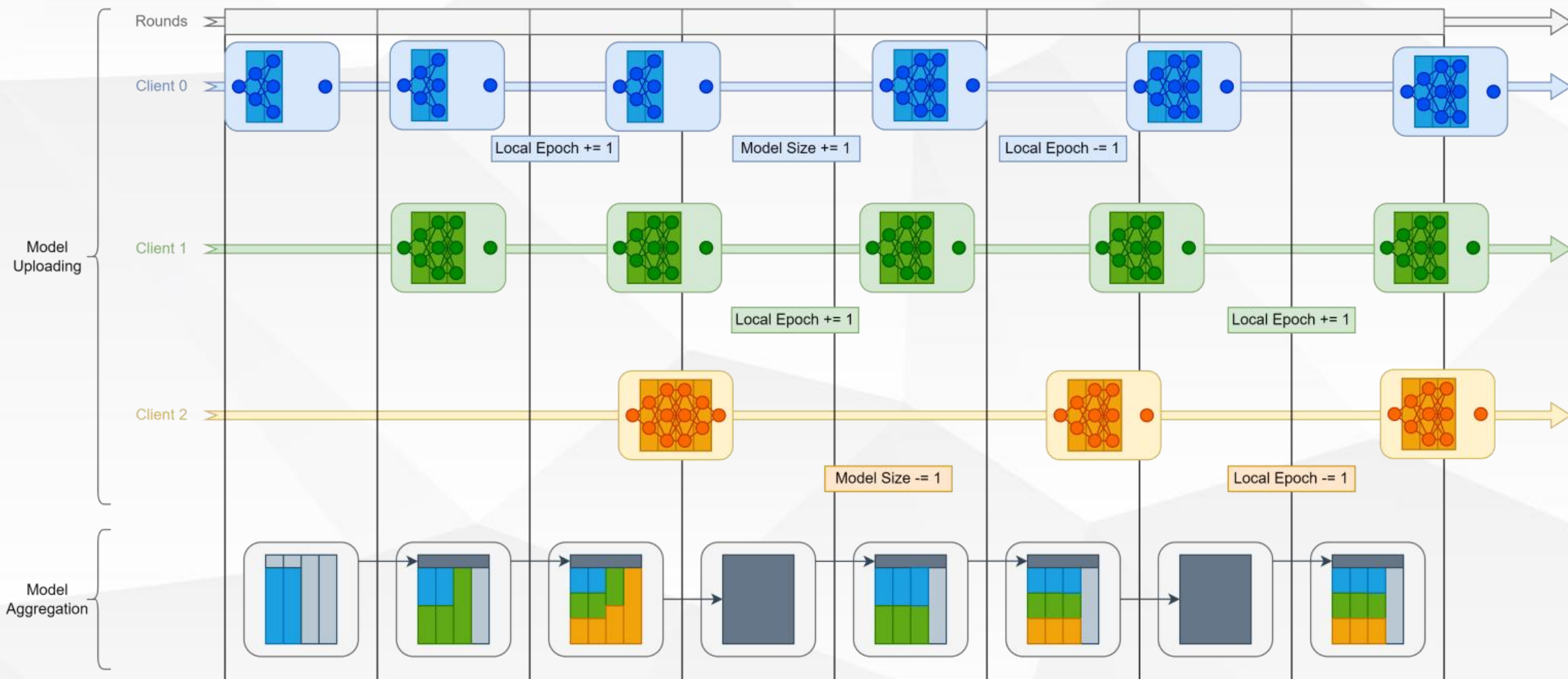
- 采用自蒸馏的方式，构造多出口神经网络模型
- 根据各客户端训练时间区别，调整各客户端的模型大小，本地训练轮次





联邦学习效率 + 异构性

采用回合制聚合的方式进一步减少通信开销，处理极端设备





收敛速度更快

每回合训练用时更少

各客户端训练用时差距更小

总参数量变小

总计算量变小

内存读写量变小

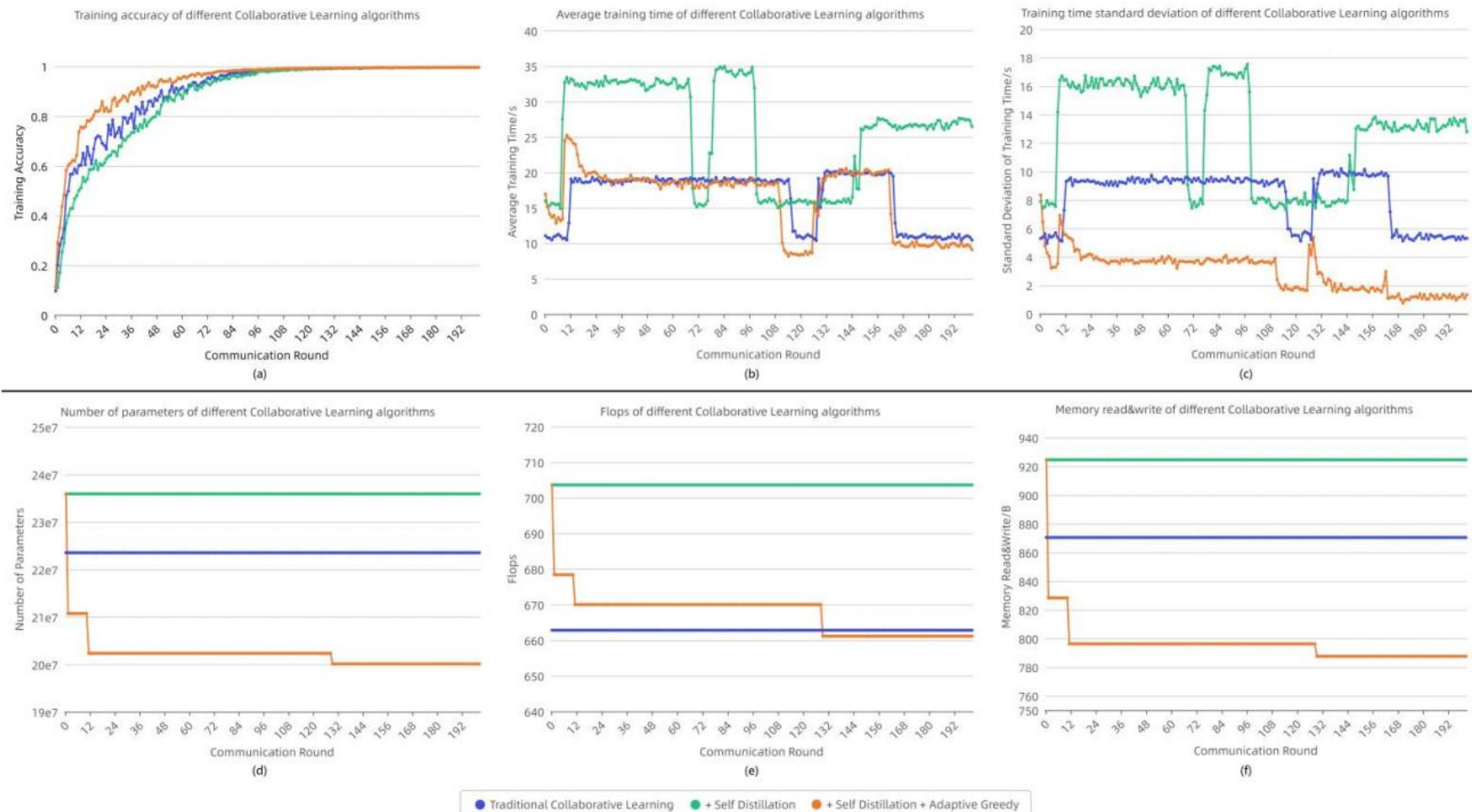


Fig.5 The training curve of different metrics in different communication rounds for the three algorithms. (a) Training accuracy. (b) Average training time. (c) Training time standard deviations. (d) The Number of parameters. (e) Training FLOPS. (f) Memory read&write volume.



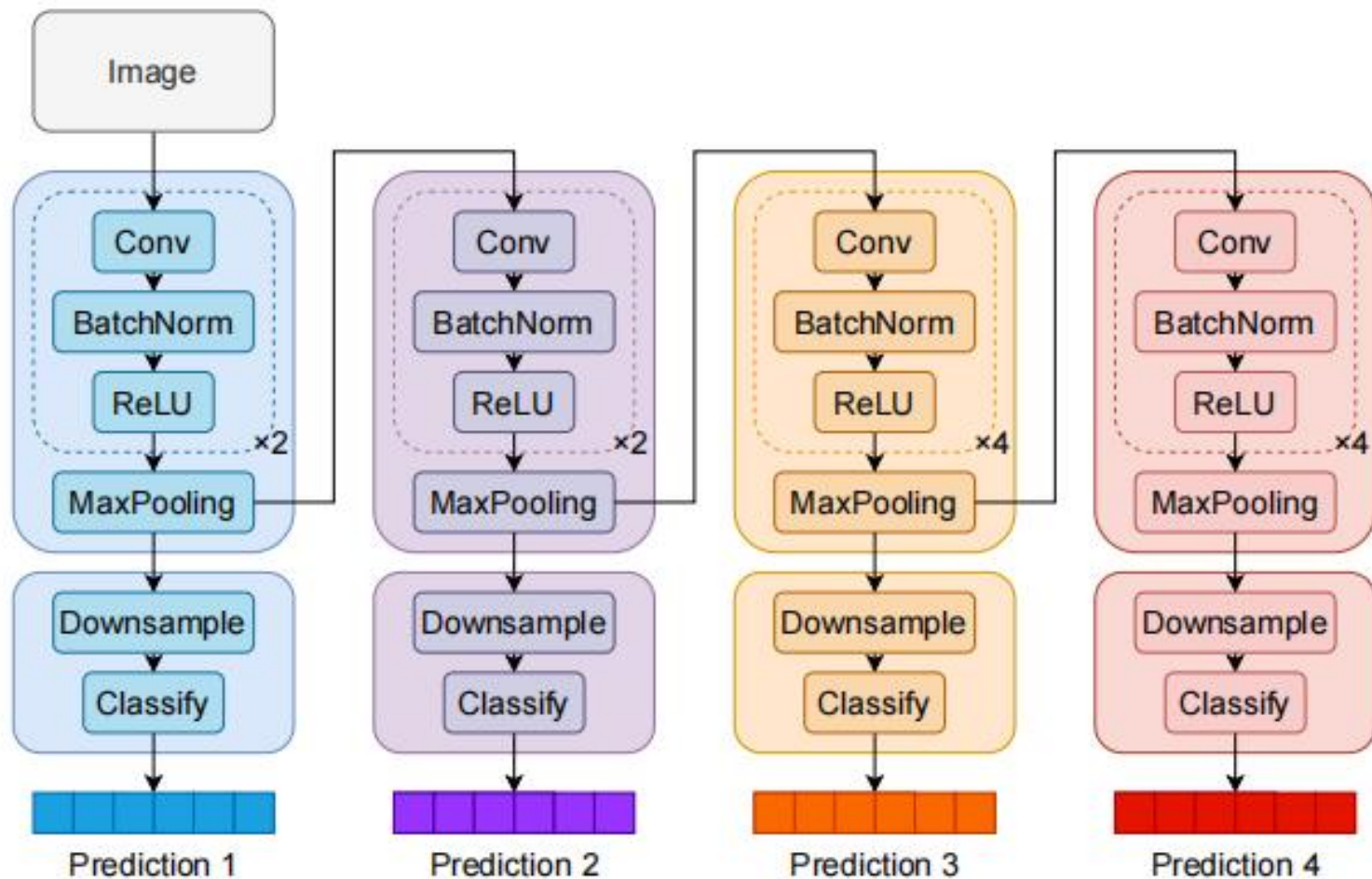
联邦学习效率 + 异构性

考虑到不同客户端存储计算能力不同，不一定能够放下所有模型

采用了多出口的神经网络模型

能够给出多种大小的模型供不同计算存储能力的设备使用

不同大小的模型对不同大小的目标物体检测能力不同



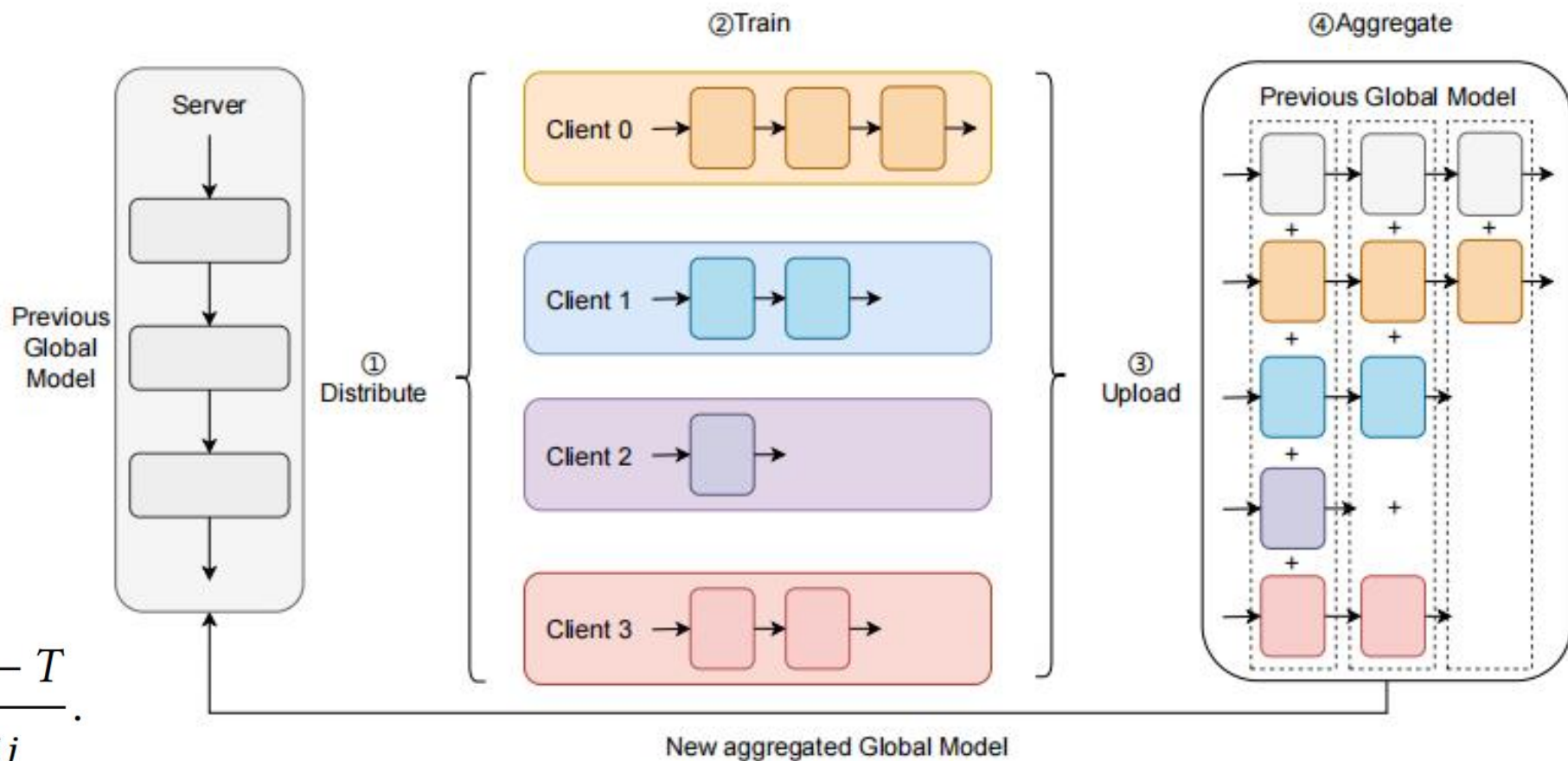


联邦学习效率 + 异构性

④考虑到模型不同，
需要设计不同部分的
聚合权重保障收
敛性

④考虑到系统异构
性，采用逼近式的
训练**批次量**调整

$$B_{ij}^{t+1} = B_{ij}^t - \gamma * B_{ij}^t * \frac{\tau_{ij} - T}{\tau_{ij}}.$$





能够在拥有和原本FedAvg等算法相当的准确率

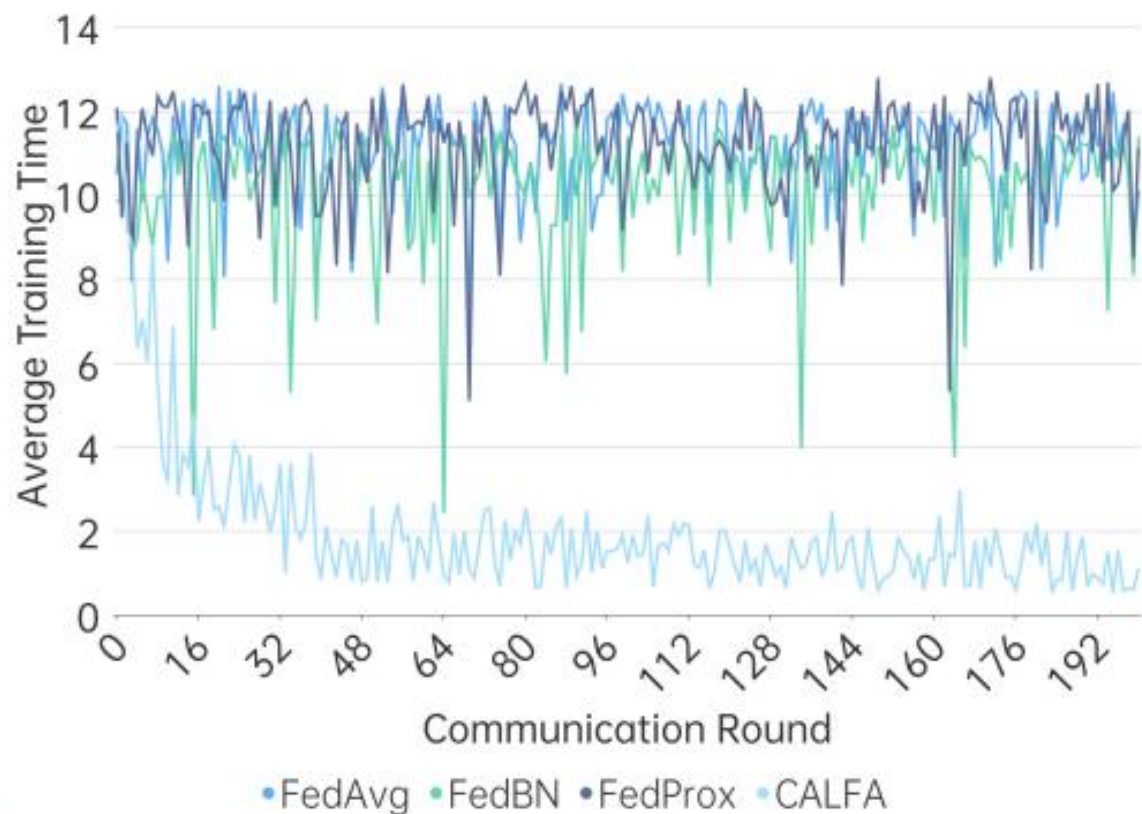
Algorithm	Accuracy	AUC	Conv.	Avg. Time/s	Std. Time/s	App.
Local	0.2913±0.0073	0.5807±0.0053	126±36	11.2562±4.8366	4.5228±1.4563	False
FedAvg[27]	0.8027 ±0.0219	0.9436±0.0104	101±52	12.5468±2.0883	5.0903±0.6760	False
FedProx[18]	0.8027 ±0.0201	0.9477 ±0.0057	121±41	12.1191±4.2455	4.5073±1.1736	False
FedBN[19]	0.7900±0.0180	0.9397±0.0088	75 ±20	11.9506±2.5389	5.0324±0.3404	False
FedPer[1]	0.3288±0.0000	0.6221±0.0192	36±27	12.3796±2.2522	4.8215±0.3423	False
FedRep[5]	0.3288±0.0000	0.6209±0.0189	28±5	15.6082±4.8483	6.0099±1.2850	False
APFL[6]	0.2320±0.0162	0.5086±0.0280	29±58	21.8305±10.0183	7.3776±3.4201	False
FedPHP[20]	0.2265±0.0275	0.5074±0.0333	9±16	11.7139±4.1097	4.8673±1.4517	False
FedMTL[34]	0.2941±0.0074	0.5594±0.0113	28±21	12.4020±2.9406	4.7836±0.5254	False
Ditto[17]	0.2904±0.0062	0.5552±0.0111	43±37	9.4509±3.3148	4.3855±0.6869	False
FedFomo[41]	0.3251±0.0018	0.6339±0.0015	179±31	8.7981±1.4074	4.0164±0.7786	False
FedProto[36]	0.3142±0.0124	0.6123±0.0034	164±106	11.5173±1.2865	4.6279±0.6379	False
CALFA w/o A_1	0.7945±0.0238	0.9472±0.0099	103±41	3.4714 ±1.2558	1.6892±0.0920	False
CALFA w/o A_2	0.8000±0.0137	0.9456±0.0068	94±42	7.5063±3.1591	3.2267±2.5267	True
CALFA	0.8018±0.0249	0.9416±0.0057	131±49	5.7313±3.4893	0.9158 ±0.8337	True



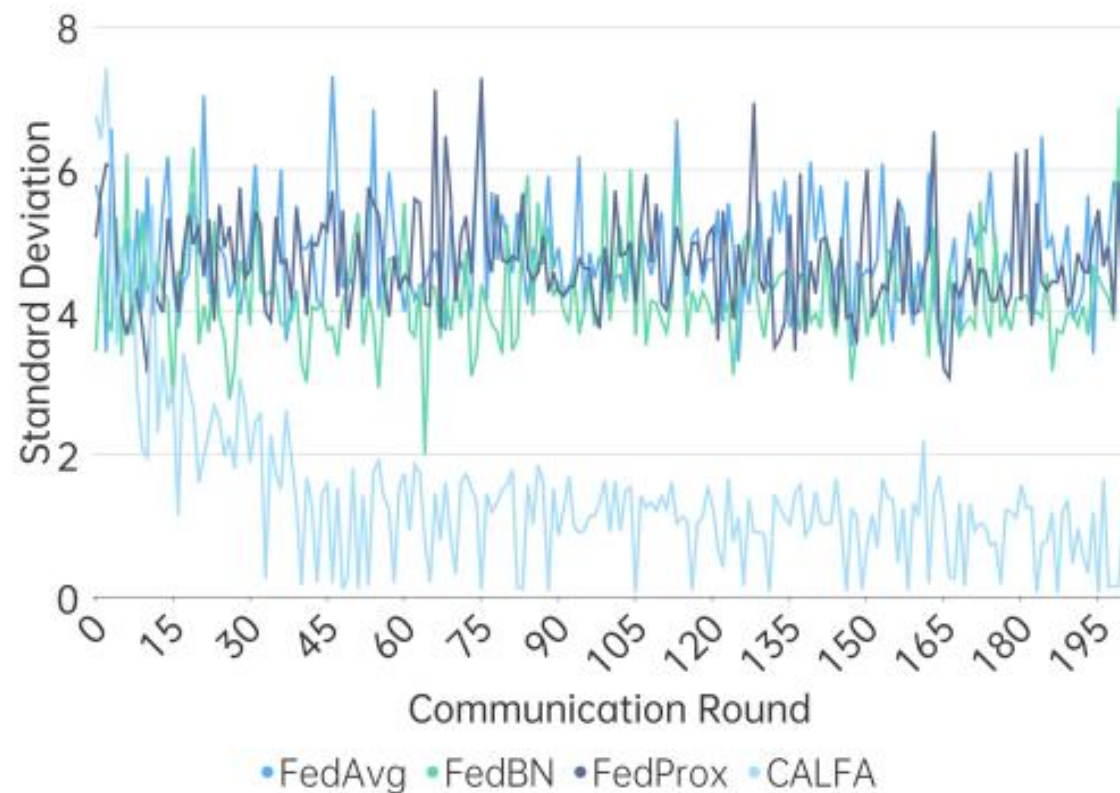
联邦学习效率 + 异构性

能够在保障准确率的情况下，减少训练用时和系统异构性

Average training time of SOTA algorithms over Agriculture crop images



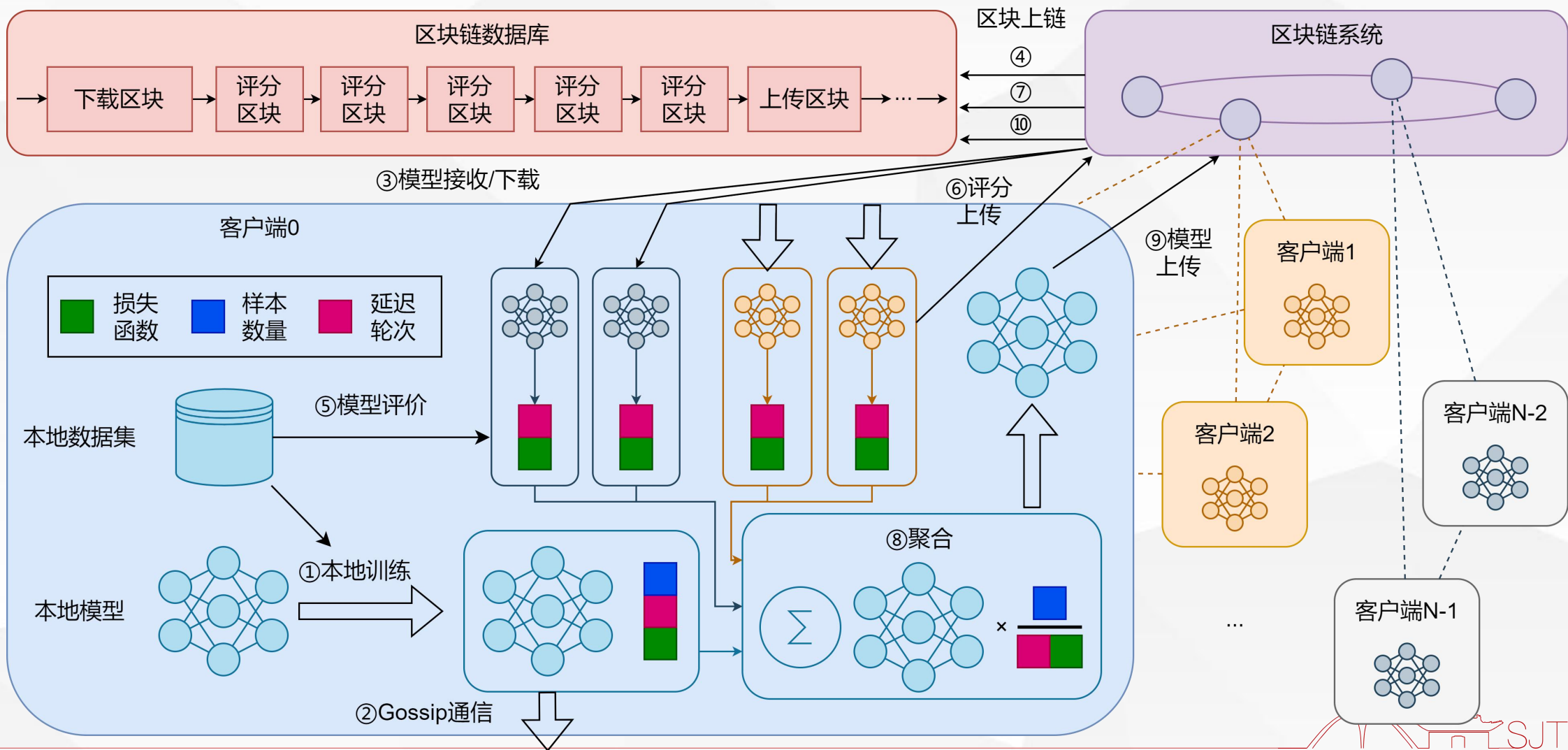
Standard deviation of training time of SOTA algorithms over Agriculture crop images





联邦学习效率 + 安全 + 异构性

半中心化：可信客户端-可信客户端（去中心） + 不可信客户端-区块链系统（中心）





联邦学习效率 + 安全 + 异构性

基于损失函数的聚合权重调整方案

$$w_{1,j}^t = \frac{1}{L(\theta_j^{t_j} | D_i)}, w_{1,k}^t = \frac{1}{L(\theta_k^{t_k} | D_i)}$$

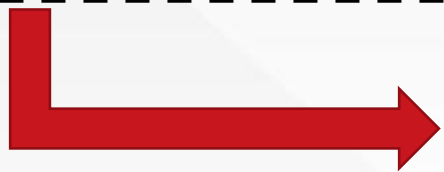
基于延迟轮次的过时模型补偿方案

$$w_{2,j}^t = \begin{cases} e^{(t_j-t)}, & t_j < t \\ 1, & t_j \geq t \end{cases}, w_{2,k}^t = \begin{cases} e^{(t_k-t)}, & t_k < t \\ 1, & t_k \geq t \end{cases}$$

原始权重

$$w_i^t = \frac{N_i}{\sum_{i=0}^{N-1} N_i}$$

最终的聚合权重

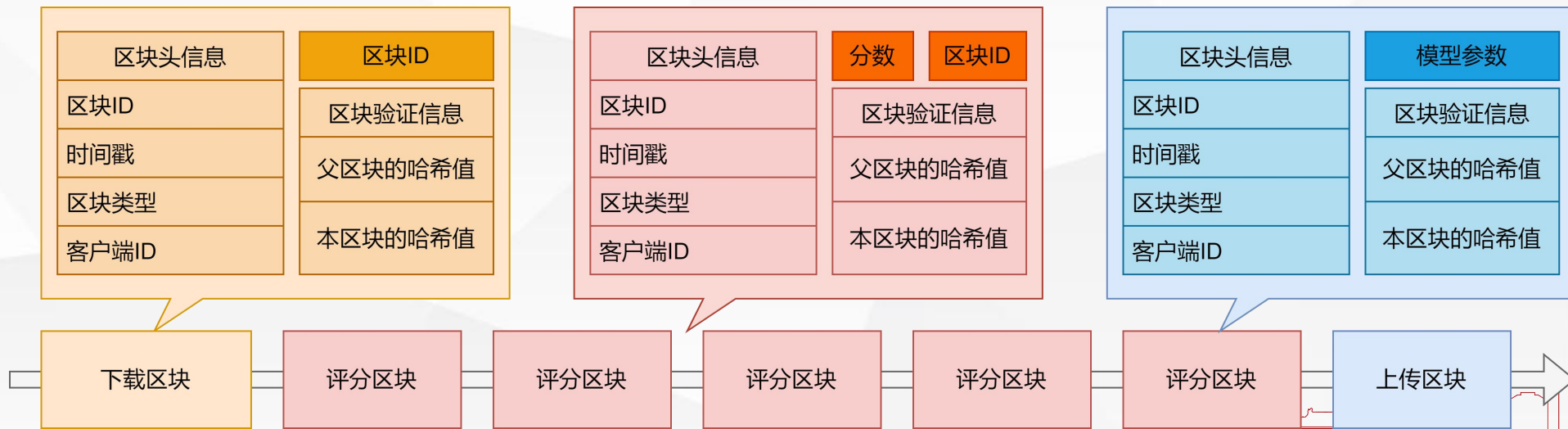


$$\theta_i^t = \frac{w_i^t w_{1,i}^t}{w^t} \tilde{\theta}_i^{t-1} + \sum_{C_j \in C_i^{neigh}} \frac{w_j^t w_{1,j}^t w_{2,j}^t}{w^t} \tilde{\theta}_j^{t_j} + \sum_{C_k \notin C_i^{neigh}} \frac{w_k^t w_{1,k}^t w_{2,k}^t}{w^t} \tilde{\theta}_k^{t_k}$$



区块设计

- 下载区块
- 评分区块
- 上传区块





联邦学习效率 + 安全 + 异构性

表 2 本文框架与其他联邦学习框架在 FMNIST-DIR 下的实验结果

算法框架	测试准确率	测试 AUC	平均训练时间(s)	设备计算时间比例(%)
FedAvg	0.7995 ± 0.0024	0.9666 ± 0.0245	1.3887 ± 0.0600	35.64 ± 3.77
FedProx	0.7993 ± 0.0022	0.9666 ± 0.0246	1.5989 ± 0.0752	36.27 ± 3.39
FedBN	0.7994 ± 0.0023	0.9666 ± 0.0245	1.6163 ± 0.0395	33.21 ± 1.74
FedPer	0.9735 ± 0.0001	0.9967 ± 0.0057	1.4584 ± 0.0864	26.43 ± 1.84
FedRep	0.9743 ± 0.0005	0.9974 ± 0.0061	2.2325 ± 0.1026	38.73 ± 4.17
FedBABU	0.7685 ± 0.0070	0.9950 ± 0.0090	1.4313 ± 0.0733	35.48 ± 3.38
APFL	0.9720 ± 0.0002	0.9976 ± 0.0077	3.6855 ± 0.1621	29.11 ± 1.21
FedPHP	0.0919 ± 0.0185	0.5072 ± 0.2313	3.9221 ± 0.3462	36.44 ± 1.55
Ditto	0.9714 ± 0.0005	0.9986 ± 0.0103	3.5450 ± 0.2870	30.81 ± 1.29
FedFomo	0.9719 ± 0.0004	0.9971 ± 0.0179	2.0139 ± 0.2788	38.14 ± 1.94
FedAMP	0.9720 ± 0.0006	0.9971 ± 0.0114	1.6826 ± 0.0546	29.34 ± 2.52
APPLE	0.9638 ± 0.0006	0.9907 ± 0.0206	30.1721 ± 13.1385	36.25 ± 4.49
FedAsync	0.8352 ± 0.0372	0.9482 ± 0.1870	1.4260 ± 0.0749	100.00 ± 0.00
Ours	0.8901 ± 0.0092	0.9760 ± 0.1303	1.8396 ± 0.0213	100.00 ± 0.00
Ours - delay	0.8749 ± 0.0070	0.9714 ± 0.1291	1.6460 ± 0.0517	100.00 ± 0.00
Ours - delay -loss	0.6595 ± 0.0772	0.8994 ± 0.2153	1.4766 ± 0.0417	100.00 ± 0.00



Q&A

饮水思源 爱国荣校